

# The DFT, the DTFT and the FFT

A complete from-scratch study guide

[Lecture 1](#) · [Lecture 2](#) · [Fast Fourier Transform](#)

**What this document is.** A direct replacement for the three lecture decks (*DTFT and DFT*, *DFT Properties and Applications*, *The Fast Fourier Transform*). Every slide, every one-line bullet and every figure in those decks is covered here — but expanded into full reasoning, with worked numbers.

**What it assumes.** Only that you have met the continuous-time Fourier series and Fourier transform. Everything discrete is built from zero.

**Also inside.** A condensed revision sheet (Part IV), a step-by-step playbook for every recurring exam question pattern (Part V), and fully solved past questions plus extra practice (Part VI).

# Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| How to use this guide                                          | 5         |
| <b>I Building the DFT and the DTFT</b>                         | <b>7</b>  |
| <b>1 From Fourier series to the DFT</b>                        | <b>8</b>  |
| 1.1 Where we are, and what we are about to build               | 8         |
| 1.2 Recap: what the CTFS actually says                         | 9         |
| 1.2.1 The winding picture — why the integral finds frequencies | 9         |
| 1.3 Step 1: sample the periodic signal                         | 10        |
| 1.3.1 Writing the samples as an impulse train                  | 10        |
| 1.4 Step 2: the integral collapses into a sum                  | 10        |
| 1.5 Step 3: only $N$ of the coefficients are distinct          | 11        |
| 1.6 Reading the bins: where did the negative frequencies go?   | 12        |
| 1.7 Step 4: getting the samples back — the inverse             | 13        |
| 1.7.1 Why the CTFS synthesis formula does not work             | 13        |
| 1.7.2 The orthogonality trick                                  | 13        |
| 1.7.3 The discrete inverse                                     | 14        |
| 1.8 The Discrete Fourier Series (DFS)                          | 15        |
| 1.8.1 The winding picture again, now for samples               | 15        |
| 1.9 The Discrete Fourier Transform (DFT)                       | 15        |
| 1.9.1 The three bridges                                        | 16        |
| 1.10 The scaffolding comes down                                | 17        |
| <b>2 The Discrete-Time Fourier Transform (DTFT)</b>            | <b>18</b> |
| 2.1 Now let the signal be aperiodic                            | 18        |
| 2.2 Compressing the frequency axis: digital frequency          | 18        |
| 2.2.1 The winding picture for an aperiodic signal              | 19        |
| 2.3 The DTFT is $2\pi$ -periodic                               | 20        |
| 2.4 The inverse DTFT                                           | 20        |
| <b>3 The DFT through the DTFT lens</b>                         | <b>22</b> |
| 3.1 The DFT is $N$ samples of the DTFT                         | 22        |
| 3.2 Reading DFT bins, once more                                | 23        |
| 3.3 Zero-padding: same spectrum, finer samples                 | 23        |
| 3.4 A complete numeric example ( $N = 4$ )                     | 25        |
| 3.5 Back to physical frequency, and to hertz                   | 26        |
| 3.5.1 In rad/s                                                 | 26        |
| 3.5.2 In hertz                                                 | 26        |
| 3.6 Part I in one picture: the square, fully derived           | 27        |
| 3.7 Part I summary                                             | 28        |

|            |                                                                  |           |
|------------|------------------------------------------------------------------|-----------|
| <b>II</b>  | <b>DFT properties and circular convolution</b>                   | <b>29</b> |
| <b>4</b>   | <b>The rules of the game</b>                                     | <b>30</b> |
| 4.1        | Everything lives on a clock . . . . .                            | 30        |
| 4.2        | The one proof move . . . . .                                     | 31        |
| 4.3        | Two properties for free . . . . .                                | 31        |
| 4.3.1      | Linearity . . . . .                                              | 31        |
| 4.3.2      | Periodicity of the bins . . . . .                                | 31        |
| 4.4        | Circular time shift . . . . .                                    | 32        |
| 4.5        | Frequency shift (modulation) . . . . .                           | 33        |
| 4.5.1      | The special case $k_0 = N/2$ : multiplying by $(-1)^n$ . . . . . | 33        |
| 4.6        | Time reversal and conjugation . . . . .                          | 34        |
| 4.6.1      | Time reversal . . . . .                                          | 34        |
| 4.6.2      | Conjugation . . . . .                                            | 34        |
| 4.7        | Conjugate symmetry: why real signals fold . . . . .              | 34        |
| 4.8        | Parseval's relation: energy is re-bookkept . . . . .             | 35        |
| 4.9        | The property table . . . . .                                     | 36        |
| <b>5</b>   | <b>Filtering with the DFT</b>                                    | <b>38</b> |
| 5.1        | Why convolution matters . . . . .                                | 38        |
| 5.2        | What multiplying DFTs actually does . . . . .                    | 38        |
| 5.3        | Circular convolution, three ways . . . . .                       | 39        |
| 5.3.1      | Picture 1 — two wheels . . . . .                                 | 39        |
| 5.3.2      | Picture 2 — linear convolution, but the taps wrap . . . . .      | 39        |
| 5.3.3      | Picture 3 — leave the taps alone, extend $x$ instead . . . . .   | 41        |
| 5.4        | Worked example: circular $\neq$ linear . . . . .                 | 42        |
| 5.5        | Zero-padding makes it linear . . . . .                           | 43        |
| 5.6        | The filtering recipe . . . . .                                   | 44        |
| 5.7        | Part II summary . . . . .                                        | 45        |
| <b>III</b> | <b>The Fast Fourier Transform</b>                                | <b>46</b> |
| <b>6</b>   | <b>The need for speed</b>                                        | <b>47</b> |
| 6.1        | What the naive DFT costs . . . . .                               | 47        |
| 6.2        | A brief history . . . . .                                        | 47        |
| 6.3        | The idea: exploit the structure of $W_N$ . . . . .               | 48        |
| <b>7</b>   | <b>Radix-2 Decimation-in-Time (DIT)</b>                          | <b>49</b> |
| 7.1        | The derivation . . . . .                                         | 49        |
| 7.2        | Filling in the upper half — and the free subtraction . . . . .   | 49        |
| 7.3        | Butterfly structures, small to large . . . . .                   | 50        |
| 7.3.1      | $N = 2$ — the fundamental butterfly . . . . .                    | 50        |
| 7.3.2      | $N = 4$ . . . . .                                                | 50        |
| 7.3.3      | $N = 8$ . . . . .                                                | 51        |
| 7.4        | Understanding bit-reversed order . . . . .                       | 53        |
| 7.5        | Pseudocode and complexity . . . . .                              | 53        |
| 7.5.1      | Complexity analysis . . . . .                                    | 54        |
| <b>8</b>   | <b>Radix-2 Decimation-in-Frequency (DIF)</b>                     | <b>56</b> |
| 8.1        | The derivation . . . . .                                         | 56        |
| 8.1.1      | Even outputs, $k = 2m$ . . . . .                                 | 56        |
| 8.1.2      | Odd outputs, $k = 2m + 1$ . . . . .                              | 56        |

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| 8.2       | DIT versus DIF — side by side . . . . .                                | 58        |
| 8.3       | Zero-padding for radix-2 . . . . .                                     | 58        |
| 8.4       | Can we avoid zero-padding? (Beyond radix-2) . . . . .                  | 59        |
| 8.5       | Part III summary . . . . .                                             | 60        |
| <b>IV</b> | <b>Revision sheet</b>                                                  | <b>61</b> |
| <b>9</b>  | <b>Everything on a few pages</b>                                       | <b>62</b> |
| 9.1       | The formula sheet . . . . .                                            | 62        |
| 9.2       | The twiddle values you should know by sight . . . . .                  | 63        |
| 9.3       | Ten checks that catch nearly every arithmetic slip . . . . .           | 64        |
| 9.4       | The intuitions, restated in one line each . . . . .                    | 64        |
| 9.5       | The five traps . . . . .                                               | 64        |
| <b>V</b>  | <b>Common question patterns</b>                                        | <b>65</b> |
| <b>10</b> | <b>The playbook</b>                                                    | <b>66</b> |
|           | Index of patterns . . . . .                                            | 66        |
| 10.1      | P1 — Compute a small DFT by hand . . . . .                             | 66        |
| 10.2      | P2 — Convert bins to frequency and back . . . . .                      | 67        |
| 10.3      | P3 — Hand-run a radix-2 FFT and draw the butterfly diagram . . . . .   | 67        |
| 10.4      | P4 — Interpret a DFT result physically . . . . .                       | 68        |
| 10.5      | P5 — Zero-padding: what changes and what does not . . . . .            | 68        |
| 10.6      | P6 — “What if you break a step of the algorithm?” . . . . .            | 69        |
| 10.7      | P7 — Derive a higher-radix (or generalised) FFT . . . . .              | 69        |
| 10.8      | P8 — Turn a problem into a convolution, then FFT it . . . . .          | 70        |
| 10.9      | P9 — Extend the DFT to more dimensions . . . . .                       | 70        |
| 10.10     | P10 — Compare DFT vs FFT complexity . . . . .                          | 71        |
| 10.11     | P11 — Circular vs linear convolution . . . . .                         | 71        |
| 10.12     | P12 — Get a new DFT from a known one . . . . .                         | 71        |
| 10.13     | P13 — Prove a property from the analysis sum . . . . .                 | 71        |
| 10.14     | P14 — Apply or verify Parseval . . . . .                               | 72        |
| 10.15     | P15 — Deduce signal properties from the spectrum (and back) . . . . .  | 72        |
| 10.16     | P16 — Is this discrete-time signal periodic? . . . . .                 | 72        |
| 10.17     | P17 — Lowest sampling rate for perfect reconstruction . . . . .        | 73        |
| 10.18     | P18 — Map an application task to a signal property . . . . .           | 73        |
| <b>VI</b> | <b>Practice questions with solutions</b>                               | <b>74</b> |
| <b>11</b> | <b>Class Test 4 — July 2025 (past paper)</b>                           | <b>75</b> |
| 11.1      | Question 1 — 4-point radix-2 DIT FFT of an impulse [8] . . . . .       | 75        |
| 11.2      | Question 2 — deriving the radix-4 DIT FFT [8] . . . . .                | 77        |
| 11.3      | Question 3 — polynomial multiplication [8] . . . . .                   | 79        |
| 11.4      | Question 4 — the 2-D DFT [8] . . . . .                                 | 80        |
| 11.5      | Question 5 [4] . . . . .                                               | 82        |
| <b>12</b> | <b>Term Final, January 2025 — on-syllabus parts</b>                    | <b>83</b> |
| 12.1      | Q4(b) — lowest sampling rate for perfect reconstruction [10] . . . . . | 83        |
| 12.2      | Q4(c) — compare DFT and radix-2 FFT complexity [15] . . . . .          | 84        |
| 12.3      | Q5(c) — is the signal periodic? [5] . . . . .                          | 85        |
| 12.4      | Q7(d) — image tasks and signal properties [7] . . . . .                | 86        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>13 Extra practice (written for this guide)</b>                 | <b>88</b> |
| 13.1 E1 — An 8-point DFT of a rectangular pulse . . . . .         | 88        |
| 13.2 E2 — Using the properties . . . . .                          | 89        |
| 13.3 E3 — Circular versus linear convolution . . . . .            | 90        |
| 13.4 E4 — Bins to hertz . . . . .                                 | 90        |
| 13.5 E5 — Parseval and the power spectrum . . . . .               | 91        |
| 13.6 E6 — Filling in a spectrum from conjugate symmetry . . . . . | 91        |
| 13.7 E7 — Prove the modulation property . . . . .                 | 92        |
| 13.8 E8 — An 8-point radix-2 DIT FFT by hand . . . . .            | 92        |
| 13.9 E9 — The same signal by DIF . . . . .                        | 93        |
| 13.10E10 — Bit reversal for $N = 16$ . . . . .                    | 94        |
| 13.11E11 — Three kinds of zero-padding . . . . .                  | 95        |
| 13.12E12 — DTFT, and the bridge to the DFT . . . . .              | 95        |
| 13.13E13 — Choosing a transform size . . . . .                    | 96        |
| 13.14E14 — Filtering via the DFT, end to end . . . . .            | 96        |
| 13.15E15 — Reading a signal off its spectrum . . . . .            | 97        |

# How to use this guide

## The map

The material splits into three lectures, and they are strictly sequential — each one is useless without the one before it.

| Part     | Source deck                | The one question it answers                                                    |
|----------|----------------------------|--------------------------------------------------------------------------------|
| Part I   | Lecture 1 — DTFT and DFT   | <i>What is the Fourier transform of a list of numbers?</i>                     |
| Part II  | Lecture 2 — DFT Properties | <i>What happens to that transform when I shift / flip / multiply the list?</i> |
| Part III | The Fast Fourier Transform | <i>How do I compute it without waiting all day?</i>                            |

## The four objects you must never confuse

Almost every mistake in this topic is a mix-up between these four. Read this table now, and re-read it after Part I — it will mean much more the second time.

| Name      | Time                                             | Frequency                                        | Formula                                                       |
|-----------|--------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------|
| CTFS      | continuous, periodic                             | discrete, infinite                               | $c_k = \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\omega_0 t} dt$    |
| CTFT      | continuous, aperiodic                            | continuous, infinite                             | $X(j\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt$ |
| DTFT      | <b>discrete</b> , aperiodic                      | continuous, <b><math>2\pi</math>-periodic</b>    | $X(e^{j\omega}) = \sum_n x[n] e^{-j\omega n}$                 |
| DFS / DFT | <b>discrete</b> , <b><math>N</math>-periodic</b> | <b>discrete</b> , <b><math>N</math>-periodic</b> | $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$                       |

### The single rule that generates the whole table

**Discrete in one domain  $\iff$  periodic in the other.**

- Sampling in *time* makes the *spectrum* periodic. (CTFT  $\rightarrow$  DTFT: period  $2\pi$ .)
- Sampling in *frequency* makes *time* periodic. (DTFT  $\rightarrow$  DFT: period  $N$ .)

Both halves of this rule are *proved* in Part I. They are not slogans — if you can reproduce both proofs you can rebuild every formula in this guide from scratch.

## How to read it

- **Boxes.** **Navy** boxes are definitions and results you must be able to state. **Teal** boxes are the key takeaways of a section. **Amber** boxes are intuition — the “why it had to be this way”. **Rose** boxes are traps: the specific mistakes that cost marks.
- **Figures** are cropped directly from the lecture slides, so the picture in your head matches the picture the examiner drew.
- **Worked examples** are numbered inside each section and are all verified numerically.
- If you are revising the night before, read **Part IV** (revision sheet) and **Part V** (question patterns) first, and dip back into Parts I–III only where Part IV says something you cannot justify.

### Notation used throughout

- $x[n]$  — a sequence (a list of numbers).  $n$  is an integer index, not a time in seconds.
- $x(t)$  — a continuous-time signal.  $t$  is in seconds.
- $T_s$  — sampling period (seconds per sample);  $f_s = 1/T_s$  — sampling rate (samples/second);  $\omega_s = 2\pi/T_s$  — sampling rate in rad/s.
- $\Omega$  (or  $\omega_{\text{phys}}$ ) — *physical* frequency, rad/s.  $\omega$  — *digital* frequency, radians *per sample*. They are linked by  $\omega = \Omega T_s$ .
- $W_N \triangleq e^{-j2\pi/N}$  — the *twiddle factor*. Note the **minus sign**; it is baked into the symbol.
- $((n))_N \triangleq n \bmod N$  — index read “on the clock”.
- $*$  is linear convolution;  $\circledast$  is circular convolution. They are *not* the same thing.

## PART I

# Building the DFT and the DTFT



# Chapter 1

## From Fourier series to the DFT

### 1.1 Where we are, and what we are about to build

You already know two transforms, and both of them live in *continuous* time:

- The **Continuous-Time Fourier Series (CTFS)** for a *periodic* signal  $x(t)$  with period  $T_0$ . It says: any such signal is a sum of harmonics at  $0, \omega_0, 2\omega_0, 3\omega_0, \dots$  where  $\omega_0 = 2\pi/T_0$ . The recipe for how much of each harmonic is present is the coefficient  $c_k$ .
- The **Continuous-Time Fourier Transform (CTFT)** for an *aperiodic* signal. Here there is no fundamental period, so no special set of harmonics; *every* frequency  $\omega$  gets its own value  $X(j\omega)$ , and the spectrum is a continuous curve.

The whole of Lecture 1 answers one question: **what happens to these two ideas when the signal is no longer a curve but a list of numbers?**



**Figure 1.1:** The four Fourier-type transforms. The left column (continuous time) you already know. Part I builds the top-right box (DFS/DFT); Part II of the lecture builds the bottom-right box (DTFT).

The answer will be two new transforms:

- Sample a *periodic* signal  $\Rightarrow$  the **Discrete Fourier Series (DFS)**, and its unnormalised twin the **Discrete Fourier Transform (DFT)**.
- Sample an *aperiodic* signal  $\Rightarrow$  the **Discrete-Time Fourier Transform (DTFT)**.

And then a punchline: those two are the same object seen from two angles. The DFT is exactly  $N$  evenly spaced samples of the DTFT.

## 1.2 Recap: what the CTFS actually says

Let  $x(t)$  be periodic with period  $T_0$ , i.e.  $x(t + T_0) = x(t)$ , and write  $\omega_0 \triangleq 2\pi/T_0$  for the fundamental frequency in rad/s.

### CTFS pair

$$\begin{aligned} \text{synthesis: } x(t) &= \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t} \\ \text{analysis: } c_k &= \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\omega_0 t} dt \end{aligned}$$

Two habits to lock in now, because *every* transform in this course repeats them:

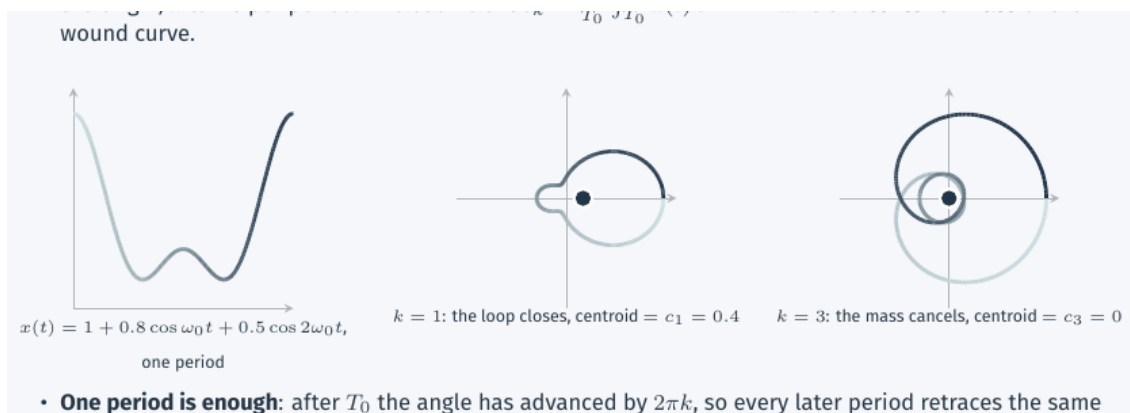
- **Analysis uses**  $e^{-j\cdots}$  (going to frequency, minus sign);
- **synthesis uses**  $e^{+j\cdots}$  (coming back to time, plus sign).

### 1.2.1 The winding picture — why the integral finds frequencies

The formula  $c_k = \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\omega_0 t} dt$  looks like an arbitrary piece of algebra. It is not. Here is what it physically does.

Take the number  $x(t)$  at each instant and treat it as a *radius*. Take  $-k\omega_0 t$  and treat it as an *angle*. Then  $x(t)e^{-jk\omega_0 t}$  is a point in the complex plane at that radius and that angle. As  $t$  runs through one period, the graph of  $x$  gets **wound around the origin**, completing exactly  $k$  turns.

The integral (divided by  $T_0$ ) is then the **average position** of that wound curve — its *centre of mass*.



**Figure 1.2:** Winding one period of  $x(t) = 1 + 0.8 \cos \omega_0 t + 0.5 \cos 2\omega_0 t$ . At  $k = 1$  the loop closes lopsidedly and the centre of mass sits away from the origin:  $c_1 = 0.4$ . At  $k = 3$  the mass is spread evenly around the origin and cancels:  $c_3 = 0$ .

### Why this is the right way to detect a frequency

If the signal genuinely contains the rate  $k\omega_0$ , then winding at that rate makes the signal's peaks always land in the same direction, so the mass piles up on one side and the centroid moves off the origin. If the signal does *not* contain that rate, the peaks land in all directions equally, the mass balances, and the centroid sits at zero.

**One period is enough.** After time  $T_0$  the winding angle has advanced by exactly  $2\pi k$  — a whole number of turns — so the second period retraces the identical loop. Averaging over more periods changes nothing.

**Why the spectrum is a set of lines, not a curve.** Only the rates  $k\omega_0$  close the loop. Any other

rate leaves the loop open, and for a truly periodic signal the mismatch accumulates over the infinitely many periods and cancels to zero. So only integer multiples of  $\omega_0$  survive: a *line spectrum*.

### 1.3 Step 1: sample the periodic signal

Now we make the signal discrete. Sample  $x(t)$  uniformly every  $T_s$  seconds, and choose  $T_s$  so that exactly  $N$  samples fit inside one period:

$$T_0 = NT_s, \quad N \in \mathbb{Z}^+. \quad (1.1)$$

Define the sequence

$$x[n] \triangleq x(nT_s), \quad n \in \mathbb{Z}. \quad (1.2)$$

#### The sequence inherits the periodicity

$$x[n + N] = x((n + N)T_s) = x(nT_s + NT_s) = x(nT_s + T_0) = x(nT_s) = x[n].$$

So  $x[n]$  is automatically  $N$ -periodic.  **$N$  samples describe it completely.**

#### 1.3.1 Writing the samples as an impulse train

We want to reuse the CTFS machinery, and CTFS is defined for functions of continuous  $t$ . So we need a *continuous-time* object that carries exactly the sample values. The standard trick is the **impulse train** (also called a Dirac comb):

$$p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s). \quad (1.3)$$

Multiply the signal by it:

$$x_p(t) = x(t) p(t) = \sum_{n=-\infty}^{\infty} x(nT_s) \delta(t - nT_s) = \sum_{n=-\infty}^{\infty} x[n] \delta(t - nT_s). \quad (1.4)$$

#### What $x_p(t)$ is, in words

$x_p(t)$  is zero everywhere except at the sampling instants  $t = 0, \pm T_s, \pm 2T_s, \dots$ , where it has a spike whose *area* equals the sample value  $x[n]$ . It is not the sequence  $x[n]$  — it is a continuous-time signal that *encodes* the sequence. We only use it as a stepping stone. Once we reach the final formula, it disappears entirely (Section 1.10).

### 1.4 Step 2: the integral collapses into a sum

$x_p(t)$  is still  $T_0$ -periodic (multiplying a  $T_0$ -periodic signal by a  $T_s$ -periodic comb, with  $T_0 = NT_s$ , gives something  $T_0$ -periodic). So it has a CTFS. Compute its coefficients:

$$\begin{aligned} c_k &= \frac{1}{T_0} \int_{T_0} x_p(t) e^{-jk\omega_0 t} dt \\ &= \frac{1}{T_0} \int_{t_0}^{t_0+T_0} \left[ \sum_{n=-\infty}^{\infty} x[n] \delta(t - nT_s) \right] e^{-jk\omega_0 t} dt \\ &= \frac{1}{T_0} \sum_{n=0}^{N-1} x[n] \int_{t_0}^{t_0+T_0} \delta(t - nT_s) e^{-jk\omega_0 t} dt = \frac{1}{T_0} \sum_{n=0}^{N-1} x[n] e^{-jk\omega_0 nT_s}. \end{aligned} \quad (1.5)$$

Two things happened in that calculation and both deserve a sentence.

1. **Why the infinite sum became  $n = 0, \dots, N-1$ .** The integral only covers one period  $[t_0, t_0 + T_0]$ , and that window is  $NT_s$  long, so it contains exactly  $N$  impulses. Choosing  $t_0 = -T_s/2$  makes those be the impulses at  $n = 0, 1, \dots, N-1$ . Every other impulse is outside the window and contributes nothing.
2. **Why the integral vanished.** This is the *sifting property* of the delta:  $\int \delta(t-a)f(t) dt = f(a)$ . The impulse “picks out” the value of the exponential at  $t = nT_s$ . *This is the moment the integral turns into a sum.*

Finally simplify the exponent using  $T_0 = NT_s$ :

$$\omega_0 T_s = \frac{2\pi}{T_0} T_s = \frac{2\pi}{NT_s} T_s = \frac{2\pi}{N}. \quad (1.6)$$

#### The finite analysis sum

$$c_k = \frac{1}{T_0} \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad (1.7)$$

#### The whole point, in one line

The Fourier *integral* became a Fourier *sum* because the signal now only exists at  $N$  instants. There is nothing to integrate *between* the samples.

### 1.5 Step 3: only $N$ of the coefficients are distinct

In the CTFS there are infinitely many coefficients  $c_k, k \in \mathbb{Z}$ . Ask what happens when we increase  $k$  by  $N$ :

$$c_{k+N} = \frac{1}{T_0} \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}(k+N)n} = \frac{1}{T_0} \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \underbrace{e^{-j2\pi n}}_{=1} = c_k. \quad (1.8)$$

The step  $e^{-j2\pi n} = 1$  holds because  $n$  is an *integer*: rotating by a whole number of full turns lands you exactly where you started.

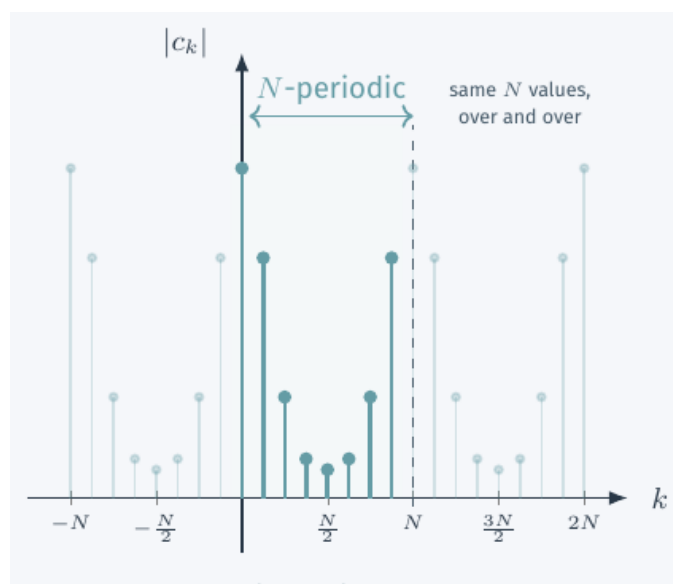
#### Sampling in time made the spectrum periodic

$$c_{k+N} = c_k \quad \text{for every } k. \quad (1.9)$$

There are **exactly  $N$  distinct coefficients**. After that the same  $N$  numbers repeat forever. By convention we keep the window  $k = 0, 1, \dots, N-1$ .

#### Why this is not a mathematical accident

Think of what  $k$  means: the harmonic  $e^{jk\omega_0 t}$  evaluated at the sample instants  $t = nT_s$  is  $e^{jk\frac{2\pi}{N}n}$ . Harmonic  $k$  and harmonic  $k+N$  are different curves in continuous time — but at the  $N$  sample points they pass through *identical values*. The samples cannot tell them apart. So there cannot be more than  $N$  distinguishable frequencies. Sampling has destroyed the ability to see fast oscillations as fast; they masquerade as slow ones. (That is aliasing, seen from the frequency side.)



**Figure 1.3:** The coefficient magnitudes  $|c_k|$  are  $N$ -periodic: the same  $N$  values, over and over. The dark stems are the window  $k = 0, \dots, N - 1$  that we keep.

## 1.6 Reading the bins: where did the negative frequencies go?

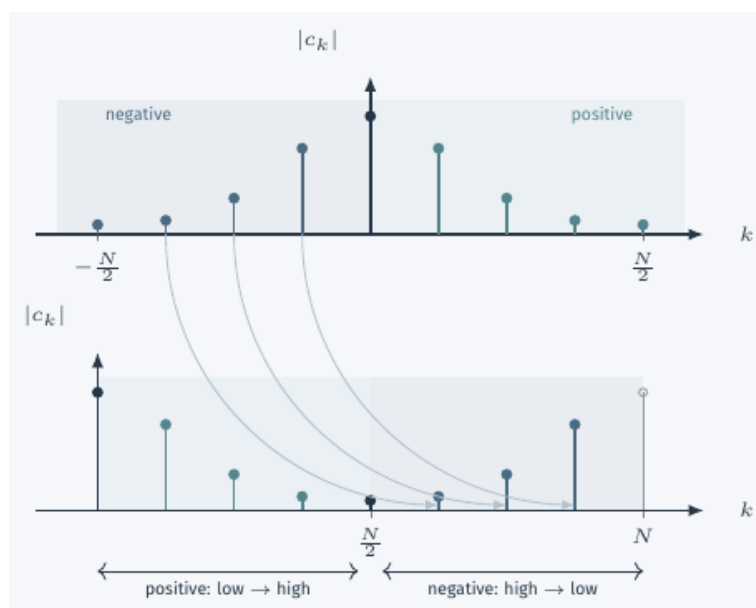
This is the single most-asked-about idea in the whole lecture, so read this section slowly.

The *natural* spectrum of a real signal is centred at  $k = 0$ : the dc term in the middle, positive frequencies to the right, negative frequencies to the left. But we just agreed to keep the window  $k = 0, 1, \dots, N - 1$ , which contains no negative index at all. Where did the negative half go?

It did not go anywhere. Periodicity (1.8) says

$$c_{-k} = c_{N-k} \quad (1.10)$$

so the coefficient that “belongs” at index  $-k$  is sitting, unchanged, at index  $N - k$ . The negative side has been **folded onto the upper bins**.



**Figure 1.4:** Top: the natural, centred spectrum, negative frequencies on the left. Bottom: the same numbers after each negative coefficient is moved to index  $N - k$  — the window  $0, \dots, N - 1$  that the DFT actually reports.

**How to read  $N$  bins — memorise this**

- Bins  $0 \rightarrow \frac{N}{2}$ : **positive** frequencies, running from dc (slowest) up to the highest representable frequency at  $k = N/2$ .
- Bins  $\frac{N}{2} \rightarrow N$ : **negative** frequencies, running from the most negative back up toward dc. High  $\rightarrow$  low in magnitude.
- Therefore a real **lowpass** signal (energy at slow rates) shows peaks at  $k = 0$  and near  $k = N$ , with a dip in the middle at  $k = N/2$ .

**Trap**

“The peak near  $k = N - 1$  means my signal has a very high frequency” — **no**. Bin  $N - 1$  is the *negative-frequency partner of bin 1*, so it means a very *low* frequency. The highest frequency a bin can represent lives in the *middle*, at  $k = N/2$ .

**1.7 Step 4: getting the samples back — the inverse**

We now have  $N$  numbers. Can we rebuild  $x[n]$  from them?

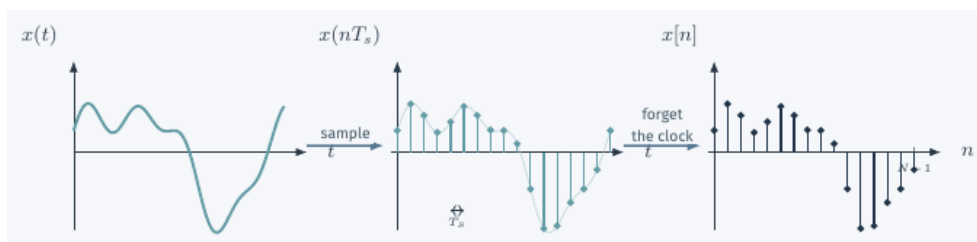
**1.7.1 Why the CTFS synthesis formula does not work**

The obvious move is to plug the  $c_k$  into the CTFS synthesis sum. But

$$\sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t} = x_p(t), \quad (1.11)$$

and  $x_p(t)$  is the *impulse train*, not the sequence. Two problems:

1. It rebuilds spikes in continuous time — an object we only ever invented as scaffolding. We wanted the list of numbers  $x[0], \dots, x[N-1]$ .
2. It uses *infinitely many* terms, while endlessly recycling the same  $N$  coefficient values. That is a waste, and it is not something a computer can do.



**Figure 1.5:** Left of the arrow: what CTFS synthesis actually reconstructs (impulses). Right: what we want (the sequence). We need a purely discrete synthesis formula.

So: keep the pattern (*analysis has  $e^{-j}$ , synthesis has  $e^{+j}$* ) but build a *finite* synthesis sum that uses only the  $N$  distinct coefficients.

**1.7.2 The orthogonality trick**

Write the analysis result with the  $1/T_0$  cleared out of the way, i.e. work with the combination  $T_0 c_k$ :

$$T_0 c_k = \sum_{m=0}^{N-1} x[m] e^{-j\frac{2\pi}{N} km}, \quad k = 0, 1, \dots, N-1. \quad (1.12)$$

Multiply both sides by  $e^{+j\frac{2\pi}{N}kn}$  and sum over the  $N$  values of  $k$ :

$$\begin{aligned}\sum_{k=0}^{N-1} (T_0 c_k) e^{j\frac{2\pi}{N}kn} &= \sum_{k=0}^{N-1} \left( \sum_{m=0}^{N-1} x[m] e^{-j\frac{2\pi}{N}km} \right) e^{j\frac{2\pi}{N}kn} \\ &= \sum_{m=0}^{N-1} x[m] \underbrace{\left( \sum_{k=0}^{N-1} e^{j\frac{2\pi}{N}k(n-m)} \right)}_{\text{call this } S(n-m)}.\end{aligned}\quad (1.13)$$

(We simply swapped the order of two finite sums, which is always allowed.)

Now evaluate the inner sum  $S(r)$  with  $r = n - m$ . It is a geometric series with ratio  $q = e^{j\frac{2\pi}{N}r}$ :

**Case  $r \neq 0$**  (i.e.  $n \neq m$ , and note  $|r| < N$  so  $q \neq 1$ ):

$$S(r) = \sum_{k=0}^{N-1} q^k = \frac{1 - q^N}{1 - q} = \frac{1 - e^{j2\pi r}}{1 - e^{j\frac{2\pi}{N}r}} = \frac{1 - 1}{1 - e^{j\frac{2\pi}{N}r}} = 0. \quad (1.14)$$

**Case  $r = 0$**  (i.e.  $n = m$ ): every term is  $e^0 = 1$ , so

$$S(0) = \sum_{k=0}^{N-1} 1 = N. \quad (1.15)$$

#### Discrete orthogonality relation

$$\sum_{k=0}^{N-1} e^{j\frac{2\pi}{N}k(n-m)} = \begin{cases} N, & n = m \\ 0, & n \neq m \end{cases} \quad (1.16)$$

This one identity is the engine behind *every* inverse formula and *every* property in this entire topic. Learn its two-line proof.

#### What orthogonality means geometrically

The  $N$  numbers  $q^0, q^1, \dots, q^{N-1}$  are  $N$  points spread evenly around the unit circle (they are the  $N$ -th roots of unity, possibly visited in a different order). Adding  $N$  vectors of equal length pointing in evenly spread directions gives zero — they cancel by symmetry. The only exception is when the “spread” is zero degrees, i.e.  $r = 0$ , where all  $N$  vectors point the same way and add up to  $N$ .

### 1.7.3 The discrete inverse

Put the orthogonality result into (1.13). Every term of the outer sum dies except  $m = n$ , which contributes  $x[n] \cdot N$ :

$$\sum_{k=0}^{N-1} (T_0 c_k) e^{j\frac{2\pi}{N}kn} = N x[n] \implies \boxed{x[n] = \frac{1}{N} \sum_{k=0}^{N-1} (T_0 c_k) e^{j\frac{2\pi}{N}kn}} \quad (1.17)$$

A *finite* sum of  $N$  terms recovers the samples *exactly*. No impulses, no infinite series.

**Notice what the two formulas actually touch**

Both the analysis sum and the synthesis sum only ever involve the  $N$  numbers  $T_0 c_k$  — never  $T_0$  or  $c_k$  separately. That combination is the natural discrete-frequency object. It is about to be given its own name.

**1.8 The Discrete Fourier Series (DFS)**

Now cut the umbilical cord. Forget  $x(t)$ , forget  $T_s$ , forget  $T_0$ . Let  $\tilde{x}[n]$  be *any*  $N$ -periodic sequence of numbers. The same two finite sums form a complete, self-contained pair.

**DFS pair**

$$\begin{aligned} \text{analysis: } a_k &= \frac{1}{N} \sum_{n=0}^{N-1} \tilde{x}[n] e^{-j\frac{2\pi}{N}kn} \\ \text{synthesis: } \tilde{x}[n] &= \sum_{k=0}^{N-1} a_k e^{j\frac{2\pi}{N}kn} \end{aligned}$$

Both  $\tilde{x}[n]$  and  $a_k$  are  $N$ -periodic:  $a_{k+N} = a_k$ .

It is the exact discrete twin of the CTFS. Compare them side by side:

|              | CTFS (period $T_0$ )                                              | DFS (period $N$ )                                                        |
|--------------|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| Analysis     | $c_k = \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\frac{2\pi}{T_0}t} dt$ | $a_k = \frac{1}{N} \sum_{n=0}^{N-1} \tilde{x}[n] e^{-jk\frac{2\pi}{N}n}$ |
| Synthesis    | $x(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\frac{2\pi}{T_0}t}$    | $\tilde{x}[n] = \sum_{k=0}^{N-1} a_k e^{jk\frac{2\pi}{N}n}$              |
| Coefficients | infinitely many                                                   | $N$ unique ( $a_{k+N} = a_k$ )                                           |
| Normaliser   | $1/T_0$ (a duration)                                              | $1/N$ (a count)                                                          |

**1.8.1 The winding picture again, now for samples**

The DFS analysis formula  $a_k = \frac{1}{N} \sum_n x[n] e^{-j\frac{2\pi}{N}kn}$  is a **plain average** of  $N$  complex numbers — literally: add  $N$  things, divide by  $N$ .

Wind the *samples* instead of the curve: sample  $n$  sits at radius  $x[n]$  and angle  $-\frac{2\pi}{N}kn$ . Their centre of mass is  $a_k$ .

**Why the integral became a sum — the picture version**

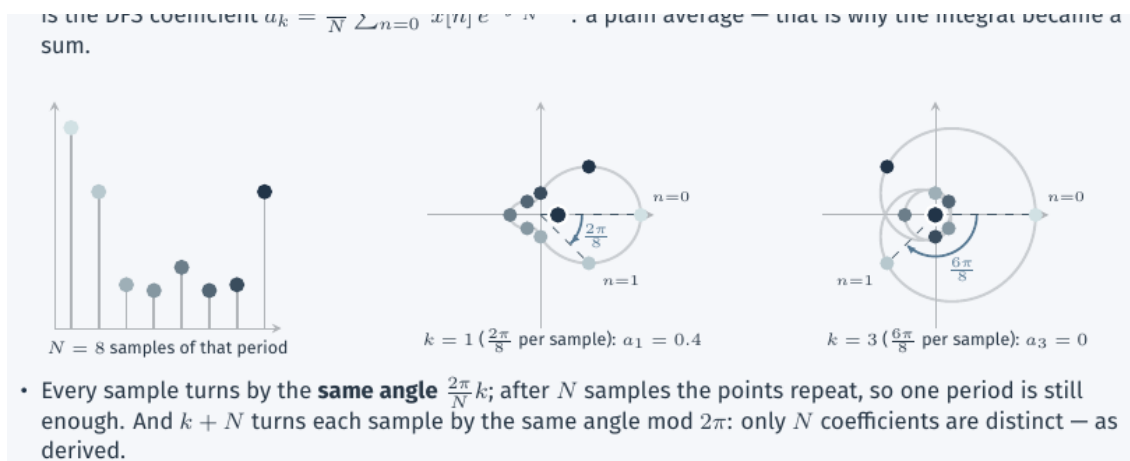
For a continuous curve you must average over a continuum of points, which is an integral. For  $N$  samples you average  $N$  numbers, which is a sum divided by  $N$ . That is the entire difference between the CTFS and the DFS.

And now the  $N$ -periodicity is obvious too: winding at rate  $k + N$  turns each sample by  $\frac{2\pi}{N}(k + N)n = \frac{2\pi}{N}kn + 2\pi n$ , i.e. by the same angle plus a whole number of full turns. The points land in *exactly* the same places. Same picture  $\Rightarrow$  same centroid  $\Rightarrow$  same coefficient.

**1.9 The Discrete Fourier Transform (DFT)**

The DFS carries a  $1/N$  in the analysis equation. In computing and in almost all engineering practice, one prefers the “bare” analysis sum with no scale factor. That is the DFT.





**Figure 1.6:** Winding  $N = 8$  samples. At  $k = 1$  each successive sample is turned by  $2\pi/8$ ; the mass is lopsided and  $a_1 = 0.4$ . At  $k = 3$  each sample turns by  $6\pi/8$ ; the points spread evenly and cancel, so  $a_3 = 0$ .

First, notation. Define the **twiddle factor**

$$W_N \triangleq e^{-j\frac{2\pi}{N}}, \quad \text{so that} \quad W_N^N = 1. \quad (1.18)$$

$W_N$  is a primitive  $N$ -th root of unity; its powers march around the unit circle in  $N$  equal steps and then repeat.

#### DFT / IDFT pair

For a length- $N$  sequence  $\{x[n]\}_{n=0}^{N-1}$ :

$$X[k] \triangleq \sum_{n=0}^{N-1} x[n] W_N^{kn} = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn} = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}, \quad n = 0, 1, \dots, N-1$$

#### Sign and scale — the two things people get wrong

- $W_N$  already contains the minus sign. So the *forward* DFT uses  $W_N^{+kn}$  and the *inverse* uses  $W_N^{-kn}$ . If you write  $W_N^{-kn}$  in the forward transform you have computed the inverse (times  $N$ ).
- The  $1/N$  lives in the **inverse** for the DFT, but in the **forward** equation for the DFS. Different books put it in different places; in this course, follow the boxes above.

### 1.9.1 The three bridges

Three separate objects have now appeared, and they are all the same  $N$  numbers up to a scale. These identities are examinable one-liners.

#### The three bridges

1.  $X[k] = N a_k$  — the DFT is the DFS, rescaled by  $N$ .
2.  $c_k = \frac{1}{T_0} X[k]$  — the CTFS coefficients of the *sampled* continuous periodic signal  $x_p(t)$  are the DFT values divided by the period  $T_0$ .
3.  $X[k] = X(e^{j\omega})|_{\omega=\frac{2\pi}{N}k}$  — the DFT is  $N$  evenly spaced samples of the DTFT. (Proved in Section 3.1.)

## 1.10 The scaffolding comes down

Look at the DFT formula once more:

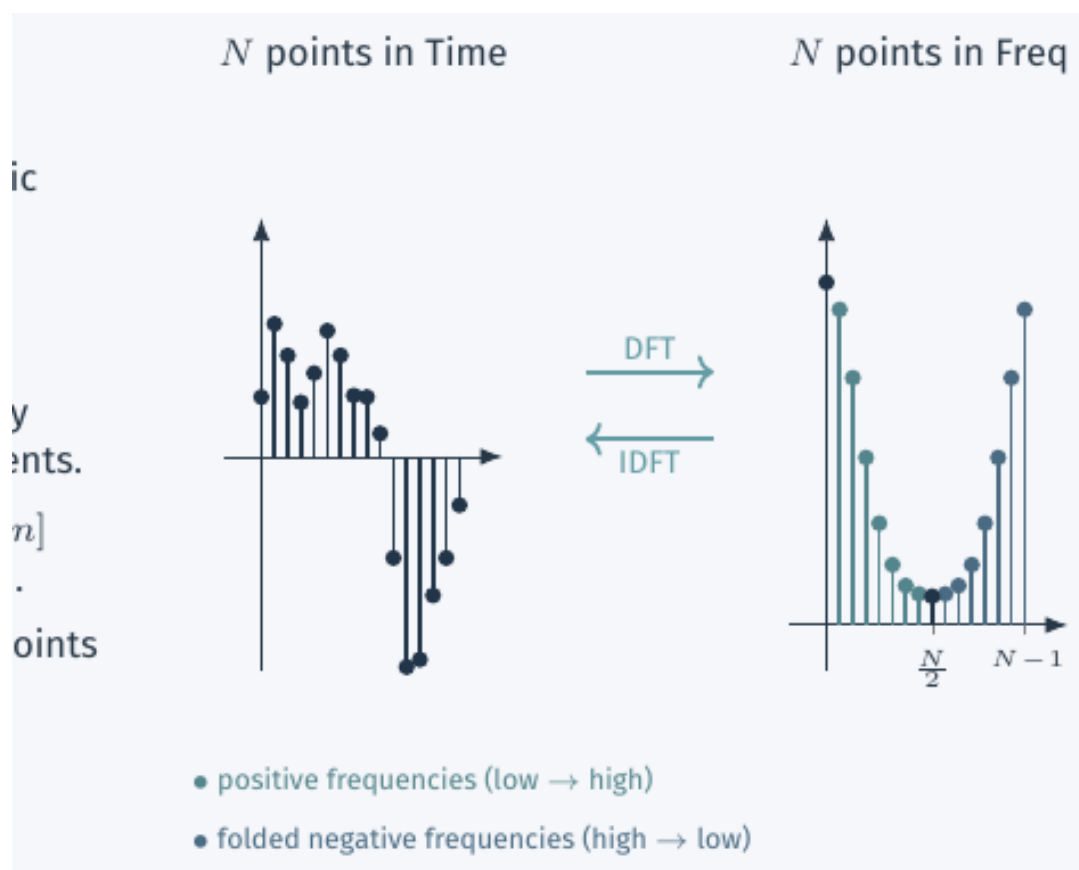
$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}.$$

Nothing in it mentions  $T_s$ ,  $T_0$ ,  $\omega_0$  or  $x(t)$ . It consumes exactly  $N$  numbers and produces exactly  $N$  numbers.

### The sampling story was scaffolding

The continuous signal, the impulse train, the CTFS — all of it existed only to *explain* why the Fourier integral turned into a finite sum. Having explained it, we throw it away.

**Any** length- $N$  (or  $N$ -periodic) list of numbers has a DFT: an audio buffer, a row of pixels, a week of temperature readings, a sequence of stock prices. No continuous ancestor is required.



**Figure 1.7:**  $N$  points in time  $\leftrightarrow$   $N$  points in frequency. The DFT maps exactly  $N$  samples to exactly  $N$  coefficients, and the IDFT maps them back perfectly. The frequency side reads as: positive frequencies low→high, then folded negatives high→low.

**Counting check.**  $N$  real (or complex) numbers in,  $N$  complex numbers out. For a *real* input this looks like it doubles the information — it does not, because conjugate symmetry (Part II, Section 4.7) forces half the output bins to be the conjugates of the other half. The count always balances.

## Chapter 2

# The Discrete-Time Fourier Transform (DTFT)

### 2.1 Now let the signal be aperiodic

Everything so far assumed periodicity. What if the signal does not repeat?

Take an aperiodic  $x(t)$  that is nonzero only on a finite stretch of length  $T_0$ . Sample it the same way; you get a finite-length sequence  $x[n]$ , nonzero only for  $0 \leq n \leq N-1$ . The impulse-train version is a *finite* sum:

$$x_p(t) = x(t)p(t) = \sum_{n=-\infty}^{\infty} x[n]\delta(t - nT_s) = \sum_{n=0}^{N-1} x[n]\delta(t - nT_s). \quad (2.1)$$

An aperiodic signal calls for the Fourier *transform*, not the series. So take the CTFT of  $x_p(t)$ :

$$X_p(j\omega) = \int_{-\infty}^{\infty} x_p(t)e^{-j\omega t}dt = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega nT_s}, \quad (2.2)$$

again by the sifting property of  $\delta$ .

### 2.2 Compressing the frequency axis: digital frequency

The expression above still carries  $T_s$ , which is a property of our measurement setup, not of the numbers. Get rid of it by **relabelling the frequency axis**: define a new variable through the substitution  $\omega T_s \mapsto \omega$ .

#### Discrete-Time Fourier Transform (DTFT)

$$X(e^{j\omega}) \triangleq X_p(j\omega) \Big|_{\omega T_s \mapsto \omega} = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n} \quad (2.3)$$

For a finite length- $N$  capture,  $X(e^{j\omega}) = \sum_{n=0}^{N-1} x[n]e^{-j\omega n}$ .

#### What the new $\omega$ means — this matters

The new  $\omega$  is **digital frequency**, measured in **radians per sample** (not radians per second).

The link to physical frequency  $\Omega$  (rad/s) is

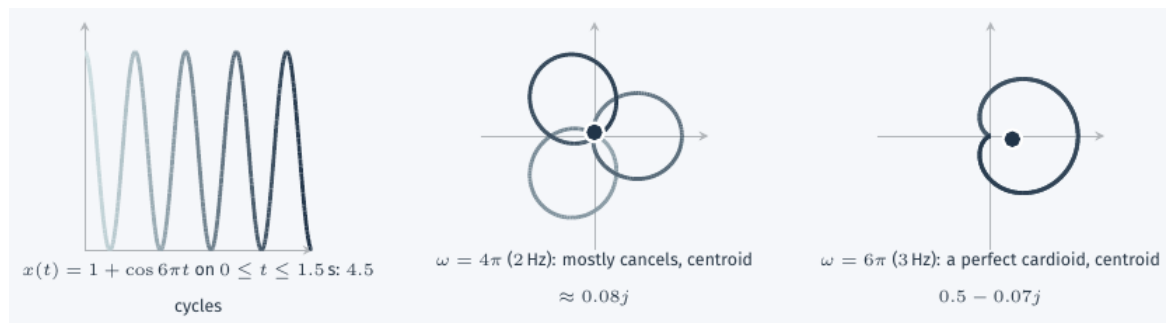
$$\omega = \Omega T_s = \frac{\Omega}{f_s} \quad (2.4)$$

So a 3 Hz tone sampled at 24 samples/s becomes  $\omega = 2\pi \cdot 3 \cdot \frac{1}{24} = \frac{\pi}{4}$  radians per sample.

### Why measuring in “per sample” is the natural choice

The sequence  $x[n]$  is just a list. It has no idea how fast it was recorded. The only thing that is meaningful *inside the list* is: “by how much does the phase advance from one entry to the next?” That is exactly  $\omega$  in radians per sample. The seconds only come back at the very end, when you choose to relabel the axis in Hz (Section 3.5).

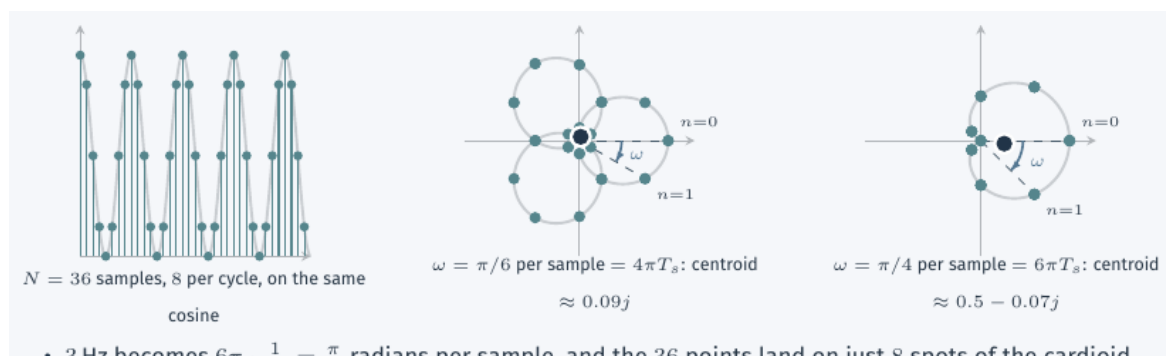
#### 2.2.1 The winding picture for an aperiodic signal



**Figure 2.1:** A finite segment of  $x(t) = 1 + \cos 6\pi t$  on  $0 \leq t \leq 1.5$  s (4.5 cycles). Nothing repeats outside the segment, so the winding runs over the whole segment. At the cosine’s own rate ( $\omega = 6\pi$ , i.e. 3 Hz) every cycle traces the same cardioid and the mass piles to the right. At any other rate ( $\omega = 4\pi$ ) the cycles spread around the circle and nearly cancel.

### Why the DTFT spectrum is a *curve*, not a set of lines

For a periodic signal, only the rates  $k\omega_0$  closed the loop, so only those survived: lines. For a finite segment, *no* rate is special — the winding simply stops when the segment ends. So every  $\omega$  produces some centroid, usually small but nonzero. Every  $\omega$  gets a value  $\Rightarrow$  a *continuous* spectrum. The small nonzero residue at “wrong” frequencies (here caused by 4.5 cycles not closing cleanly) is the continuous *skirt* around the peak.



**Figure 2.2:** The same segment, now sampled ( $N = 36$  samples, 8 per cycle). Consecutive points are turned by the same angle  $\omega$  — the digital frequency. At  $\omega = \pi/4$  per sample the 36 points land on only 8 distinct spots on the cardioid, piling up there.

## 2.3 The DTFT is $2\pi$ -periodic

This is the discrete-time counterpart of Section 1.5, and its proof is one line.

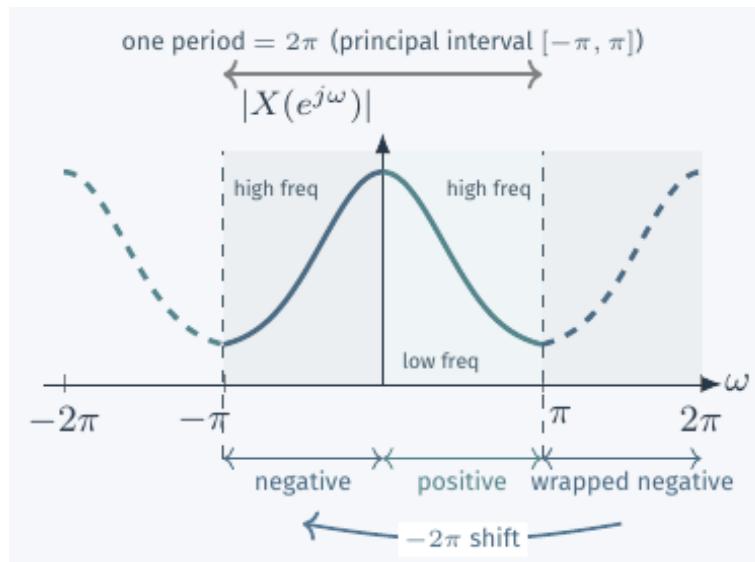
### Period of the DTFT

Each term satisfies  $e^{-j(\omega+2\pi)n} = e^{-j\omega n} \underbrace{e^{-j2\pi n}}_{=1} = e^{-j\omega n}$  because  $n$  is an *integer*. Therefore

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}) \quad (2.5)$$

### The picture-proof

Turning each sample by  $\omega + 2\pi$  instead of  $\omega$  adds a whole extra turn per sample — which lands every point in exactly the same place. Same winding picture  $\Rightarrow$  same centroid  $\Rightarrow$  same transform value. A sampled signal simply **cannot distinguish**  $\omega$  from  $\omega + 2\pi$ .



**Figure 2.3:** One period of the DTFT is any  $2\pi$ -wide window. The usual principal interval is  $[-\pi, \pi]$ ; the segment  $[\pi, 2\pi)$  is the folded copy of  $[-\pi, 0)$ , shifted right by  $2\pi$ .

### How to read one period

- **Any**  $2\pi$ -wide window holds the complete information. Nothing is lost by choosing one window over another.
- Common choice 1: the principal interval  $[-\pi, \pi]$  — symmetric, dc in the middle, negative frequencies naturally on the left.
- Common choice 2:  $[0, 2\pi)$  — what a computer reports. Here  $[\pi, 2\pi)$  is the folded copy of  $[-\pi, 0)$ .
- $\omega = \pm\pi$  is the *highest* digital frequency: phase flips by  $\pi$  every sample, i.e. the alternating sequence  $+1, -1, +1, -1, \dots$ . You cannot oscillate faster than “flip every sample”.
- Undoing the relabel  $\omega = \Omega T_s$ : the window  $[-\pi, \pi]$  corresponds to  $[-\frac{\omega_s}{2}, \frac{\omega_s}{2}]$  where  $\omega_s = 2\pi/T_s$ . In other words:  $\omega = \pi$  is **half the sampling rate**.

## 2.4 The inverse DTFT

Same pattern as before — synthesis uses  $e^{+j\omega n}$  — but the spectrum is now a *continuous function* of  $\omega$ . So instead of summing over  $N$  bins we *integrate* over one  $2\pi$  period.

Propose

$$\hat{x}[n] \triangleq \frac{1}{2\pi} \int_{2\pi} X(e^{j\omega}) e^{j\omega n} d\omega, \quad (2.6)$$

and check it works. Substitute the analysis sum and swap sum and integral:

$$\hat{x}[n] = \sum_{m=-\infty}^{\infty} x[m] \frac{1}{2\pi} \int_{2\pi} e^{j\omega(n-m)} d\omega. \quad (2.7)$$

The inner integral is the continuous cousin of the orthogonality relation. With  $r = n - m$  an integer:

$$\frac{1}{2\pi} \int_{-\pi}^{\pi} e^{j\omega r} d\omega = \begin{cases} 1, & r = 0 \\ 0, & r \neq 0 \end{cases} \quad (2.8)$$

because for  $r \neq 0$  the integrand completes  $|r|$  whole cycles around the unit circle and averages to zero, while for  $r = 0$  the integrand is the constant 1.

Only  $m = n$  survives, so  $\hat{x}[n] = x[n]$ . Done.

#### Inverse DTFT

$$x[n] = \frac{1}{2\pi} \int_{2\pi} X(e^{j\omega}) e^{j\omega n} d\omega \quad (2.9)$$

#### DFS vs DTFT, one sentence each

**DFS:**  $x[n]$  is a *finite sum* of  $N$  complex exponentials.

**DTFT:**  $x[n]$  is a *continuum superposition* — an integral — of exponentials  $e^{j\omega n}$  over one  $2\pi$  period.

## Chapter 3

# The DFT through the DTFT lens

### 3.1 The DFT is $N$ samples of the DTFT

There is nothing to derive here — just stare at the two definitions:

$$X(e^{j\omega}) = \sum_{n=0}^{N-1} x[n] e^{-j\omega n} \quad (\text{DTFT of the length-}N \text{ sequence})$$

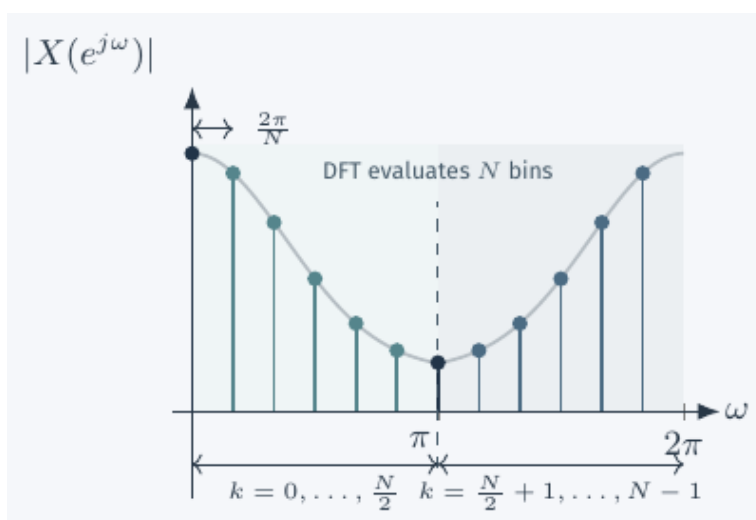
$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad (\text{DFT})$$

The second is the first with  $\omega$  replaced by  $\frac{2\pi}{N}k$ .

#### The third bridge

$$X[k] = X(e^{j\omega}) \Big|_{\omega=\frac{2\pi}{N}k}, \quad k = 0, 1, \dots, N-1 \quad (3.1)$$

The  $N$ -point DFT takes  $N$  **evenly spaced samples** of one  $2\pi$  period of the DTFT, at spacing  $\Delta\omega = \frac{2\pi}{N}$ .



**Figure 3.1:** The DTFT is the grey curve; the DFT is the  $N$  stems sitting on it, at spacing  $2\pi/N$  across  $[0, 2\pi)$ . Bins  $0, \dots, N/2$  climb from dc to the highest positive frequency; the rest hold the folded negatives.

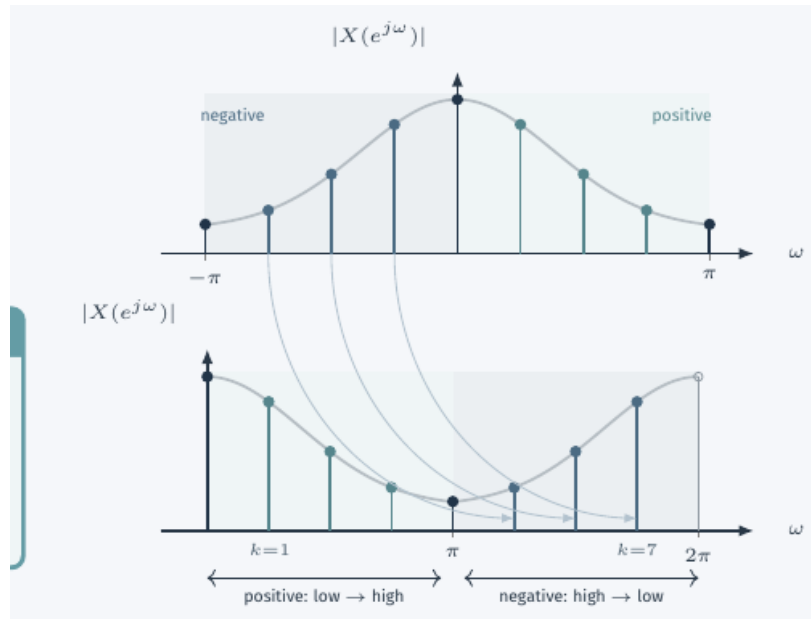
### One transform, two views

**Periodic view:**  $X[k]$  are the Fourier-series coefficients (times  $N$ ) of the  $N$ -periodic extension of  $x$ .

**Aperiodic view:**  $X[k]$  are  $N$  samples of the DTFT of the finite-length  $x$ .

*Same  $N$  numbers.* Which view you take depends only on which is more convenient for the question in front of you.

## 3.2 Reading DFT bins, once more



**Figure 3.2:** Bin  $k$  sits at digital frequency  $\omega_k = \frac{2\pi}{N}k$ . Top: the centred view on  $[-\pi, \pi]$ . Bottom: the computed view on  $[0, 2\pi)$  — the same numbers, re-indexed.

For  $N = 8$  the bin frequencies are:

| $k$                                         | 0 | 1               | 2               | 3                | 4     | 5                 | 6                | 7                |
|---------------------------------------------|---|-----------------|-----------------|------------------|-------|-------------------|------------------|------------------|
| $\omega_k = \frac{2\pi k}{8}$ (as computed) | 0 | $\frac{\pi}{4}$ | $\frac{\pi}{2}$ | $\frac{3\pi}{4}$ | $\pi$ | $\frac{5\pi}{4}$  | $\frac{3\pi}{2}$ | $\frac{7\pi}{4}$ |
| $\omega_k$ folded into $[-\pi, \pi]$        | 0 | $\frac{\pi}{4}$ | $\frac{\pi}{2}$ | $\frac{3\pi}{4}$ | $\pi$ | $-\frac{3\pi}{4}$ | $-\frac{\pi}{2}$ | $-\frac{\pi}{4}$ |

Read left to right: dc  $\rightarrow$  rising positive frequency  $\rightarrow$  the maximum at  $\pi$   $\rightarrow$  negative frequencies falling back toward dc.

### To fold a bin index into the principal interval

If  $k \leq N/2$ :  $\omega_k = \frac{2\pi k}{N}$  (positive).

If  $k > N/2$ :  $\omega_k = \frac{2\pi(k-N)}{N}$  (negative). Subtract  $N$  from  $k$ , then convert. Do not subtract  $2\pi$  from the answer and hope — it is the same thing, but the index form is harder to get wrong.

## 3.3 Zero-padding: same spectrum, finer samples

Given a length- $N$  sequence  $x[n]$ , append zeros until the total length is  $M \geq N$ , then take the  $M$ -point DFT.

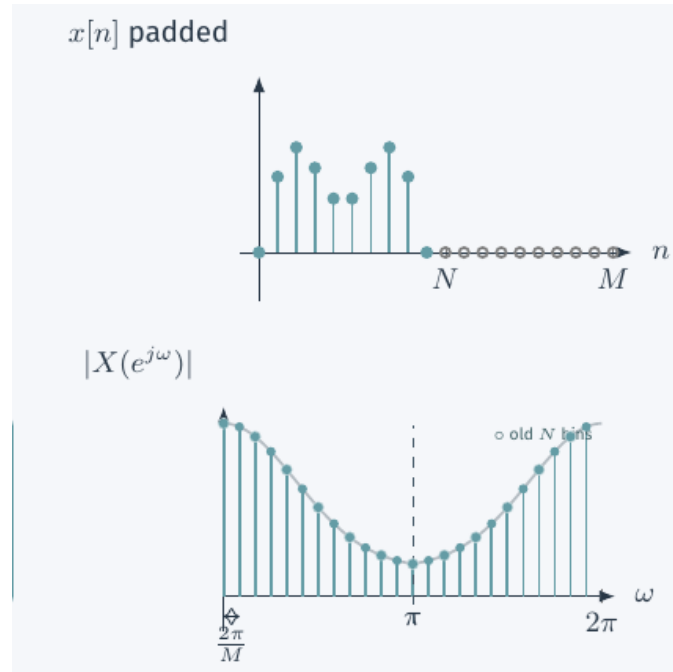
**What happens to the DTFT?** Nothing. The appended zeros contribute  $0 \cdot e^{-j\omega n} = 0$  to the sum. The DTFT curve  $X(e^{j\omega})$  is *identical*.



**What happens to the DFT?** It now samples that same curve at  $M$  points instead of  $N$ :

$$X_M[k] = \sum_{n=0}^{M-1} x[n] e^{-j\frac{2\pi}{M}kn} = X(e^{j\omega}) \Big|_{\omega=\frac{2\pi}{M}k'} \quad (3.2)$$

i.e. at spacing  $\frac{2\pi}{M}$  instead of  $\frac{2\pi}{N}$  — *finer*.



**Figure 3.3:** Top:  $x[n]$  padded from  $N$  to  $M$  with zeros. Bottom: the same DTFT curve, now sampled at  $M$  points (small circles mark where the original  $N$  bins were).

#### More pixels, not more information

Zero-padding **interpolates the picture** of a spectrum you already had. It gives a smoother, denser plot. It **cannot** recover detail the  $N$  real samples never captured.

**Concretely:** two tones 0.1 Hz apart, observed for 1 second, are smeared into one lump in the DTFT itself. Zero-padding to a million points will plot that single lump beautifully smoothly — and it will still be one lump. To resolve them you need a *longer recording*, i.e. more real samples.

#### Two different uses of the same word — do not confuse them

“Zero-padding” shows up three times in this course, for three different reasons:

1. **Here (Part I):** to interpolate the spectrum plot — cosmetic, optional.
2. **Part II, Section 5.5:** to make circular convolution equal linear convolution — *mandatory for correctness*, needs  $N \geq L + M - 1$ .
3. **Part III, Section 8.3:** to make the length a power of 2 so a radix-2 FFT can run — needs  $N = 2^{\lceil \log_2 M \rceil}$ .

Same operation, three different reasons. Exam questions love to ask which one you mean.

#### Caution the slides flag explicitly

If  $x[n]$  is genuinely *periodic* and you are using the DFT in its “Fourier series” role, zero-padding **changes the signal**: you have altered its period and its shape. The padded se-

quence's periodic extension is no longer the original periodic signal, so its coefficients no longer mean what they meant. Padding is safe for the *finite-length / aperiodic* interpretation, not for the periodic one.

### 3.4 A complete numeric example ( $N = 4$ )

**Worked example 1.** Compute the 4-point DFT of  $x[n] = \{1, 2, 3, 4\}$ .

With  $N = 4$ ,  $W_4 = e^{-j2\pi/4} = e^{-j\pi/2} = -j$ . So  $W_4^{kn} = (-j)^{kn}$ , and the powers cycle  $1, -j, -1, +j, 1, \dots$  with period 4. Tabulate  $W_4^{kn}$ :

| $W_4^{kn}$ | $n = 0$ | $n = 1$ | $n = 2$ | $n = 3$ |
|------------|---------|---------|---------|---------|
| $k = 0$    | 1       | 1       | 1       | 1       |
| $k = 1$    | 1       | $-j$    | $-1$    | $j$     |
| $k = 2$    | 1       | $-1$    | 1       | $-1$    |
| $k = 3$    | 1       | $j$     | $-1$    | $-j$    |

Now  $X[k] = \sum_{n=0}^3 x[n]W_4^{kn}$  is just “dot the row with  $\{1, 2, 3, 4\}$ ”:

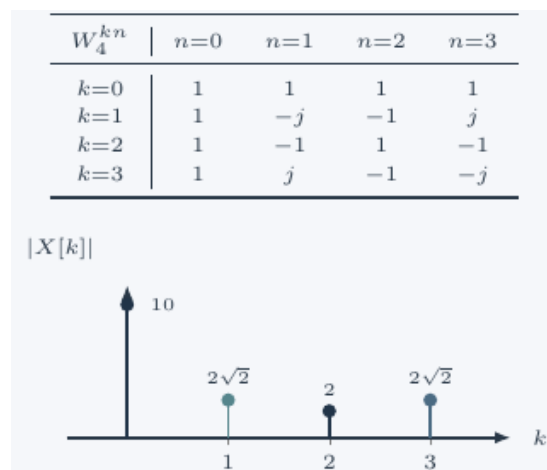
$$X[0] = 1 + 2 + 3 + 4 = 10$$

$$X[1] = 1 - 2j - 3 + 4j = -2 + 2j$$

$$X[2] = 1 - 2 + 3 - 4 = -2$$

$$X[3] = 1 + 2j - 3 - 4j = -2 - 2j$$

Magnitudes:  $|X| = \{10, 2\sqrt{2}, 2, 2\sqrt{2}\}$ .



**Figure 3.4:** The  $W_4^{kn}$  table and the resulting magnitude spectrum  $|X[k]| = \{10, 2\sqrt{2}, 2, 2\sqrt{2}\}$ .

#### Two checks you should always do

1.  **$X[0]$  is the sum of the samples.** Setting  $k = 0$  makes every exponential equal 1.  $X[0] = 1 + 2 + 3 + 4 = 10$ . ✓ (So  $X[0]/N$  is the average, the “dc value”.)
2. **Conjugate symmetry.**  $x$  is real, so  $X[N - k] = X^*[k]$ . Here  $X[3] = -2 - 2j = (-2 + 2j)^* = X^*[1]$ . ✓ Bin 3 is bin 1’s negative-frequency partner:  $\omega_3 = \frac{2\pi \cdot 3}{4} = \frac{3\pi}{2} \equiv -\frac{\pi}{2}$ , which is  $-\omega_1$ . (Full proof in Part II.)

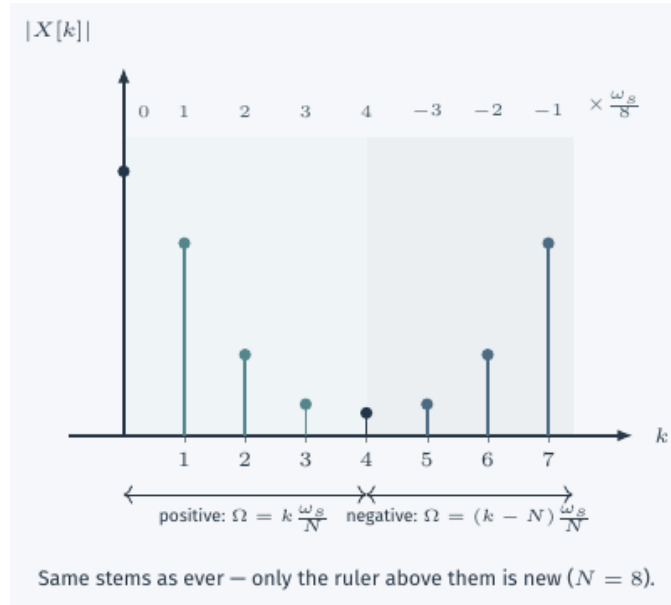
### 3.5 Back to physical frequency, and to hertz

The DTFT/DFT axis is normalised:  $\omega$  is radians per sample. To put real units back, undo the relabel  $\omega = \Omega T_s$ .

#### 3.5.1 In rad/s

For bin  $k$  at  $\omega_k = \frac{2\pi}{N}k$ :

$$\Omega_k = \frac{\omega_k}{T_s} = \frac{2\pi k}{NT_s} = k \frac{\omega_s}{N}, \quad \omega_s = \frac{2\pi}{T_s}. \quad (3.3)$$



**Figure 3.5:** Same  $N = 8$  stems, new ruler. Bins  $0 \rightarrow N/2$  span  $0 \rightarrow \omega_s/2$ ; bins  $N/2 \rightarrow N$  span  $-\omega_s/2 \rightarrow 0$ . Relabelling the axis changes units, not samples.

#### 3.5.2 In hertz

Divide by  $2\pi$ :  $f = \Omega/2\pi$ , and  $f_s = 1/T_s$ .

##### Bins to hertz

$$\Delta f = \frac{f_s}{N} = \frac{1}{NT_s} = \frac{1}{T_0} \quad (\text{the bin spacing / frequency resolution}) \quad (3.4)$$

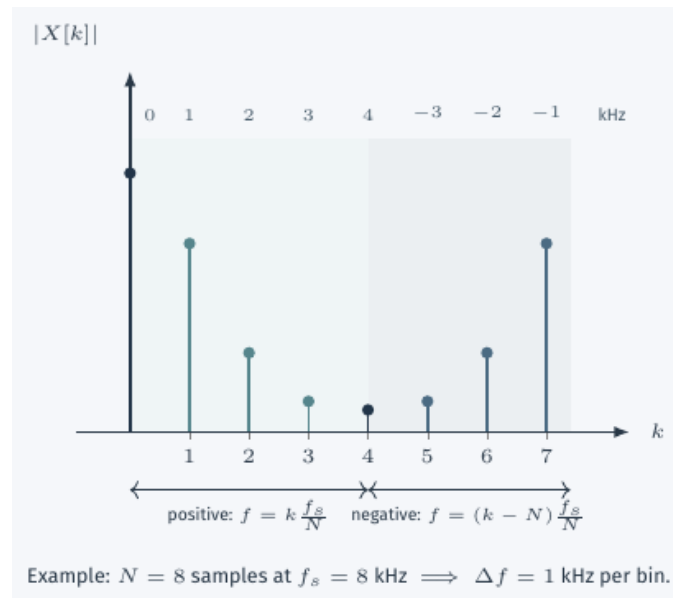
$$f_k = \begin{cases} k \frac{f_s}{N}, & 0 \leq k \leq \frac{N}{2} \\ (k - N) \frac{f_s}{N}, & \frac{N}{2} < k \leq N - 1 \end{cases} \quad (3.5)$$

##### The resolution identity, and what it costs you

$$\Delta f = \frac{1}{T_0} \quad \text{where } T_0 = NT_s \text{ is the recording length in seconds.} \quad (3.6)$$

Frequency resolution depends **only on how long you recorded**, not on how fast you sampled and not on how much you zero-pad. To halve  $\Delta f$ , record for twice as long.

Sampling faster (bigger  $f_s$ ) increases the *highest* frequency you can see ( $f_s/2$ ), not the fineness of the bins.



**Figure 3.6:** Bin index  $\rightarrow$  hertz, for  $N = 8$  at  $f_s = 8$  kHz:  $\Delta f = 1$  kHz per bin, so the bins are 0, 1, 2, 3, 4, -3, -2, -1 kHz.

**Worked example 1.**  $N = 8$  samples at  $f_s = 8$  kHz. Which bin does a 3 kHz tone land in? What about -3 kHz? What about 6 kHz?

$$\Delta f = f_s / N = 8000 / 8 = 1000 \text{ Hz per bin.}$$

- 3 kHz  $\implies k = 3 / 1 = 3$ . (Positive half, since  $3 < N/2 = 4$ .)
- -3 kHz  $\implies$  use the second case:  $(k - 8) \cdot 1 = -3 \implies k = 5$ .
- 6 kHz is *above*  $f_s/2 = 4$  kHz — it violates the sampling theorem and **aliases**.  $6 - 8 = -2$  kHz, so it appears as a -2 kHz component, i.e. in bin  $k = 6$ , indistinguishable from a genuine -2 kHz tone.

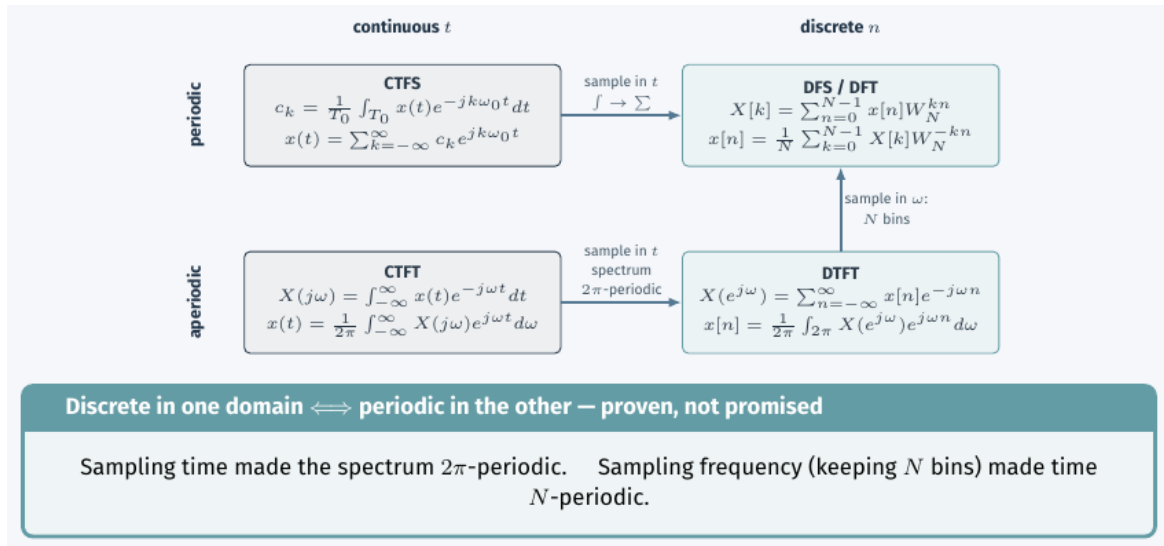
**Practical note.** The ordering  $0, \Delta f, 2\Delta f, \dots, \frac{f_s}{2}, \dots, -2\Delta f, -\Delta f$  is exactly what `numpy.fft.fftfreq` returns, and it is why spectrum analysers plot only 0 to  $f_s/2$  — the rest is the mirror image for a real signal.

### 3.6 Part I in one picture: the square, fully derived

Discrete in one domain  $\iff$  periodic in the other — proven, not promised

- Section 2.3: sampling in time made the spectrum  $2\pi$ -periodic. Proof:  $e^{-j2\pi n} = 1$  for integer  $n$ .
- Section 1.5: keeping only  $N$  frequency bins made time  $N$ -periodic. Proof: the same identity read on the other index.

Both proofs are the same one-line fact about integers and full turns. If you remember only one thing from Part I, remember that.



**Figure 3.7:** The complete family. Sampling in time moves you left  $\rightarrow$  right and makes the spectrum periodic. Sampling in frequency moves you CTFT/DTFT  $\rightarrow$  DFS/DFT and makes time periodic.

### 3.7 Part I summary

#### All of Part I

**DFS pair** ( $\tilde{x}[n]$  is  $N$ -periodic):

$$a_k = \frac{1}{N} \sum_{n=0}^{N-1} \tilde{x}[n] e^{-j\frac{2\pi}{N}kn}, \quad \tilde{x}[n] = \sum_{k=0}^{N-1} a_k e^{j\frac{2\pi}{N}kn}$$

**DFT/IDFT pair** ( $W_N = e^{-j2\pi/N}$ ):

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}$$

**DTFT pair:**

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}, \quad x[n] = \frac{1}{2\pi} \int_{2\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

**Three bridges:**  $c_k = \frac{1}{T_0} X[k]$ ,  $X[k] = N a_k$ ,  $X[k] = X(e^{j\omega})|_{\omega=\frac{2\pi}{N}k}$ .

**Bin frequencies:**  $\omega_k = \frac{2\pi k}{N}$  rad/sample,  $f_k = k \frac{f_s}{N}$  (fold with  $k - N$  for  $k > N/2$ ),  $\Delta f = \frac{f_s}{N} = \frac{1}{T_0}$ .

**Reading bins:**  $0 \rightarrow \frac{N}{2}$  positive low  $\rightarrow$  high;  $\frac{N}{2} \rightarrow N$  negative high  $\rightarrow$  low.

## PART II

# DFT properties and circular convolution

## Chapter 4

# The rules of the game

### 4.1 Everything lives on a clock

Part I built the DFT. Part II works out its *grammar*: what happens to  $X[k]$  when you shift, flip, conjugate or modulate  $x[n]$ .

Everything in this part follows from one structural fact, so start with it.

**The  $N$  points are arranged in a circle, not a line**

The DFT only ever sees  $N$  numbers,  $x[0], \dots, x[N-1]$ . But the IDFT formula

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}$$

is perfectly happy to be evaluated at *any* integer  $n$  — and if you evaluate it at  $n + N$  you get back  $x[n]$  (because  $W_N^{-k(n+N)} = W_N^{-kn} W_N^{-kN} = W_N^{-kn}$ ).

So the object the DFT *actually* works with is not a finite list; it is an infinite  **$N$ -periodic** sequence. Walking off the right-hand end brings you back to the left-hand end.

Two pieces of notation make this precise.

**Modular index and periodic extension**

**Modular index:**  $((n))_N \triangleq n \bmod N$ , always giving a result in  $\{0, 1, \dots, N-1\}$ .

$$((9))_8 = 1, \quad ((-1))_8 = 7, \quad ((-2))_8 = 6.$$

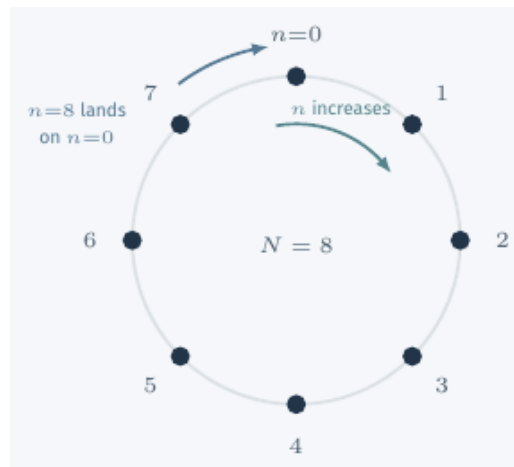
**Periodic extension:**  $\tilde{x}[n] \triangleq x[ ((n))_N ]$ , so  $\tilde{x}[n+N] = \tilde{x}[n]$  and  $\tilde{x}$  agrees with  $x$  on  $0, \dots, N-1$ .

**The mod of a negative number**

In this subject  $((-1))_N$  means  $N-1$ , *not*  $-1$ . Take the remainder that lands in  $[0, N-1]$ . If your calculator or your C code gives  $-1$ , add  $N$ . (Python's `%` already does the right thing.)

**The theme of the whole of Part II**

**Nothing falls off the end of  $N$  points — it *wraps*. Every shift becomes a rotation of the clock.**



**Figure 4.1:** Indices live on a clock face with  $N$  positions. Walking forward past position  $N - 1$  lands you back on 0. The bins  $k$  live on the same clock — we proved  $X[k + N] = X[k]$  in Part I.

## 4.2 The one proof move

Every property below is proved the same way. Learn the move once and you never have to memorise a proof again.

### The one proof move

1. Substitute the modified signal into the analysis sum  $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$ .
2. Change the summation variable (re-index).
3. Note that both  $\tilde{x}[n]$  and  $W_N^{kn}$  are  $N$ -periodic in  $n$ , so **any window of  $N$  consecutive terms gives the same total**. Slide the window back to  $0, \dots, N - 1$ .

### Why step 3 is legal

Suppose the re-indexing left you summing over  $r = -m, \dots, N - 1 - m$ . The terms at  $r = -m, \dots, -1$  are, by periodicity, numerically identical to the terms at  $r = N - m, \dots, N - 1$ . So sliding the window just swaps each missing term for its identical twin. The total is unchanged. *The wrap is invisible in the algebra* — but it is very visible when you compute a convolution, which is why Part II ends where it does.

## 4.3 Two properties for free

### 4.3.1 Linearity

The analysis sum is a weighted sum of the samples, and sums are linear:

$$a x[n] + b y[n] \longleftrightarrow a X[k] + b Y[k]. \quad (4.1)$$

### The length condition (remember it — it comes back in Part II)

Both sequences must have the **same length**  $N$ . If they don't, zero-pad the shorter one first. Adding a length-5 and a length-8 sequence's DFTs makes no sense: their bins sit at different frequencies ( $2\pi k/5$  versus  $2\pi k/8$ ).

### 4.3.2 Periodicity of the bins

$W_N^{(k+N)n} = W_N^{kn} W_N^{Nn} = W_N^{kn}$ , so

$$X[k + N] = X[k]. \quad (4.2)$$



Proved in Part I; restated here because it is what makes  $((k))_N$  meaningful on the frequency side.

## 4.4 Circular time shift

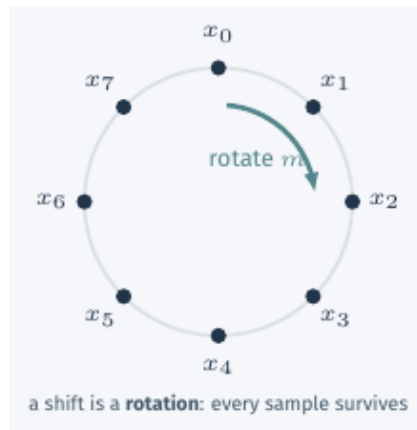
**Question:** what happens to  $X[k]$  if the signal is delayed by  $m$  samples — on the clock?

Take the DFT of  $\tilde{x}[n - m]$  and substitute  $r = n - m$ :

$$\begin{aligned} \sum_{n=0}^{N-1} \tilde{x}[n - m] W_N^{kn} &\stackrel{r=n-m}{=} \sum_{r=-m}^{N-1-m} \tilde{x}[r] W_N^{k(r+m)} = W_N^{km} \sum_{r=-m}^{N-1-m} \tilde{x}[r] W_N^{kr} \\ &= W_N^{km} \sum_{r=0}^{N-1} \tilde{x}[r] W_N^{kr} \quad (\text{the one proof move: slide the window}) \\ &= W_N^{km} X[k]. \end{aligned}$$

### Circular time-shift property

$$x[((n - m))_N] \longleftrightarrow W_N^{km} X[k] = e^{-j\frac{2\pi}{N}km} X[k] \quad (4.3)$$



**Figure 4.2:** A circular shift is a **rotation** of the clock. Every sample survives; none is lost off the end.

### Phase only — the single most useful consequence

$|W_N^{km}| = 1$ , because it is a point on the unit circle. Therefore

$$|X_{\text{shifted}}[k]| = |X[k]| \quad \text{for every } k, \quad (4.4)$$

and only the phase changes, by the *linear* amount  $-\frac{2\pi}{N}km$ .

**In words:** the magnitude spectrum does not know *where* the signal sits in time. All timing information lives in  $\angle X[k]$ .

### Why the phase shift is *linear* in $k$

A delay of  $m$  samples delays every component. But a fixed *time* delay is a bigger *phase* delay for a faster component — a component that completes twice as many cycles per sample gets twice the phase lag. Hence the phase lag  $\propto k$ : a straight line in  $k$ , with slope  $-2\pi m/N$ .

This is why “linear phase” filters are prized: a phase response that is a straight line in frequency is nothing but a pure delay, which does not distort the waveform’s shape.

**Worked example 1.**  $x = \{1, 2, 3, 4\}$  has  $X = \{10, -2 + 2j, -2, -2 - 2j\}$ . Find the DFT of  $y = \{4, 1, 2, 3\}$ .

$y$  is  $x$  circularly shifted right by  $m = 1$ :  $y[n] = x[(n - 1)_N]$ . So  $Y[k] = W_4^k X[k] = (-j)^k X[k]$ :

$$\begin{aligned} Y[0] &= W_4^0 X[0] = 10, & Y[1] &= W_4^1 X[1] = (-j)(-2 + 2j) = 2 + 2j, \\ Y[2] &= W_4^2 X[2] = (-1)(-2) = 2, & Y[3] &= W_4^3 X[3] = (j)(-2 - 2j) = 2 - 2j. \end{aligned}$$

Check the magnitudes:  $\{10, 2\sqrt{2}, 2, 2\sqrt{2}\}$  — identical to  $|X[k]|$ . ✓

Check  $Y[0]$ : it must be  $4 + 1 + 2 + 3 = 10$ . ✓

## 4.5 Frequency shift (modulation)

The mirror image of the shift property. Multiply the signal by a complex tone, and the spectrum rotates.

### Modulation property

$$W_N^{-k_0 n} x[n] = e^{j\frac{2\pi}{N}k_0 n} x[n] \longleftrightarrow X[(k - k_0)_N] \quad (4.5)$$

**Proof (same move, other side).**

$$\sum_{n=0}^{N-1} (W_N^{-k_0 n} x[n]) W_N^{kn} = \sum_{n=0}^{N-1} x[n] W_N^{(k-k_0)n} = X[(k - k_0)_N].$$

The  $((\cdot))_N$  appears because  $k - k_0$  may fall outside  $0, \dots, N - 1$  — and the bins are on the clock, so it wraps.

### 4.5.1 The special case $k_0 = N/2$ : multiplying by $(-1)^n$

Put  $k_0 = N/2$ :

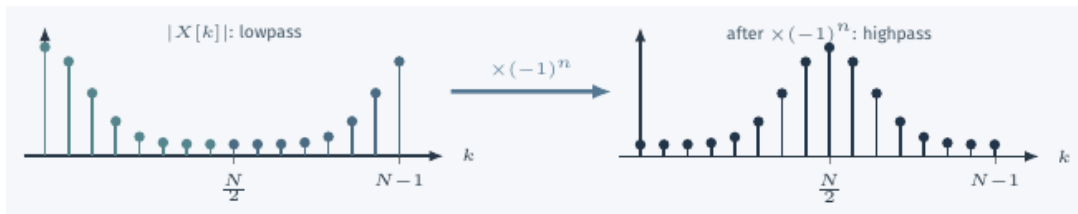
$$e^{j\frac{2\pi}{N} \cdot \frac{N}{2} \cdot n} = e^{j\pi n} = (-1)^n. \quad (4.6)$$

### Flipping every other sign swaps low and high frequencies

$$(-1)^n x[n] \longleftrightarrow X[(k - \frac{N}{2})_N] \quad (4.7)$$

The spectrum slides by *half the clock*. What was at dc ( $k = 0$ ) moves to  $k = N/2$  — the highest frequency — and vice versa.

**A lowpass signal becomes a highpass signal, for free, with no multiplications.**



**Figure 4.3:** Before: energy concentrated at  $k = 0$  and near  $k = N - 1$  (lowpass, per the bin-reading rule). After multiplying by  $(-1)^n$ : energy concentrated around  $k = N/2$  (highpass).

### Why $(-1)^n$ is the fastest thing a sampled signal can do

The sequence  $(-1)^n = +1, -1, +1, -1, \dots$  flips sign every single sample. Nothing can oscillate faster than that on a sampled grid. Its digital frequency is  $\omega = \pi$ , which is bin  $k = N/2$ . Multiplying by it shifts every component by that maximum amount.

## 4.6 Time reversal and conjugation

### 4.6.1 Time reversal

Reversing on a clock means: index 0 stays put, and  $1 \leftrightarrow N-1$ ,  $2 \leftrightarrow N-2$ , and so on. (Position 0 is the “pivot”; the rest flip around it.)

Substitute  $r = -n$  and slide the window:

$$\sum_{n=0}^{N-1} \tilde{x}[-n] W_N^{kn} = \sum_{r=-(N-1)}^0 \tilde{x}[r] W_N^{(-k)r} = \sum_{r=0}^{N-1} \tilde{x}[r] W_N^{(-k)r} = X[(-k)_N]. \quad (4.8)$$

### Time reversal

$$x[(-n)_N] \longleftrightarrow X[(-k)_N] = X[N-k] \quad (\text{for } k = 1, \dots, N-1; \text{ } X[0] \text{ at } k = 0) \quad (4.9)$$

### 4.6.2 Conjugation

Conjugating the sum flips the sign of the kernel's exponent, because  $(W_N^{kn})^* = W_N^{-kn} = W_N^{(-k)n}$ :

$$\sum_{n=0}^{N-1} x^*[n] W_N^{kn} = \left( \sum_{n=0}^{N-1} x[n] W_N^{(-k)n} \right)^* = X^*[(-k)_N]. \quad (4.10)$$

### Conjugation

$$x^*[n] \longleftrightarrow X^*[(-k)_N] = X^*[N-k] \quad (4.11)$$

## 4.7 Conjugate symmetry: why real signals fold

Now the payoff. Suppose  $x[n]$  is **real**. Then  $x^*[n] = x[n]$ , so their DFTs must be equal too. The left side gives  $X[k]$ ; the right side, by the conjugation property, gives  $X^*[(-k)_N]$ . Therefore:

### Real signal $\Rightarrow$ conjugate-symmetric spectrum

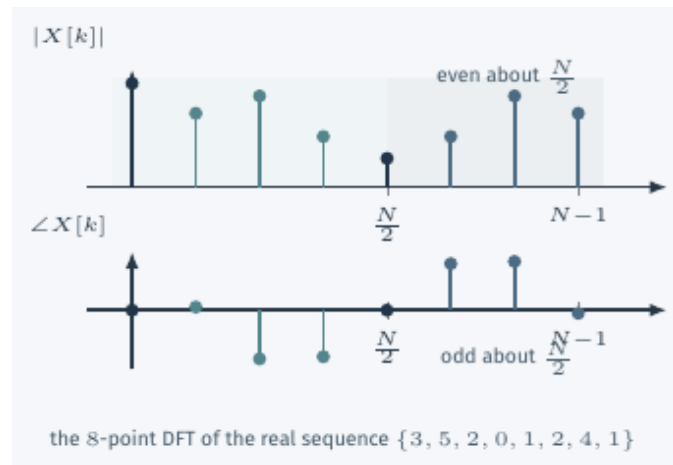
$$X[k] = X^*[(-k)_N] \iff X[N-k] = X^*[k] \quad (4.12)$$

Write  $X[k] = |X[k]|e^{j\angle X[k]}$  and take magnitude and angle of both sides:

### Two symmetries about $k = N/2$

$$\begin{aligned} |X[N-k]| &= |X[k]| & \text{magnitude is **even** about } k = \frac{N}{2} \\ \angle X[N-k] &= -\angle X[k] & \text{phase is **odd** about } k = \frac{N}{2} \end{aligned}$$

Equivalently:  $\text{Re } X[k]$  is even and  $\text{Im } X[k]$  is odd.



**Figure 4.4:** The 8-point DFT of the real sequence  $\{3, 5, 2, 0, 1, 2, 4, 1\}$ . Magnitude even about  $N/2$ ; phase odd about  $N/2$ .

#### Consequences you will be asked about

- **Half the bins are redundant.** Only  $k = 0, 1, \dots, N/2$  carry independent information; the rest are forced. This is why a spectrum analyser plots only 0 to  $f_s/2$ .
- **The bookkeeping balances.**  $N$  real inputs produce  $N/2 + 1$  complex outputs, but  $X[0]$  and  $X[N/2]$  are forced to be *real* (they equal their own conjugates). Count:  $2(N/2 + 1) - 2 = N$  real numbers out,  $N$  real numbers in. ✓
- **Corollary:** if  $\tilde{x}$  is real *and even* (i.e.  $x[(-n)_N] = x[n]$ ), then  $X[k]$  is purely **real**. Combining reversal + conjugation forces  $X[k] = X^*[k]$ .
- **Corollary:** if  $\tilde{x}$  is real and *odd*,  $X[k]$  is purely imaginary.
- **This is why the fold picture of Part I works:** bin  $N - k$  is bin  $k$ 's negative-frequency partner, and for a real signal it carries the same magnitude.

**Worked example 1.** Verify on  $x = \{1, 2, 3, 4\}$ ,  $X = \{10, -2 + 2j, -2, -2 - 2j\}$ .

$N = 4$ . Check  $X[N - 1] = X[3] = -2 - 2j$  and  $X^*[1] = (-2 + 2j)^* = -2 - 2j$ . ✓  
Also  $X[0] = 10$  is real and  $X[N/2] = X[2] = -2$  is real, as forced. ✓

## 4.8 Parseval's relation: energy is re-bookkept

### Parseval's relation for the DFT

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2 \quad (4.13)$$

**Proof (one substitution).** All sums run 0 to  $N - 1$ . Write  $|x[n]|^2 = x[n]x^*[n]$  and replace  $x^*[n]$  by

the conjugate of the IDFT:

$$\begin{aligned}
 \sum_n x[n] x^*[n] &= \sum_n x[n] \cdot \frac{1}{N} \sum_k X^*[k] W_N^{kn} && \text{(conjugating } W_N^{-kn} \text{ gives } W_N^{kn}) \\
 &= \frac{1}{N} \sum_k X^*[k] \underbrace{\sum_n x[n] W_N^{kn}}_{= X[k]} && \text{(swap the two finite sums)} \\
 &= \frac{1}{N} \sum_k |X[k]|^2.
 \end{aligned}$$

**Worked example 1.** Check on  $x = \{1, 2, 3, 4\}$ .

Time side:  $1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$ .

Bin side:  $\frac{1}{4}(10^2 + |-2 + 2j|^2 + |-2|^2 + |-2 - 2j|^2) = \frac{1}{4}(100 + 8 + 4 + 8) = \frac{120}{4} = 30$ . ✓

### The power spectrum

Parseval says  $\frac{|X[k]|^2}{N}$  is **the energy sitting in bin  $k$** . Plotting  $\frac{|X[k]|^2}{N}$  against  $k$  gives the *power spectrum* — the standard way to show “where the energy is”.

### Why it had to be true

The  $N$  tones  $W_N^{-kn}$  are mutually orthogonal (Part I, Section 1.7.2). So the DFT is nothing but a **change of orthogonal basis** in  $\mathbb{C}^N$  — a rotation of the coordinate axes. Rotating your axes does not change the length of a vector; it only re-describes it. Energy is re-bookkept, never created or destroyed. The  $1/N$  is present only because the basis vectors  $W_N^{-kn}$  have length  $\sqrt{N}$  rather than 1.

## 4.9 The property table

| Property                 | Time domain                        | DFT domain                      |
|--------------------------|------------------------------------|---------------------------------|
| linearity                | $a x[n] + b y[n]$                  | $a X[k] + b Y[k]$               |
| periodicity              | $x[((n))_N]$                       | $X[k + N] = X[k]$               |
| circular time shift      | $x[((n - m))_N]$                   | $W_N^{km} X[k]$                 |
| modulation (freq. shift) | $W_N^{-k_0 n} x[n]$                | $X[((k - k_0))_N]$              |
| time reversal            | $x[((-n))_N]$                      | $X[((-k))_N]$                   |
| conjugation              | $x^*[n]$                           | $X^*[((-k))_N]$                 |
| real signal              | $x[n] \in \mathbb{R}$              | $X[N - k] = X^*[k]$             |
| duality                  | $X[n]$ (bins replayed as a signal) | $N x[((-k))_N]$                 |
| circular convolution     | $x \circledast h$                  | $X[k] H[k]$                     |
| multiplication           | $x[n] h[n]$                        | $\frac{1}{N} X \circledast H$   |
| Parseval                 | $\sum_n  x[n] ^2$                  | $= \frac{1}{N} \sum_k  X[k] ^2$ |

**One table, one proof**

Every row above follows from the analysis sum by *the one proof move*: substitute, re-index, and let the clock absorb the wrap. The last two rows (convolution and multiplication) are the rest of this part.

**On duality.** If you take the  $N$  numbers  $X[0], \dots, X[N-1]$  and feed them back into the DFT *as if they were a time signal*, you recover  $N x[(-k)_N]$  — the original signal, time-reversed and scaled by  $N$ . It follows immediately from comparing the DFT and IDFT formulas: they differ only in the sign of the exponent and the  $1/N$ .

## Chapter 5

# Filtering with the DFT

### 5.1 Why convolution matters

From the first half of the course: an LTI system is completely described by its impulse response  $h[n]$ , and the output is the **linear convolution**

$$y[n] = (x * h)[n] = \sum_m x[m] h[n - m]. \quad (5.1)$$

Every filter — smoothing, echo, equalisation, edge detection — is this one sum.

#### Two facts about linear convolution you must know cold

- **Lengths add (minus one).** If  $x$  has length  $L$  and  $h$  has length  $M$ , then  $y = x * h$  has length  $L + M - 1$ , supported on  $n = 0, \dots, L + M - 2$ .
- **Cost.** About  $L \cdot M$  complex multiplications if you compute it directly. For a 3-second audio clip at 48 kHz filtered by a 1000-tap filter, that is  $1.4 \times 10^8$  multiplications — and that is a *short* filter.

#### Why the length is $L + M - 1$

The output is nonzero exactly when the flipped, shifted  $h$  overlaps  $x$  at all. The first overlap is when  $h$ 's left edge meets  $x$ 's left edge ( $n = 0$ ); the last is when  $h$ 's right edge passes  $x$ 's right edge ( $n = L - 1 + M - 1$ ). Count the integers from 0 to  $L + M - 2$ : that is  $L + M - 1$  of them.

#### The dream

In *every* Fourier world so far, convolution in time became multiplication of spectra. If that holds for the DFT, then filtering becomes: transform, multiply  $N$  numbers, transform back. Does it hold? **Almost.** And the “almost” is the whole of the rest of this part.

### 5.2 What multiplying DFTs actually does

Take two length- $N$  sequences, multiply their DFTs bin by bin, and invert. What signal comes out?

$$\begin{aligned}
y[n] &= \frac{1}{N} \sum_{k=0}^{N-1} X[k] H[k] W_N^{-kn} \\
&= \frac{1}{N} \sum_{k=0}^{N-1} \left( \sum_{m=0}^{N-1} x[m] W_N^{km} \right) H[k] W_N^{-kn} && \text{(substitute the definition of } X[k]) \\
&= \sum_{m=0}^{N-1} x[m] \underbrace{\left( \frac{1}{N} \sum_{k=0}^{N-1} H[k] W_N^{-k(n-m)} \right)}_{\text{the IDFT of } H, \text{ evaluated at } n-m} && \text{(swap the sums, collect } W_N^{km-kn}) \quad (5.2)
\end{aligned}$$

The bracket is exactly the IDFT formula for  $h$ , but evaluated at index  $n - m$ . And here is the catch:  $n - m$  **can be negative**, or  $\geq N$ .

What does the IDFT return off the grid? Exactly what Section 4.1 said: the *periodic extension*. Because  $W_N^{-k(n-m+N)} = W_N^{-k(n-m)}$ , the bracket equals  $\tilde{h}[n - m] = h[((n - m))_N]$ .

#### Multiplying DFTs convolves *circularly*

$$y[n] = \sum_{m=0}^{N-1} x[m] h[((n - m))_N] \triangleq (x \circledast h)[n] \quad (5.3)$$

#### The circular convolution theorem (both sequences length $N$ )

$$(x \circledast h)[n] = \sum_{m=0}^{N-1} x[m] h[((n - m))_N] \longleftrightarrow X[k] H[k] \quad (5.4)$$

and dually

$$x[n] h[n] \longleftrightarrow \frac{1}{N} (X \circledast H)[k]. \quad (5.5)$$

#### The wrap arrives uninvited

This is **almost** convolution — but the index into  $h$  wraps around the clock instead of running off the end. Multiplying DFTs does *not* give you the linear convolution you wanted. Fixing that is Section 5.5.

## 5.3 Circular convolution, three ways

The same sum has three equivalent pictures. Different exam questions ask for different ones, so learn all three.

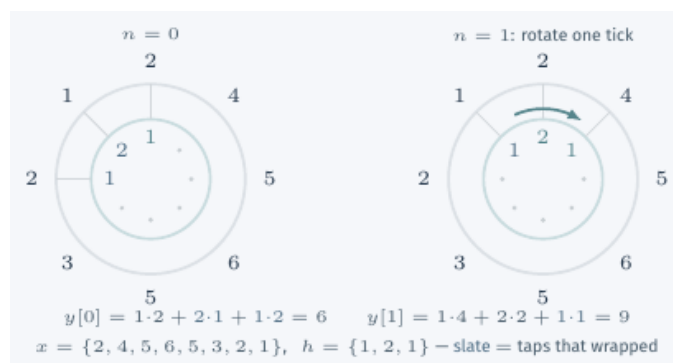
### 5.3.1 Picture 1 — two wheels

Put  $x$  on an outer wheel, running clockwise. Put  $h$  *backwards* on an inner wheel. To get one output value: line the wheels up, multiply across the spokes, and add. To get the next output, rotate the inner wheel one tick.

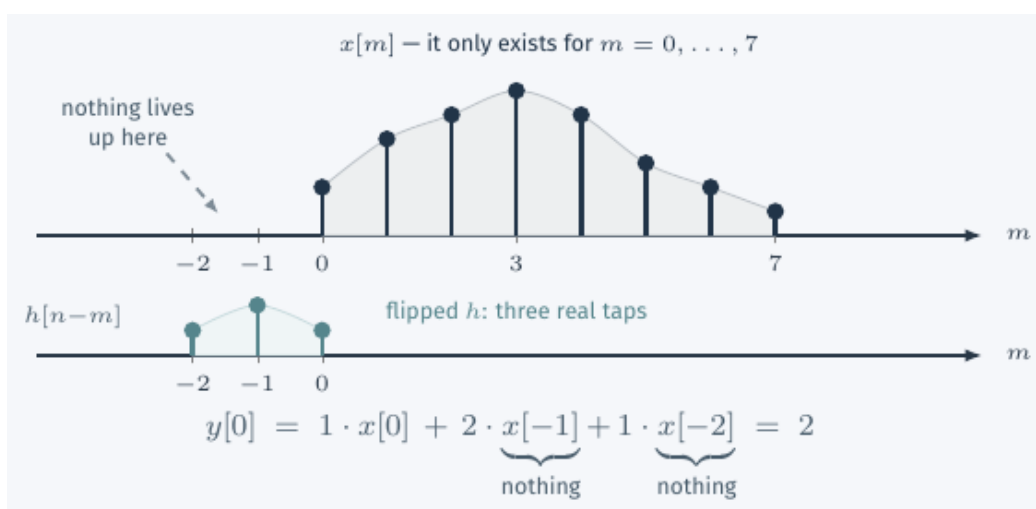
### 5.3.2 Picture 2 — linear convolution, but the taps wrap

Compare what the two convolutions do to the taps that hang off the edge.

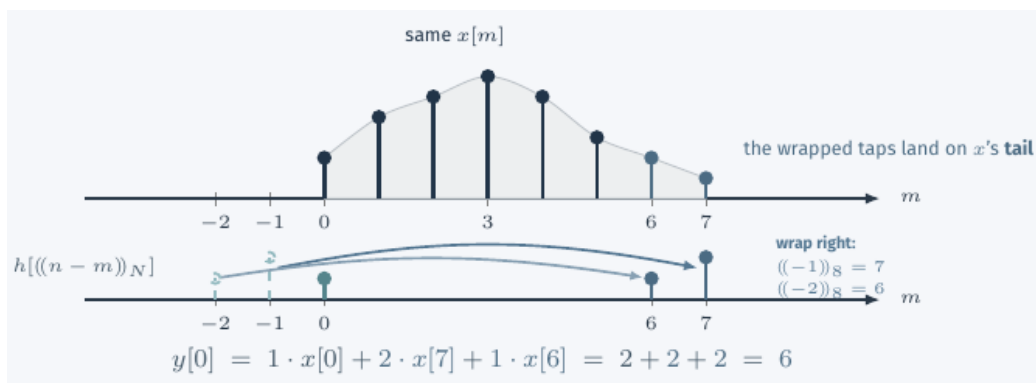




**Figure 5.1:**  $x = \{2, 4, 5, 6, 5, 3, 2, 1\}$  on the outer wheel,  $h = \{1, 2, 1\}$  backwards on the inner.  $y[0] = 1 \cdot 2 + 2 \cdot 1 + 1 \cdot 2 = 6$ ; rotate one tick for  $y[1] = 1 \cdot 4 + 2 \cdot 2 + 1 \cdot 1 = 9$ .



**Figure 5.2: Linear.** At  $n = 0$ , flipping  $h = \{1, 2, 1\}$  puts taps at  $m = 0, -1, -2$ . Nothing lives at  $m = -1$  or  $m = -2$ , so those two terms are silently **dropped**:  $y[0] = 1 \cdot x[0] = 2$ .



**Figure 5.3: Circular.** The same taps, but indices are read on the clock:  $((-1))_8 = 7$  and  $((-2))_8 = 6$ . The off-edge taps are not dropped — they are **moved** onto  $x$ 's tail:  $y[0] = 1 \cdot x[0] + 2 \cdot x[7] + 1 \cdot x[6] = 2 + 2 + 2 = 6$ .

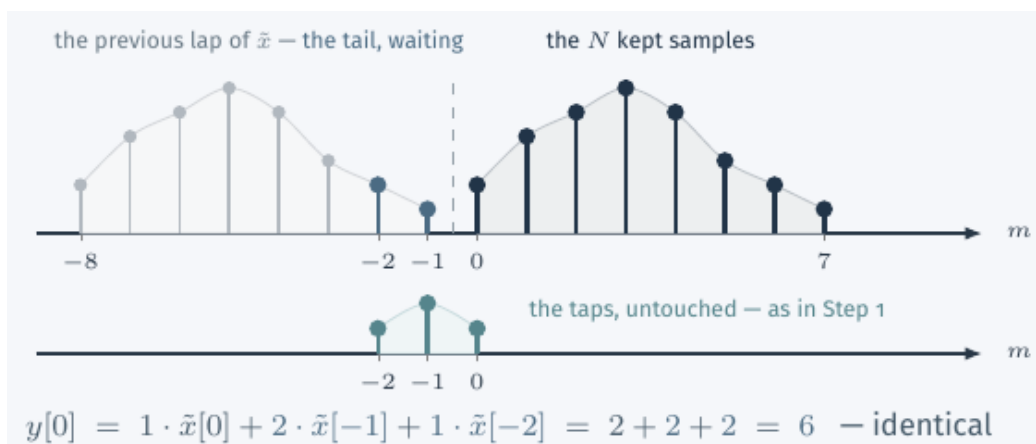
### The entire difference between circular and linear, in one line

**Linear:** taps hanging past the edge multiply nothing, so their terms vanish.

**Circular:** every off-edge tap is folded back into  $0, \dots, N-1$  and multiplies the *other end* of  $x$ .

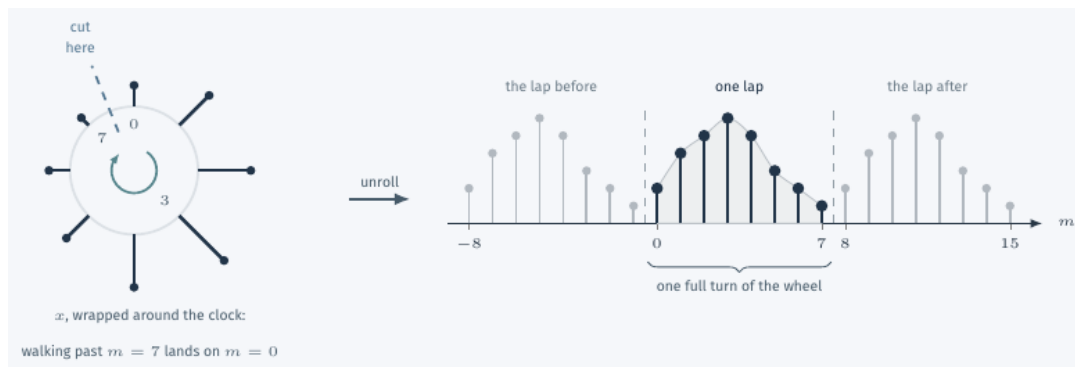
### 5.3.3 Picture 3 — leave the taps alone, extend $x$ instead

Same products, read the other way round. Don't move the taps; instead replace  $x$  by its periodic extension  $\tilde{x}$ , which supplies values at  $m = -1, -2, \dots$  — and those values are precisely  $x[7], x[6], \dots$



**Figure 5.4:** Leave the flipped  $h$  where it fell and slide it along the unrolled  $\tilde{x}$ . The previous lap's tail is waiting at  $m = -1, -2$ , giving the identical  $y[0] = 2 + 2 + 2 = 6$ .

$$\sum_m x[m] h[((n-m))_N] = \sum_j h[j] \tilde{x}[n-j] \quad (5.6)$$



**Figure 5.5:** Going round and round the wheel is the same as walking along the periodic extension  $\tilde{x}$ . Cut the clock and unroll it: lap after lap after lap.

### Two readings of one sum

Either **rotate** the flipped  $h$  on the clock, or **slide** the flipped  $h$  along the unrolled  $\tilde{x}$ . Identical arithmetic. Use whichever the question makes easier — the wheel for small  $N$  by hand, the unrolled picture for seeing *why* the corruption happens.

## 5.4 Worked example: circular $\neq$ linear

**Worked example 1.**  $x = \{2, 4, 5, 6, 5, 3, 2, 1\}$  ( $L = 8$ ),  $h = \{1, 2, 1\}$  ( $M = 3$ ). Compare the linear and the 8-point circular convolution.

**Linear** (length  $L + M - 1 = 8 + 3 - 1 = 10$ ):

$$x * h = \{2, 8, 15, 20, 22, 19, 13, 8, 4, 1\}$$

**Circular**,  $N = 8$  (pad  $h$  to  $\{1, 2, 1, 0, 0, 0, 0, 0\}$ ):

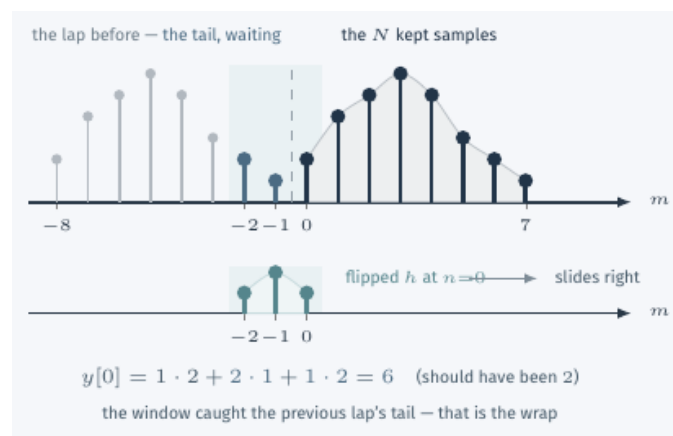
$$x \circledast h = \{6, 9, 15, 20, 22, 19, 13, 8\}$$

Compare them:

| $n$      | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7 | 8 | 9 |
|----------|---|---|----|----|----|----|----|---|---|---|
| linear   | 2 | 8 | 15 | 20 | 22 | 19 | 13 | 8 | 4 | 1 |
| circular | 6 | 9 | 15 | 20 | 22 | 19 | 13 | 8 | — | — |

Six of the eight values agree. The first two are corrupted — and it is not a coincidence that there are exactly **two**: the linear result had a tail of length  $M - 1 = 2$  (the samples 4 and 1 at  $n = 8, 9$ ), and those two got folded back onto  $n = 0, 1$ :

$$6 = 2 + 4, \quad 9 = 8 + 1.$$



**Figure 5.6:** On the unrolled clock, at  $n = 0$  the sliding  $h$  grabs the previous lap's tail samples 1 and 2 — values that should not have been there.

### The exact relationship: circular is linear, folded

$$(x \circledast h)[n] = \sum_{p=-\infty}^{\infty} (x * h)[n + pN], \quad n = 0, \dots, N-1 \quad (5.7)$$

i.e. the circular result is the linear result chopped into blocks of length  $N$  and added on top of each other. This is called **time-domain aliasing**, and it is the exact mirror of the frequency-domain aliasing you get from under-sampling.

### The counting rule — how many outputs are corrupted?

The linear result has length  $L + M - 1$ . Taking  $N$  points wraps the last  $(L + M - 1) - N$  of them back onto the front. So:

$$\text{number of corrupted outputs} = \max(0, L + M - 1 - N), \quad (5.8)$$

and they are always the *first* ones,  $n = 0, 1, \dots$

In the example:  $8 + 3 - 1 - 8 = 2$  corrupted, at  $n = 0, 1$ . ✓

**Worked example 2.** A tiny one you can do in your head.  $x = \{1, 2, 3, 4\}$ ,  $h = \{1, 1\}$ ,  $N = 4$ .

Linear ( $L + M - 1 = 5$ ):  $\{1, 3, 5, 7, 4\}$ .

Circular,  $N = 4$  (pad  $h$  to  $\{1, 1, 0, 0\}$ ):  $y[n] = x[n] + x[(n - 1)_N]$ , so

$$y[0] = x[0] + x[3] = 1 + 4 = 5, \quad y[1] = 2 + 1 = 3, \quad y[2] = 3 + 2 = 5, \quad y[3] = 4 + 3 = 7.$$

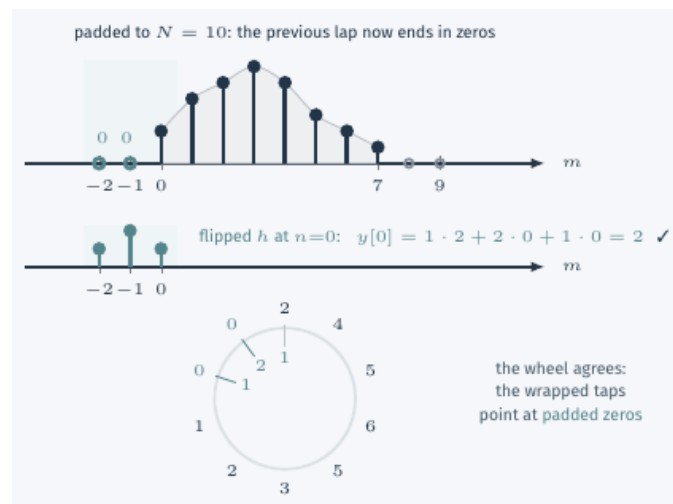
Result  $\{5, 3, 5, 7\}$ . Corrupted count  $= 4 + 2 - 1 - 4 = 1$ , at  $n = 0$  only, and indeed  $5 = 1 + 4$  (the linear tail value 4 folded onto the front). ✓

## 5.5 Zero-padding makes it linear

The fix is now obvious. Circular convolution is the linear result folded mod  $N$ . *If the linear result fits inside  $N$  points, nothing folds.*

### The zero-padding rule

$$\text{Pad both sequences with zeros to length } N \geq L + M - 1 \implies x \circledast h = x * h \quad (5.9)$$



**Figure 5.7:** Padded to  $N = 10$ : the previous lap now ends in zeros, so the wrapped taps grab zeros instead of data.  $y[0] = 1 \cdot 2 + 2 \cdot 0 + 1 \cdot 0 = 2$  — the correct linear value.

### The zeros are not data — they are *room for the tail*

The last  $M - 1$  output samples of a linear convolution are the “ring-out” of the filter after the input has ended. On an  $N$ -point clock with  $N = L$ , there is nowhere for that ring-out to go, so it wraps onto the beginning. Padding to  $N \geq L + M - 1$  gives it somewhere to land.

Compare with Part I: *there*, padding bought a finer picture (cosmetic). *Here*, padding buys correctness (mandatory).

**Worked example 1.** Redo Section 5.4 with padding.

$L = 8$ ,  $M = 3$ , so choose  $N \geq 10$ . Take  $N = 10$ : pad  $x \rightarrow \{2, 4, 5, 6, 5, 3, 2, 1, 0, 0\}$  and  $h \rightarrow \{1, 2, 1, 0, 0, 0, 0, 0, 0, 0\}$ . Then

$$x \circledast_{10} h = \{2, 8, 15, 20, 22, 19, 13, 8, 4, 1\}$$

which is exactly the linear convolution. ✓

#### Common exam slip

$N \geq L + M - 1$ , not  $N \geq L$  and not  $N \geq M$ . And you must pad **both** sequences to the *same*  $N$  — the DFTs have to have bins at the same frequencies before you can multiply them.

If a question also wants an FFT, you need  $N$  to be a power of 2 *as well*: take  $N = 2^{\lceil \log_2(L+M-1) \rceil}$ .

## 5.6 The filtering recipe

### Fast filtering with the DFT (input length $L$ , filter length $M$ )

1. Choose  $N \geq L + M - 1$  (and a power of 2 if you will use a radix-2 FFT). Zero-pad both  $x$  and  $h$  to length  $N$ .
2. Compute  $X[k]$  and  $H[k]$  — two  $N$ -point DFTs.
3. Multiply bin by bin:  $Y[k] = X[k] H[k]$ . ( $N$  complex multiplications.)
4. Invert:  $y[n] = \text{IDFT}\{Y\}$ . The first  $L + M - 1$  samples are the linear convolution; anything beyond is zero.

### Honest accounting — as of the end of Lecture 2, this recipe *loses*

Each  $N$ -point DFT costs about  $N^2$  complex multiplications, and we need three transforms (two forward, one inverse) plus  $N$  multiplications:

$$3N^2 + N \quad \text{versus} \quad L \cdot M \quad \text{for the direct sum.}$$

With  $L = 8$ ,  $M = 3$ : direct is 24 multiplications; the DFT route with  $N = 10$  is about 310. **The transform is the bottleneck**, not the multiply.

This is precisely the problem Part III fixes: the FFT drops each transform to  $\sim \frac{N}{2} \log_2 N$ , and this recipe becomes the *fastest known* way to convolve long sequences.

**In practice.** `scipy.signal.fftconvolve` is exactly this recipe. For streaming audio, where  $x$  is effectively infinite, you cannot pad the whole signal — instead you chop it into blocks and use the *overlap-add* or *overlap-save* variants, which run the recipe chunk by chunk and stitch the ring-outs back together.

## 5.7 Part II summary

### All of Part II

**The theme:** on  $N$  points everything wraps.  $((n))_N = n \bmod N$ .

**Properties:**  $x[((n-m))_N] \leftrightarrow W_N^{km} X[k]$  (magnitude unchanged, linear phase);  $W_N^{-k_0 n} x[n] \leftrightarrow X[((k-k_0))_N]$ ;  $(-1)^n x[n]$  swaps lowpass  $\leftrightarrow$  highpass; real  $x[n] \Rightarrow X[N-k] = X^*[k]$ ;  $\sum_n |x[n]|^2 = \frac{1}{N} \sum_k |X[k]|^2$ .

**The two theorems that matter:**

$$x \circledast h \leftrightarrow X[k]H[k], \quad x[n]h[n] \leftrightarrow \frac{1}{N} X \circledast H$$

$$\text{circular} = \text{linear} \iff N \geq L + M - 1 \quad (\text{zero-pad!})$$

**The three pictures:** the clock (shifts are rotations); two wheels (rotate, multiply, add); sliding on  $\tilde{x}$  (the neighbouring copy leaking into the window *is* the wrap — padding replaces it with zeros).

### PART III

# The Fast Fourier Transform

# Chapter 6

## The need for speed

### 6.1 What the naive DFT costs

Compute the DFT straight from its definition:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad W_N = e^{-j\frac{2\pi}{N}}. \quad (6.1)$$

For *one* value of  $k$  you need

- $N$  complex multiplications ( $x[n]$  times  $W_N^{kn}$ , for  $n = 0, \dots, N-1$ ), and
- $N-1$  complex additions (to add those  $N$  products together).

There are  $N$  values of  $k$ . So the whole transform costs

#### Cost of the naive DFT

$$N^2 \text{ complex multiplications and } N(N-1) \text{ complex additions} \implies O(N^2). \quad (6.2)$$

#### Why $O(N^2)$ is a disaster in practice

Take **one second of CD-quality audio**:  $f_s = 44.1$  kHz, so  $N = 44,100$ .

$$N^2 \approx 1.95 \times 10^9 \approx 2 \text{ billion complex multiplications} \quad (6.3)$$

— for *one second* of sound. Real-time spectral analysis would be impossible; so would JPEG, MP3, MRI reconstruction and 5G.

The problem is the *growth rate*. Double  $N$  and the work goes up **four-fold**.

**A note on “complex” operations.** One complex multiplication  $(a + jb)(c + jd)$  is 4 real multiplications and 2 real additions. One complex addition is 2 real additions. Unless a question says otherwise, count in *complex* operations — that is what the lecture does.

### 6.2 A brief history

- **1805 — Carl Friedrich Gauss.** Developed the radix-2 algorithm in order to interpolate the orbits of asteroids from observations. This *predates* Fourier’s own work on harmonic analysis



by more than a decade. Gauss left it as an unpublished note in his collected works, written in neo-Latin, and nobody noticed for 160 years.

- **1965 — James W. Cooley and John W. Tukey.** Independently rediscovered the algorithm at IBM and Princeton. Their short paper is generally credited with launching modern Digital Signal Processing as a field.
- Many variants exist today (radix-4, split-radix, mixed-radix, Bluestein's chirp-z, Rader's algorithm, Bailey's four-step). This course studies the original **Cooley–Tukey radix-2** algorithm.

### 6.3 The idea: exploit the structure of $W_N$

The naive algorithm treats the  $N^2$  numbers  $W_N^{kn}$  as  $N^2$  *different* numbers. They are not.  $W_N^N = 1$ , so the exponent only matters modulo  $N$ : there are only  $N$  distinct values, each reused  $N$  times. Cooley–Tukey exploits two facts:

#### The two identities the FFT is built on

$$\text{Periodicity: } W_N^{k+N} = W_N^k \quad (6.4)$$

$$\text{Symmetry: } W_N^{k+N/2} = W_N^k W_N^{N/2} = W_N^k e^{-j\pi} = -W_N^k \quad (6.5)$$

$$\text{Halving: } W_N^2 = e^{-j\frac{2\pi}{N} \cdot 2} = e^{-j\frac{2\pi}{N/2}} = W_{N/2} \quad (6.6)$$

#### Read those three lines as pictures on the unit circle

- (6.4): going  $N$  steps around a circle of  $N$  steps returns you home.
- (6.5): going half way around ( $N/2$  steps) is a rotation by  $180^\circ$  — which is just a sign flip. **This is what turns a multiplication into a free subtraction**, and it is where the savings come from.
- (6.6): taking every second step of an  $N$ -step circle is the same as taking every step of an  $N/2$ -step circle. **This is what makes a half-size DFT appear.**

The radix-2 algorithm then:

- successively splits an  $N$ -point DFT into two  $N/2$ -point DFTs, and
- assumes  $N$  is a power of 2 (if not, zero-pad — Section 8.3).

There are two ways to do the splitting, and they give the two standard variants: **Decimation-in-Time (DIT)** and **Decimation-in-Frequency (DIF)**.

## Chapter 7

# Radix-2 Decimation-in-Time (DIT)

### 7.1 The derivation

“Decimation in time” means: **split the input sequence**  $x[n]$  into its even-indexed and odd-indexed samples.

Write  $n = 2r$  for the evens and  $n = 2r + 1$  for the odds, each with  $r = 0, \dots, \frac{N}{2} - 1$ :

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} = \underbrace{\sum_{r=0}^{N/2-1} x[2r] W_N^{2rk}}_{\text{evens}} + \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] W_N^{(2r+1)k}}_{\text{odds}}. \quad (7.1)$$

Now apply the halving identity (6.6) to both:

$$W_N^{2rk} = (W_N^2)^{rk} = W_{N/2}^{rk}, \quad W_N^{(2r+1)k} = W_N^k W_{N/2}^{rk}. \quad (7.2)$$

Pull the  $W_N^k$  (which does not depend on  $r$ ) out of the second sum:

#### The DIT split

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk}}_{G[k]} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{rk}}_{H[k]} \quad (7.3)$$

where

- $G[k]$  is the  $N/2$ -point DFT of the even-indexed samples, and
- $H[k]$  is the  $N/2$ -point DFT of the odd-indexed samples.

#### What we have gained — and the problem left over

An  $N$ -point DFT is now two  $N/2$ -point DFTs plus a bit of glue. But:  $G[k]$  and  $H[k]$  are only *defined* for  $k = 0, \dots, \frac{N}{2} - 1$  (an  $N/2$ -point DFT has  $N/2$  outputs), while  $X[k]$  needs  $k$  up to  $N - 1$ .

So we must extend  $G$  and  $H$  to the upper half.

### 7.2 Filling in the upper half — and the free subtraction

The extension is free: an  $N/2$ -point DFT is periodic with period  $N/2$ , so

$$G[k + \frac{N}{2}] = G[k], \quad H[k + \frac{N}{2}] = H[k]. \quad (7.4)$$

That already lets us write  $X[k] = G[k] + W_N^k H[k]$  for *all*  $k$ , reading  $G, H$  periodically. But we can do better — and “better” is the entire point of the FFT.

For the upper half,  $\frac{N}{2} \leq k < N$ , use the symmetry identity (6.5)  $W_N^{k+N/2} = -W_N^k$ :

$$\begin{aligned} X[k + \frac{N}{2}] &= G[k + \frac{N}{2}] + W_N^{k+N/2} H[k + \frac{N}{2}] \\ &= G[k] - W_N^k H[k]. \end{aligned}$$

### Radix-2 Decimation-in-Time FFT — the butterfly

For  $0 \leq k < \frac{N}{2}$ :

$$\begin{aligned} X[k] &= G[k] + W_N^k H[k] \\ X[k + \frac{N}{2}] &= G[k] - W_N^k H[k] \end{aligned}$$

### Why this is the whole trick

Look at what the two lines share: the product  $W_N^k H[k]$  appears in *both*, once with a + and once with a −.

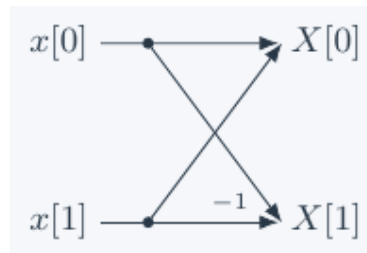
So **one** complex multiplication produces **two** output values. The rest is an addition and a subtraction, which are cheap. That is the “butterfly”, and it is where the factor-of- $N / \log N$  saving comes from.

## 7.3 Butterfly structures, small to large

### 7.3.1 $N = 2$ — the fundamental butterfly

With  $N = 2$ :  $W_2^0 = 1$ , and there is only  $k = 0$ .

$$X[0] = x[0] + x[1]W_2^0 = x[0] + x[1], \quad X[1] = x[0] - x[1]W_2^0 = x[0] - x[1]. \quad (7.5)$$



**Figure 7.1:** The 2-point DFT. This crossing shape is the butterfly; every FFT diagram is made of copies of it. The  $-1$  on the lower branch is the subtraction.

**How to read a signal-flow graph.** A line carries a value from left to right. A number written on a line means “multiply by this”; an unlabelled line means multiply by 1. Where two lines meet at a dot, the values are **added**.

### 7.3.2 $N = 4$

A 4-point DFT is two 2-point DFTs plus one combining stage.

**Stage 1** (two 2-point DFTs):

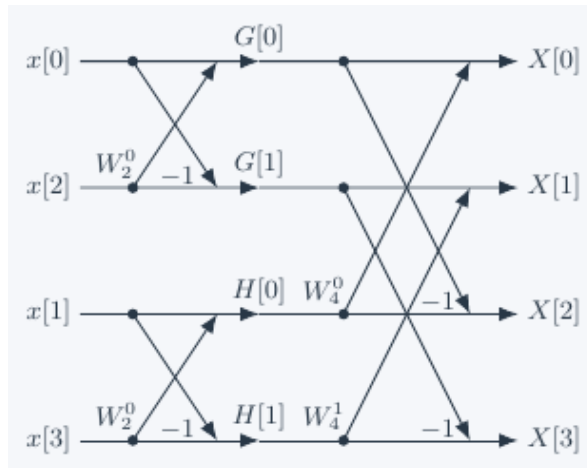
- evens  $\{x[0], x[2]\} \rightarrow G[0] = x[0] + x[2], G[1] = x[0] - x[2];$

- odds  $\{x[1], x[3]\} \rightarrow H[0] = x[1] + x[3], H[1] = x[1] - x[3]$ .

**Stage 2** (combine with twiddles  $W_4^k$ ):

$$\begin{aligned} X[0] &= G[0] + W_4^0 H[0] & X[2] &= G[0] - W_4^0 H[0] \\ X[1] &= G[1] + W_4^1 H[1] & X[3] &= G[1] - W_4^1 H[1] \end{aligned}$$

with  $W_4^0 = 1$  and  $W_4^1 = -j$ .



**Figure 7.2:** The 4-point DIT FFT. Note the input order  $x[0], x[2], x[1], x[3]$  — evens first, then odds.

**Worked example 1.** Compute the 4-point DIT FFT of  $x = \{1, 2, 3, 4\}$  and check against Section 3.4.

Reorder: evens  $\{x[0], x[2]\} = \{1, 3\}$ , odds  $\{x[1], x[3]\} = \{2, 4\}$ .

**Stage 1:**  $G[0] = 1 + 3 = 4$ ,  $G[1] = 1 - 3 = -2$ ;  $H[0] = 2 + 4 = 6$ ,  $H[1] = 2 - 4 = -2$ .

**Stage 2:** ( $W_4^0 = 1$ ,  $W_4^1 = -j$ )

$$\begin{aligned} X[0] &= 4 + 1 \cdot 6 = 10 & X[2] &= 4 - 1 \cdot 6 = -2 \\ X[1] &= -2 + (-j)(-2) = -2 + 2j & X[3] &= -2 - (-j)(-2) = -2 - 2j \end{aligned}$$

Matches the direct computation  $\{10, -2 + 2j, -2, -2 - 2j\}$ . ✓

### 7.3.3 $N = 8$

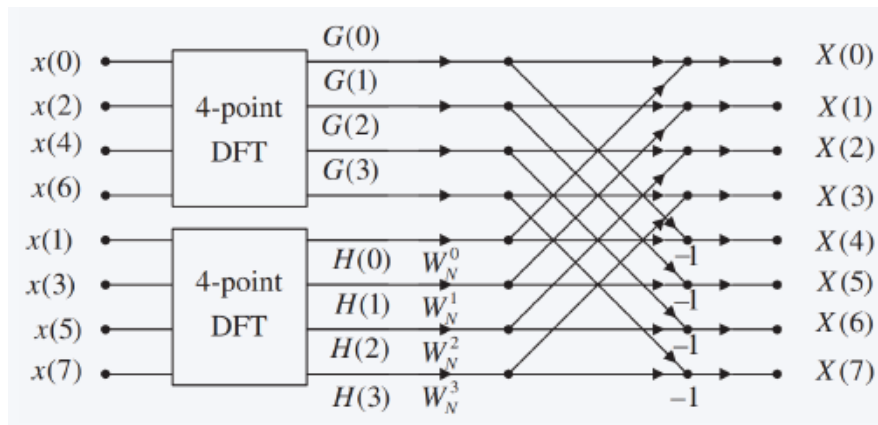
For  $N = 8$  the first split produces two 4-point DFTs, combined with  $W_8^k$ ,  $k = 0, 1, 2, 3$ :

$$X[k] = G[k] + W_8^k H[k], \quad X[k + 4] = G[k] - W_8^k H[k], \quad k = 0, \dots, 3. \quad (7.6)$$

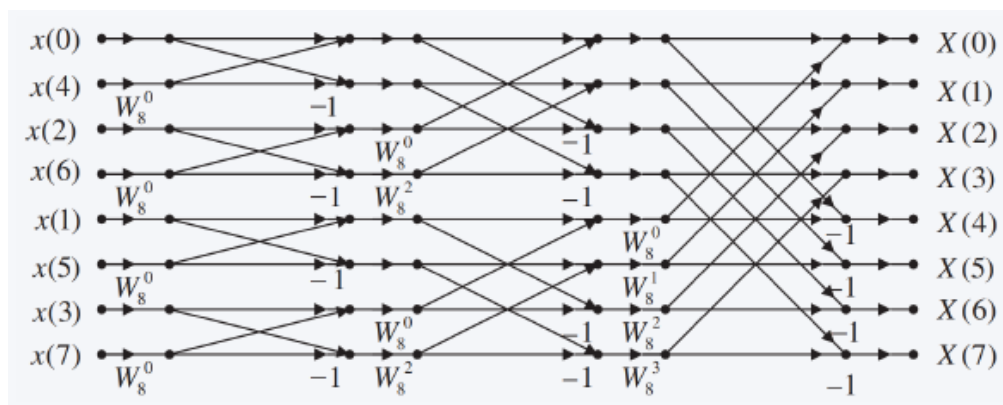
Recurring — expanding each 4-point DFT into two 2-point DFTs — gives the complete structure:

#### The shape of every radix-2 FFT diagram

- There are  $\log_2 N$  **stages** (columns of butterflies).
- Each stage contains exactly  $N/2$  **butterflies**.
- Total butterflies:  $\frac{N}{2} \log_2 N$ . Each is 1 complex multiply, 1 add, 1 subtract.
- In DIT, the twiddles get *denser* as you move right: stage 1 uses only  $W^0$ , the last stage uses  $W_N^0, \dots, W_N^{N/2-1}$ .



**Figure 7.3:** First iteration for  $N = 8$ : the evens  $x[0], x[2], x[4], x[6]$  feed one 4-point DFT and the odds  $x[1], x[3], x[5], x[7]$  feed the other.



**Figure 7.4:** The complete 8-point DIT FFT:  $\log_2 8 = 3$  stages, each with  $N/2 = 4$  butterflies. Inputs are in bit-reversed order; outputs come out in natural order.

## 7.4 Understanding bit-reversed order

**Why does the input come out shuffled?** Because we sorted it, repeatedly, by parity.

For  $N = 8$  the successive splits are:

1. **Split 1:** evens (0, 2, 4, 6) then odds (1, 3, 5, 7).
2. **Split 2:** inside the evens, split again — (0, 4) then (2, 6); inside the odds — (1, 5) then (3, 7).
3. **Split 3:** single elements. Final order:  $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$ .

Now look at what each split *tests*:

- “even before odd” = sorting by the **rightmost** bit (0 before 1);
- “even-even before odd-even” = sorting by the **second-rightmost** bit;
- and so on, up to the leftmost bit.

Sorting by bits from the right instead of the left is precisely sorting by the **bit-reversed** index.

| Original index | Binary | Bit-reversed binary | New position |
|----------------|--------|---------------------|--------------|
| 0              | 000    | 000                 | 0            |
| 1              | 001    | 100                 | 4            |
| 2              | 010    | 010                 | 2            |
| 3              | 011    | 110                 | 6            |
| 4              | 100    | 001                 | 1            |
| 5              | 101    | 101                 | 5            |
| 6              | 110    | 011                 | 3            |
| 7              | 111    | 111                 | 7            |

### How to bit-reverse, by hand, in the exam

1. Write the index in binary using exactly  $\log_2 N$  bits (pad with leading zeros!).
2. Read the bits backwards.
3. Convert back to decimal.

For  $N = 8$  (3 bits):  $3 = 011_2 \rightarrow 110_2 = 6$ . So  $x[3]$  goes to position 6.

**Shortcut:** bit reversal is its own inverse (reversing twice gives the original), so the permutation is a set of *swaps*. For  $N = 8$  only two swaps are needed:  $1 \leftrightarrow 4$  and  $3 \leftrightarrow 6$ . Indices 0, 2, 5, 7 are palindromes and stay put.

### Padding matters

For  $N = 16$  you must use 4 bits:  $3 = 0011_2 \rightarrow 1100_2 = 12$ , *not*  $011 \rightarrow 110 = 6$ . Forgetting the leading zeros is the standard mistake.

## 7.5 Pseudocode and complexity

### Iterative radix-2 DIT FFT

```

Bit-reverse the array  $x$  of length  $N$  ( $N = 2^v$ )
for  $s = 1$  to  $\log_2 N$ :
     $M \leftarrow 2^s$ 
     $W_M \leftarrow e^{-j2\pi/M}$ 
    for  $l = 0$  to  $N - M$  step  $M$ :
```

// for each stage  
// block size: 2, 4, 8, ..., N  
// each  $M$ -point block

```

W ← 1
for k = 0 to M/2 - 1:
    g ← x[l + k]
    h ← W · x[l + k + M/2]
    x[l + k] ← g + h
    x[l + k + M/2] ← g - h
    W ← W · WM

```

// the butterfly  
// G[k]  
// twiddle × H[k]  
// X[k]  
// X[k + M/2]  
// next twiddle

### Reading the pseudocode

- It is **in-place**: the answer overwrites the input array. No second array is needed. That is why bit-reversal must happen *first* — the algorithm assumes the even-indexed samples are already sitting next to each other.
- The twiddle is built by repeated multiplication ( $W \leftarrow W \cdot W_M$ ) rather than recomputed with a cosine each time — cheaper.
- Stage  $s$  works on blocks of size  $M = 2^s$ : stage 1 does 2-point DFTs, stage 2 does 4-point DFTs, and so on until one  $N$ -point DFT.

### 7.5.1 Complexity analysis

| Part                   | Count                       | Why                                      |
|------------------------|-----------------------------|------------------------------------------|
| Bit-reversal           | $O(N)$                      | one pass, swapping $x[k]$ with $x[k']$   |
| Outer loop (stages)    | $\log_2 N$                  | each halves the problem                  |
| Inner loops together   | $N/2$ butterflies per stage | they cover all $N$ elements, 2 at a time |
| <b>Multiplications</b> | $\frac{N}{2} \log_2 N$      | one per butterfly                        |
| <b>Additions</b>       | $N \log_2 N$                | one add + one subtract per butterfly     |

### Total complexity

$$O(N \log_2 N) \quad (7.7)$$

compared with  $O(N^2)$  for the naive DFT.

### How big is the win, really?

The speed-up is

$$\frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N},$$

which **grows** with  $N$ :

| $N$    | naive $N^2$ mults  | FFT $\frac{N}{2} \log_2 N$ mults | speed-up |
|--------|--------------------|----------------------------------|----------|
| 8      | 64                 | 12                               | 5×       |
| 1,024  | 1,048,576          | 5,120                            | 205×     |
| 4,096  | 16,777,216         | 24,576                           | 683×     |
| 65,536 | $4.29 \times 10^9$ | 524,288                          | 8,192×   |

That is why the FFT did not merely make things faster — it made a whole class of problems possible.

**Where the  $\log N$  comes from — the divide-and-conquer recurrence**

Let  $T(N)$  be the cost of an  $N$ -point FFT. The algorithm does two half-size FFTs plus  $N/2$  twiddle multiplications and  $N$  additions to combine:

$$T(N) = 2 T(N/2) + O(N), \quad T(1) = O(1). \quad (7.8)$$

This is the same recurrence as merge sort, and it solves to  $T(N) = O(N \log N)$ . You can halve  $N$  only  $\log_2 N$  times before you reach single points; each level of halving costs  $O(N)$  total work.



## Chapter 8

# Radix-2 Decimation-in-Frequency (DIF)

### 8.1 The derivation

DIT split the *input*. DIF splits the *output* — but it starts by splitting the summation range into a first half and a second half of *time* indices:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=N/2}^{N-1} x[n] W_N^{kn} \\ &= \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=0}^{N/2-1} x[n + \frac{N}{2}] W_N^{k(n+N/2)} \quad (\text{re-index the second sum}) \end{aligned}$$

Now use  $W_N^{kN/2} = e^{-j\frac{2\pi}{N}k\frac{N}{2}} = e^{-j\pi k} = (-1)^k$ :

$$X[k] = \sum_{n=0}^{N/2-1} \left( x[n] + (-1)^k x[n + \frac{N}{2}] \right) W_N^{kn}. \quad (8.1)$$

#### Why the next step is to split $k$

The bracket depends on  $k$  *only through the sign*  $(-1)^k$ , i.e. only through whether  $k$  is even or odd. So splitting the outputs into even  $k$  and odd  $k$  makes the bracket a constant — that is “decimation in frequency”.

#### 8.1.1 Even outputs, $k = 2m$

$(-1)^{2m} = +1$ , and  $W_N^{2mn} = W_{N/2}^{mn}$ :

$$X[2m] = \sum_{n=0}^{N/2-1} \underbrace{(x[n] + x[n + \frac{N}{2}])}_{a[n]} W_{N/2}^{mn} = \text{DFT}_{N/2}\{a[n]\}. \quad (8.2)$$

#### 8.1.2 Odd outputs, $k = 2m + 1$

$(-1)^{2m+1} = -1$ , and  $W_N^{(2m+1)n} = W_N^n W_N^{2mn} = W_N^n W_{N/2}^{mn}$ :

$$X[2m + 1] = \sum_{n=0}^{N/2-1} \underbrace{(x[n] - x[n + \frac{N}{2}])}_{b[n]} W_N^n W_{N/2}^{mn} = \text{DFT}_{N/2}\{b[n]\}. \quad (8.3)$$

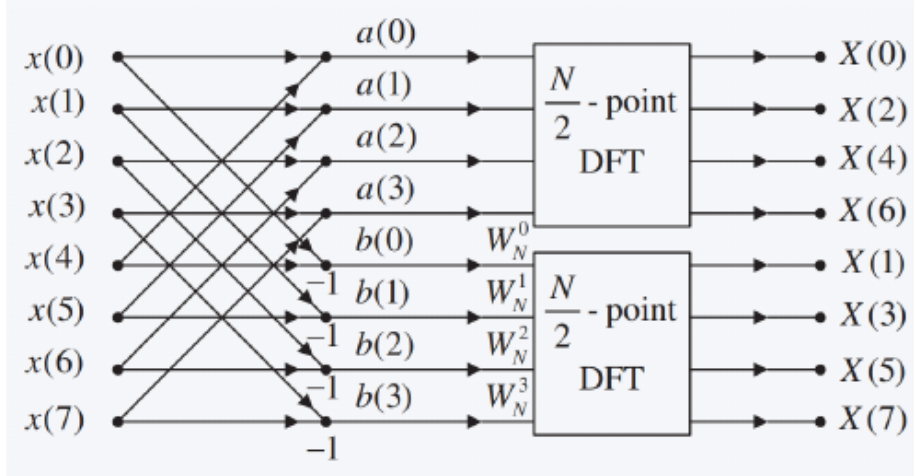
### Radix-2 Decimation-in-Frequency FFT — the butterfly

For  $n = 0, \dots, \frac{N}{2} - 1$ :

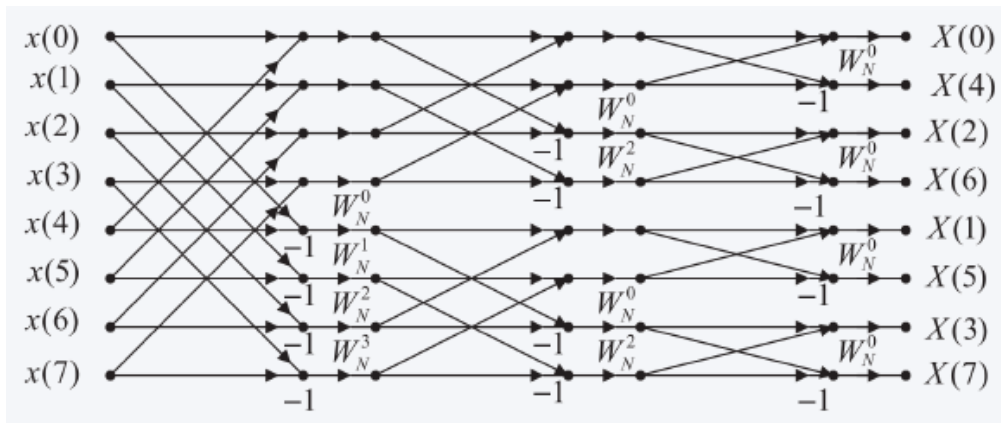
$$a[n] = x[n] + x[n + \frac{N}{2}]$$

$$b[n] = (x[n] - x[n + \frac{N}{2}]) W_N^n$$

Then  $X[\text{even } k] = \text{DFT}_{N/2}\{a\}$  and  $X[\text{odd } k] = \text{DFT}_{N/2}\{b\}$ .



**Figure 8.1:** The DIF structure for  $N = 8$ : additions and subtractions happen first, then the multiplication by  $W_N^n$ , then two  $N/2$ -point DFTs. The outputs emerge grouped as evens  $X[0], X[2], X[4], X[6]$  and odds  $X[1], X[3], X[5], X[7]$ .



**Figure 8.2:** The complete 8-point DIF FFT. Inputs are in natural order; outputs emerge in bit-reversed order — exactly the reverse of DIT.

**Worked example 1.** Compute the 4-point DIF FFT of  $x = \{1, 2, 3, 4\}$ .

$N = 4$ , so  $N/2 = 2$  and  $n = 0, 1$ .

**Stage 1:**

$$a[0] = x[0] + x[2] = 1 + 3 = 4,$$

$$a[1] = x[1] + x[3] = 2 + 4 = 6$$

$$b[0] = (x[0] - x[2])W_4^0 = (1 - 3)(1) = -2,$$

$$b[1] = (x[1] - x[3])W_4^1 = (2 - 4)(-j) = 2j$$

**Stage 2:** two 2-point DFTs.

$$X[0] = a[0] + a[1] = 4 + 6 = 10,$$

$$X[1] = b[0] + b[1] = -2 + 2j,$$

$$X[2] = a[0] - a[1] = 4 - 6 = -2$$

$$X[3] = b[0] - b[1] = -2 - 2j$$

Same answer as DIT and as the direct sum. ✓

Note the order in which the outputs came out:  $X[0], X[2], X[1], X[3]$  — that is bit-reversed order.

## 8.2 DIT versus DIF — side by side

|                        | DIT                                                      | DIF                                                  |
|------------------------|----------------------------------------------------------|------------------------------------------------------|
| What gets split        | the input sequence, by parity of $n$                     | the output bins, by parity of $k$                    |
| Butterfly order        | multiply by twiddle <i>first</i> , then add/subtract     | add/subtract <i>first</i> , then multiply by twiddle |
| Input order            | bit-reversed                                             | natural                                              |
| Output order           | natural                                                  | bit-reversed                                         |
| Where the twiddles are | dense in the <i>last</i> stages                          | dense in the <i>first</i> stages                     |
| Operation count        | $\frac{N}{2} \log_2 N$ mults, $N \log_2 N$ adds          | <i>identical</i>                                     |
| Relationship           | each is the transpose (flow-graph reversal) of the other |                                                      |

### Which to use?

Computationally they are equivalent. The choice is about **data ordering**:

- If your data arrives in natural order and you can tolerate scrambled output — use DIF.
- If you need natural-order output — use DIT (and pay for one bit-reversal pass at the start).
- **Neat trick:** in the filtering recipe of Section 5.6 you never look at  $X[k]$  or  $H[k]$  individually — you only multiply them bin by bin. So use a DIF forward transform (natural in, scrambled out) and a DIT inverse transform (scrambled in, natural out), and *no bit-reversal pass is needed at all*.

### Why DIT and DIF give the same count

Both do the same total amount of splitting:  $\log_2 N$  levels,  $N/2$  butterflies per level. They differ only in *when* the twiddle multiplication happens relative to the add/subtract. If you draw a DIT flow graph, reverse every arrow, and swap inputs with outputs, you get the DIF flow graph. Reversing a flow graph never changes the number of multiplications or additions.

## 8.3 Zero-padding for radix-2

### The radix-2 requirement

Radix-2 FFT algorithms require the length to be *exactly* a power of two,  $N = 2^m$ . You cannot “mostly” halve a sequence of length 7.

**The problem.** Real data comes in whatever length it comes in —  $M$  samples where  $M$  is usually not a power of 2. If you insist on using exactly  $M$  samples you are stuck with the  $O(M^2)$  naive DFT.

**The solution.** Append zeros until you reach the next power of two:

$$N = 2^{\lceil \log_2 M \rceil}. \quad (8.4)$$

#### What zero-padding does and does not do here

- ✓ **Bonus:** it interpolates the spectrum, giving a smoother plot without altering the underlying  $X(e^{j\omega})$  (Part I, Section 3.3).
- × **Warning:** if  $x[n]$  is genuinely *periodic* and you are treating the DFT as a Fourier series, padding **changes the period and the shape** of the periodic signal. The coefficients you get are then the coefficients of a different signal.

**Worked example 1.**  $M = 1000$  samples. What  $N$  do you pad to, and what does it cost?

$\log_2 1000 \approx 9.97$ , so  $\lceil \cdot \rceil = 10$  and  $N = 2^{10} = 1024$ . You add 24 zeros.

Cost:  $\frac{1024}{2} \times 10 = 5120$  multiplications, versus  $1000^2 = 10^6$  for a direct 1000-point DFT. Padding increased  $N$  and still made it  $195\times$  faster.

## 8.4 Can we avoid zero-padding? (Beyond radix-2)

The lecture ends with this question, so here is the short answer to each option — enough to write a confident paragraph in an exam.

| Method                             | Works when              | Idea                                                                                                                                                                                           |
|------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Radix-4                            | $N = 4^m$               | Split into <i>four</i> sub-sequences instead of two. Fewer stages ( $\log_4 N$ ), and the internal twiddles are $\pm 1, \pm j$ , which are free. About 25% fewer multiplications than radix-2. |
| Split-radix                        | $N = 2^m$               | Mixes radix-2 for the even outputs and radix-4 for the odd ones. Lowest known operation count for a power-of-two FFT.                                                                          |
| Mixed-radix (general Cooley–Tukey) | $N = N_1 N_2$ composite | Factorise $N$ any way you like and recurse. Handles e.g. $N = 12 = 3 \times 4$ .                                                                                                               |
| Rader’s algorithm                  | $N$ prime               | Re-expresses a prime-length DFT as a length- $(N - 1)$ <i>circular convolution</i> , which is then done by any FFT.                                                                            |
| Bluestein (chirp-z)                | <b>any</b> $N$          | Rewrites $kn = \frac{k^2 + n^2 - (k-n)^2}{2}$ to turn the DFT into a convolution, which is padded to a convenient power of two. Cost $O(N \log N)$ for arbitrary $N$ .                         |

**The honest answer.** Yes, you can compute an  $O(N \log N)$  transform for any  $N$  at all, and library FFTs (FFTW, `numpy.fft`) do exactly that by choosing among these algorithms automatically. Zero-padding to a power of two is still the common choice because it is simple, because power-of-two FFTs are the most heavily optimised, and because in signal processing padding is usually harmless (Part I) or actively required (Part II).

## 8.5 Part III summary

### All of Part III

**Naive DFT:**  $N^2$  complex mults,  $N(N-1)$  complex adds  $\Rightarrow O(N^2)$ .

**Key identities:**  $W_N^{k+N/2} = -W_N^k$  (symmetry, gives the free subtraction);  $W_N^2 = W_{N/2}$  (halving, makes a half-size DFT appear).

**DIT:** split input by parity of  $n$ .  $X[k] = G[k] + W_N^k H[k]$ ,  $X[k + \frac{N}{2}] = G[k] - W_N^k H[k]$  for  $0 \leq k < \frac{N}{2}$ . Input bit-reversed, output natural.

**DIF:** split output by parity of  $k$ .  $a[n] = x[n] + x[n + \frac{N}{2}]$ ,  $b[n] = (x[n] - x[n + \frac{N}{2}])W_N^n$ ; then  $X[2m] = \text{DFT}_{N/2}\{a\}$ ,  $X[2m+1] = \text{DFT}_{N/2}\{b\}$ . Input natural, output bit-reversed.

**Structure:**  $\log_2 N$  stages  $\times \frac{N}{2}$  butterflies.

**Cost:**  $\frac{N}{2} \log_2 N$  complex mults,  $N \log_2 N$  complex adds  $\Rightarrow O(N \log_2 N)$ . Speed-up over naive  $= \frac{2N}{\log_2 N}$ .

**Radix-2 needs**  $N = 2^m$ ; otherwise pad to  $N = 2^{\lceil \log_2 M \rceil}$ .

PART IV

## Revision sheet

# Chapter 9

## Everything on a few pages

This part assumes you have read Parts I–III. It is deliberately dense: formulas, the one-line reason each is true, and the checks that catch mistakes.

### 9.1 The formula sheet

#### The four transforms

|      |                                                                           |                                                                                  |
|------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| CTFS | $c_k = \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\omega_0 t} dt$                | $x(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t}$                          |
| CTFT | $X(j\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt$             | $x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega$ |
| DTFT | $X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$          | $x[n] = \frac{1}{2\pi} \int_{2\pi} X(e^{j\omega}) e^{j\omega n} d\omega$         |
| DFS  | $a_k = \frac{1}{N} \sum_{n=0}^{N-1} \tilde{x}[n] e^{-j\frac{2\pi}{N} kn}$ | $\tilde{x}[n] = \sum_{k=0}^{N-1} a_k e^{j\frac{2\pi}{N} kn}$                     |
| DFT  | $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$                                   | $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}$                             |

with  $\omega_0 = 2\pi/T_0$  and  $W_N = e^{-j2\pi/N}$ .

#### The three bridges

$$c_k = \frac{1}{T_0} X[k], \quad X[k] = N a_k, \quad X[k] = X(e^{j\omega}) \Big|_{\omega=\frac{2\pi}{N}k}$$

#### Frequency axes — know all three rulers

$$\underbrace{k}_{\text{bin}} \longrightarrow \underbrace{\omega_k = \frac{2\pi k}{N}}_{\text{rad/sample}} \longrightarrow \underbrace{\Omega_k = \frac{\omega_k}{T_s} = k \frac{\omega_s}{N}}_{\text{rad/s}} \longrightarrow \underbrace{f_k = k \frac{f_s}{N}}_{\text{Hz}}$$

$$\Delta f = \frac{f_s}{N} = \frac{1}{NT_s} = \frac{1}{T_0} \quad \omega = \pi \iff f = \frac{f_s}{2}$$

For  $k > N/2$  replace  $k$  by  $k - N$  to get the (negative) folded frequency.

### The property table

|                     |                                                                          |
|---------------------|--------------------------------------------------------------------------|
| $a x[n] + b y[n]$   | $a X[k] + b Y[k]$                                                        |
| $x[((n-m))_N]$      | $W_N^{km} X[k]$ (magnitude unchanged; phase $-\frac{2\pi}{N} km$ )       |
| $W_N^{-k_0 n} x[n]$ | $X[((k-k_0))_N]$                                                         |
| $(-1)^n x[n]$       | $X[\left((k-\frac{N}{2})\right)_N]$ (lowpass $\leftrightarrow$ highpass) |
| $x[(-n))_N]$        | $X[((-k))_N]$                                                            |
| $x^*[n]$            | $X^*[((-k))_N]$                                                          |
| $x[n]$ real         | $X[N-k] = X^*[k]$ : $ X $ even, $\angle X$ odd about $N/2$               |
| $x$ real & even     | $X[k]$ real ( $x$ real & odd $\Rightarrow X[k]$ imaginary)               |
| $X[n]$ (duality)    | $N x[(-k))_N]$                                                           |
| $x \otimes h$       | $X[k] H[k]$                                                              |
| $x[n] h[n]$         | $\frac{1}{N} X \otimes H$                                                |
| $\sum_n  x[n] ^2$   | $= \frac{1}{N} \sum_k  X[k] ^2$ (Parseval)                               |

### Convolution

Linear:  $y = x * h$  has length  $L + M - 1$ ; direct cost  $\approx LM$ .  
Circular:  $(x \otimes h)[n] = \sum_{m=0}^{N-1} x[m] h[((n-m))_N]$ , length  $N$ .  
Relation:  $(x \otimes h)[n] = \sum_p (x * h)[n + pN]$  — linear, folded mod  $N$ .  
Corrupted outputs:  $\max(0, L + M - 1 - N)$ , always the first ones.  
**Equality condition:**  $N \geq L + M - 1$  (zero-pad both).

### FFT

Naive DFT:  $N^2$  complex mults,  $N(N-1)$  adds.  
Radix-2:  $\log_2 N$  stages  $\times \frac{N}{2}$  butterflies  $= \frac{N}{2} \log_2 N$  mults,  $N \log_2 N$  adds.  
Speed-up  $= \frac{2N}{\log_2 N}$ .  
**DIT** ( $0 \leq k < \frac{N}{2}$ ):  $X[k] = G[k] + W_N^k H[k]$ ,  $X[k + \frac{N}{2}] = G[k] - W_N^k H[k]$ ,  $G = \text{DFT}_{N/2}\{\text{evens}\}$ ,  $H = \text{DFT}_{N/2}\{\text{odds}\}$ . *Input bit-reversed, output natural.*  
**DIF** ( $0 \leq n < \frac{N}{2}$ ):  $a[n] = x[n] + x[n + \frac{N}{2}]$ ,  $b[n] = (x[n] - x[n + \frac{N}{2}]) W_N^n$ ;  $X[2m] = \text{DFT}_{N/2}\{a\}$ ,  $X[2m+1] = \text{DFT}_{N/2}\{b\}$ . *Input natural, output bit-reversed.*  
Key identities:  $W_N^{k+N/2} = -W_N^k$ ,  $W_N^2 = W_{N/2}$ ,  $W_N^{kN/2} = (-1)^k$ ,  $W_N^{N/4} = -j$ .

## 9.2 The twiddle values you should know by sight

|           |              |                                   |              |                                    |
|-----------|--------------|-----------------------------------|--------------|------------------------------------|
| $N = 2$ : | $W_2^0 = 1$  | $W_2^1 = -1$                      |              |                                    |
| $N = 4$ : | $W_4^0 = 1$  | $W_4^1 = -j$                      | $W_4^2 = -1$ | $W_4^3 = +j$                       |
| $N = 8$ : | $W_8^0 = 1$  | $W_8^1 = \frac{1}{\sqrt{2}}(1-j)$ | $W_8^2 = -j$ | $W_8^3 = -\frac{1}{\sqrt{2}}(1+j)$ |
|           | $W_8^4 = -1$ | $W_8^5 = -W_8^1$                  | $W_8^6 = +j$ | $W_8^7 = -W_8^3$                   |

### The three shortcuts that save you in an exam

- $W_N^{N/2} = -1$  always.  $W_N^{N/4} = -j$  always.  $W_N^0 = 1$  always.
- $W_N^{k+N/2} = -W_N^k$ : once you know the first half you know the second half.



3. Exponents are read **mod**  $N$ :  $W_8^{11} = W_8^3$ .

### 9.3 Ten checks that catch nearly every arithmetic slip

1.  $X[0] = \sum_n x[n]$ . (Set  $k = 0$ ; all exponentials become 1.)
2.  $x[0] = \frac{1}{N} \sum_k X[k]$ . (Set  $n = 0$  in the IDFT.)
3. If  $N$  is even,  $X[N/2] = \sum_n (-1)^n x[n]$ . (Since  $W_N^{Nn/2} = (-1)^n$ .)
4. If  $x$  is real,  $X[N - k] = X^*[k]$ ; in particular  $X[0]$  and  $X[N/2]$  are real.
5. If  $x$  is real and even,  $X$  is real. If real and odd,  $X$  is imaginary.
6. Parseval:  $\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$ .
7. A circular shift must leave  $|X[k]|$  unchanged.
8.  $\sum_n y[n]$  for  $y = x * h$  (or  $x \circledast h$ ) equals  $(\sum_n x[n])(\sum_n h[n])$ , because  $Y[0] = X[0]H[0]$ .
9. A linear convolution must have length exactly  $L + M - 1$ .
10. After an FFT, the outputs must match the direct DFT for at least one bin — check  $k = 0$ .

### 9.4 The intuitions, restated in one line each

- **Winding.** The transform winds the signal around the origin at rate  $\omega$  and reports the centre of mass. Mass off-centre = the signal contains that rate.
- **Why the integral became a sum.** A sampled signal has nothing between the samples, so averaging over a continuum becomes averaging over  $N$  numbers.
- **Why the spectrum is  $2\pi$ -periodic.** Turning each sample by  $\omega + 2\pi$  instead of  $\omega$  puts every point in the same place. Samples cannot tell the two rates apart.
- **Why only  $N$  bins.** Harmonic  $k$  and harmonic  $k + N$  agree at every sample instant.
- **Where the negative frequencies went.** Onto bins  $N/2 \rightarrow N - 1$ , by periodicity.
- **Why zero-padding does not add information.** Adding zeros adds nothing to the DTFT sum; you are just plotting the same curve at more points.
- **Why shifts are phase-only.**  $|W_N^{km}| = 1$ ; a delay is a rotation, and rotations do not change lengths.
- **Why Parseval holds.** The DFT is a rotation of orthogonal axes in  $\mathbb{C}^N$ ; rotations preserve length.
- **Why multiplying DFTs wraps.** The IDFT off the index grid returns the *periodic extension*, so  $h[n - m]$  becomes  $h[(n - m)_N]$ .
- **Why padding fixes it.** The linear result is  $L + M - 1$  long; give it  $N \geq L + M - 1$  slots and nothing has to fold back.
- **Why the FFT is fast.**  $W_N^{k+N/2} = -W_N^k$  turns one multiplication into two outputs, and  $W_N^2 = W_{N/2}$  lets you recurse.  $\log_2 N$  levels,  $O(N)$  work per level.

### 9.5 The five traps

#### Ranked by how often they cost marks

1. **Circular vs linear.** Multiplying DFTs gives *circular* convolution. If the question says “filter”, “LTI output” or “ $y = x * h$ ”, you must pad to  $N \geq L + M - 1$ .
2. **Bin  $N - 1$  is a low frequency,** not a high one. Highest is at  $k = N/2$ .
3. **Sign of the exponent.** Forward DFT:  $W_N^{+kn} = e^{-j2\pi kn/N}$ . Inverse:  $W_N^{-kn}$  and a  $1/N$ .
4. **Bit-reversal needs the right number of bits.**  $\log_2 N$  bits, with leading zeros.
5. **mod  $N$  of a negative index** lands in  $[0, N - 1]$ :  $((-1))_8 = 7$ , not  $-1$ .

**PART V**

# **Common question patterns**

# Chapter 10

## The playbook

Every question in the CT and in the on-syllabus part of the term final falls into one of the patterns below. Each pattern gets: what it looks like, the exact steps, and the trap.

Use this part as a lookup table during revision: read the pattern name, cover the steps, and see if you can reproduce them.

### Index of patterns

| #   | Pattern                                                          | Seen in            |
|-----|------------------------------------------------------------------|--------------------|
| P1  | Compute a small DFT by hand                                      | term final, CT4 Q1 |
| P2  | Convert bins $\leftrightarrow$ frequency (rad/sample, rad/s, Hz) | standard           |
| P3  | Hand-run a radix-2 FFT and draw the butterfly                    | CT4 Q1(a)          |
| P4  | Interpret a DFT result physically                                | CT4 Q1(b,c)        |
| P5  | Zero-padding: what changes and what does not                     | CT4 Q1(c)          |
| P6  | “What if you break a step of the algorithm?”                     | CT4 Q1(d)          |
| P7  | Derive a higher-radix or generalised FFT                         | CT4 Q2             |
| P8  | Turn a problem into a convolution, then FFT it                   | CT4 Q3             |
| P9  | Extend the DFT to more dimensions                                | CT4 Q4             |
| P10 | Compare DFT vs FFT complexity (essay + numbers)                  | term final Q4(c)   |
| P11 | Circular vs linear convolution                                   | standard           |
| P12 | Use a property to get a new DFT from a known one                 | standard           |
| P13 | Prove a DFT property from the analysis sum                       | standard           |
| P14 | Apply/verify Parseval                                            | standard           |
| P15 | Deduce signal properties from the spectrum (and back)            | standard           |
| P16 | Is this discrete-time signal periodic?                           | term final Q5(c)   |
| P17 | Lowest sampling rate for perfect reconstruction                  | term final Q4(b)   |
| P18 | Map an application task to a signal property                     | term final Q7(d)   |

### 10.1 P1 — Compute a small DFT by hand

**Looks like:** “Compute the  $N$ -point DFT of  $x[n] = \{\dots\}$ .”

#### Steps

1. Write  $W_N = e^{-j2\pi/N}$  and list its powers  $W_N^0, \dots, W_N^{N-1}$  as concrete numbers. (For  $N = 4$ :  $1, -j, -1, j$ . For  $N = 8$ , use the table in Part IV.)

2. Build the  $N \times N$  table of  $W_N^{kn}$ , reading exponents **mod**  $N$ .
3. Each  $X[k]$  is the dot product of row  $k$  with the sample list.
4. **Check:**  $X[0] = \sum_n x[n]$ ; if  $x$  is real,  $X[N - k] = X^*[k]$ .

#### Shortcuts worth spotting before you start:

- $x[n] = \delta[n]$  (impulse at  $n = 0$ )  $\Rightarrow X[k] = 1$  for all  $k$  (flat).
- $x[n] = \delta[n - n_0] \Rightarrow X[k] = W_N^{kn_0}$  (flat magnitude, linear phase).
- $x[n] = 1$  for all  $n \Rightarrow X[k] = N\delta[k]$  (all energy in bin 0).
- $x[n] = e^{j\frac{2\pi}{N}k_0n} \Rightarrow X[k] = N\delta[k - k_0]$  (one bin lights up).
- $x[n] = \cos(\frac{2\pi}{N}k_0n) \Rightarrow X[k] = \frac{N}{2}$  at  $k = k_0$  and at  $k = N - k_0$  (two bins, because  $\cos = \frac{1}{2}(e^{j\cdot} + e^{-j\cdot})$ ).
- If  $x$  is real and even about  $n = 0$  on the clock,  $X$  is real — no need to track imaginary parts.

## 10.2 P2 — Convert bins to frequency and back

**Looks like:** “Which bin does a 3 kHz tone land in?” / “What frequency does bin 5 correspond to?” / “What is the frequency resolution?”

#### Steps

1. Write down  $N, f_s$  (or  $T_s$ ).
2. Bin spacing:  $\Delta f = f_s / N$ .
3. Bin  $\rightarrow$  frequency:  $f_k = k \Delta f$  if  $k \leq N/2$ ;  $f_k = (k - N) \Delta f$  if  $k > N/2$ .
4. Frequency  $\rightarrow$  bin:  $k = f / \Delta f$ ; if  $f < 0$ , add  $N$ .
5. If  $|f| > f_s/2$  the tone **aliases**: subtract multiples of  $f_s$  until it lands in  $[-f_s/2, f_s/2)$ , then convert.

**Trap:** resolution  $\Delta f = 1/T_0$  depends only on the *recording length*  $T_0 = NT_s$ . Zero-padding does not improve it.

## 10.3 P3 — Hand-run a radix-2 FFT and draw the butterfly diagram

**Looks like:** “Compute the  $N$ -point radix-2 DIT (or DIF) FFT of  $x[n] = \{\dots\}$ . Draw a butterfly diagram and show your calculations on the diagram.”

#### Steps — DIT

1. Check  $N$  is a power of 2. If not, zero-pad to  $2^{\lceil \log_2 N \rceil}$  and say so.
2. **Reorder the input into bit-reversed order.** ( $\log_2 N$  bits, leading zeros kept.) For  $N = 4$ :  $x[0], x[2], x[1], x[3]$ . For  $N = 8$ :  $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$ .
3. Draw  $\log_2 N$  stages, each with  $N/2$  butterflies. Stage  $s$  operates on blocks of size  $2^s$ .
4. Work left to right. Each butterfly: top out =  $g + W \cdot h$ , bottom out =  $g - W \cdot h$ . Stage  $s$  uses twiddles  $W_{2^s}^0, \dots, W_{2^s}^{2^{s-1}-1}$  (stage 1 uses only  $W^0 = 1$ ).
5. **Write the running value on every edge of the diagram** — that is what “show your calculations on the diagram” means, and it is where the marks are.
6. Outputs are in natural order  $X[0], \dots, X[N - 1]$ .
7. **Check:**  $X[0]$  must equal  $\sum_n x[n]$ .

**Steps — DIF**

1. Input in **natural** order.
2. Stage 1: for  $n = 0, \dots, \frac{N}{2} - 1$  compute  $a[n] = x[n] + x[n + \frac{N}{2}]$  (top halves) and  $b[n] = (x[n] - x[n + \frac{N}{2}]) W_N^n$  (bottom halves). Note: subtract *first*, then multiply by the twiddle.
3. Recurse on each half with  $N \rightarrow N/2$  (so the twiddles become  $W_{N/2}^n$ ).
4. Outputs emerge **bit-reversed**; the last thing you do is re-label them.

**Trap:** in DIT the twiddle multiplies the *lower* input *before* the add/subtract; in DIF it multiplies the difference *after*. Drawing the wrong one loses the structure mark even if the numbers come out right.

## 10.4 P4 — Interpret a DFT result physically

**Looks like:** “If  $x[n]$  is viewed as samples of an aperiodic continuous-time signal, what might that signal be?” / “Interpret your result.”

**Steps**

1. Identify the *shape* of  $x[n]$ : an impulse? a constant? a rectangle? a single tone?
2. Name the continuous-time signal with that shape: impulse  $\rightarrow \delta(t)$ ; constant  $\rightarrow$  dc; rectangle  $\rightarrow$  a gate/pulse; one tone  $\rightarrow$  a sinusoid.
3. Recall the matching continuous-time transform pair:

$$\delta(t) \leftrightarrow 1 \text{ (flat)}, \quad 1 \leftrightarrow 2\pi\delta(\omega), \quad \text{rect} \leftrightarrow \text{sinc}, \quad \cos \Omega_0 t \leftrightarrow \text{two impulses}.$$

4. Say explicitly how the discrete result mirrors the continuous one, and where the difference lies (periodicity of the discrete spectrum, folding of the negative half).

**The most-used pair in this course:** an impulse at the origin has a **perfectly flat** spectrum, in every one of the four transforms. Flat spectrum = “all frequencies present, equally”. That is exactly what a single infinitely-short click sounds like.

## 10.5 P5 — Zero-padding: what changes and what does not

**Steps**

1. State which of the **three reasons** for padding the question is about: (a) finer spectrum plot, (b) making circular = linear, (c) reaching a power of 2 for radix-2.
2. For (a): the DTFT is **unchanged**; the  $M$ -point DFT samples it at spacing  $2\pi/M$  instead of  $2\pi/N$ . No new information; *resolution*  $\Delta f = 1/T_0$  is unchanged.
3. For (b): quote  $N \geq L + M - 1$ .
4. For (c):  $N = 2^{\lceil \log_2 M \rceil}$ .
5. Add the caveat: for a genuinely *periodic*  $x[n]$  treated as a Fourier series, padding changes the period and hence the signal.

**Worked instance (CT4 Q1c).**  $x[n] = \{1, 0, 0, 0\}$  has a flat 4-point DFT  $X[k] = 1$ . Zero-padded to 32, the DTFT is still  $X(e^{j\omega}) = \sum_n x[n]e^{-j\omega n} = 1$ , constant. So the 32-point DFT is  $X[k] = 1$  for  $k = 0, \dots, 31$ : still flat, just sampled at 32 points instead of 4. The picture got denser; the

content did not change.

## 10.6 P6 — “What if you break a step of the algorithm?”

**Looks like:** “Imagine you forgot to bit-reverse the input. What signal  $x'[n]$  has the DFT you actually computed?”

### Steps

1. Write down what the algorithm *assumes* about the array it is handed. (A DIT butterfly network assumes position  $i$  holds  $x[\text{bitrev}(i)]$ .)
2. Write down what you actually handed it. (Position  $i$  holds  $x[i]$ .)
3. Solve for the “logical” signal  $x'$  that would have produced this array *correctly*: here  $x'[\text{bitrev}(i)] = x[i]$ , i.e.  $x'[n] = x[\text{bitrev}(n)]$ .
4. Conclude: the output is the true DFT of the **bit-reversed permutation** of the input.
5. Sanity-check on the given data. (If the input is an impulse at  $n = 0$ , bit-reversal moves nothing, so the output is unchanged.)

**Generalisation.** Any “you skipped/scrambled a step” question is answered the same way: the algorithm computed a *correct* transform of a *different* input. Find that input.

## 10.7 P7 — Derive a higher-radix (or generalised) FFT

**Looks like:** “You have established  $X[k] = \sum_{m=0}^{r-1} W_N^{mk} G_m[k]$  for  $0 \leq k < N/r$ . Derive  $X[k + N/r]$ ,  $X[k + 2N/r]$ , ... and draw the butterfly.”

### Steps

1. Substitute  $k \rightarrow k + p\frac{N}{r}$  into the given expression.
2. Use **periodicity of the sub-DFTs**:  $G_m$  has period  $N/r$ , so  $G_m[k + p\frac{N}{r}] = G_m[k]$ . The sub-transforms are untouched.
3. Simplify the twiddle:

$$W_N^{m(k+pN/r)} = W_N^{mk} \cdot W_N^{mpN/r} = W_N^{mk} \cdot (e^{-j2\pi/r})^{mp}. \quad (10.1)$$

For  $r = 2$  that factor is  $(-1)^{mp}$ ; for  $r = 4$  it is  $(-j)^{mp}$ .

4. Write the  $r$  output equations explicitly.
5. For the butterfly: set  $N = r$ , so  $N/r = 1$ ,  $k = 0$ , and  $G_m[0] = x[m]$ . The butterfly is then just the  $r$ -point DFT, drawn with the multipliers you found.

**The radix-4 result (worth memorising, it is a likely repeat)**

$$X[k + p\frac{N}{4}] = \sum_{m=0}^3 (-j)^{pm} W_N^{mk} G_m[k], \quad p = 0, 1, 2, 3 \quad (10.2)$$

which expands to

$$\begin{aligned} X[k] &= G_0 + W_N^k G_1 + W_N^{2k} G_2 + W_N^{3k} G_3 \\ X[k + \frac{N}{4}] &= G_0 - jW_N^k G_1 - W_N^{2k} G_2 + jW_N^{3k} G_3 \\ X[k + \frac{2N}{4}] &= G_0 - W_N^k G_1 + W_N^{2k} G_2 - W_N^{3k} G_3 \\ X[k + \frac{3N}{4}] &= G_0 + jW_N^k G_1 - W_N^{2k} G_2 - jW_N^{3k} G_3 \end{aligned}$$

(arguments  $[k]$  suppressed on the  $G_m$ ). Note the multiplier pattern is the  $4 \times 4$  DFT matrix  $(-j)^{pm}$  — which is exactly what it should be.

## 10.8 P8 — Turn a problem into a convolution, then FFT it

**Looks like:** “Multiply two polynomials” / “multiply two large integers” / “compute a cross-correlation” — “propose an efficient algorithm and analyse its complexity.”

### Steps

1. **Naive first.** Write the direct double loop and state its cost ( $O(N^2)$ ).
2. **Identify the convolution.** Write the target as  $c[n] = \sum_k a[k]b[n-k]$  and state the valid range of  $n$ .
3. **Say the magic sentence:** convolution in time = multiplication of DFTs — but only if the DFT is long enough that circular = linear.
4. **Choose the length:**  $N' \geq L + M - 1$ , rounded up to a power of 2 for radix-2.
5. **Give the four-step algorithm:** pad  $\rightarrow$  two FFTs  $\rightarrow$  pointwise multiply  $\rightarrow$  one inverse FFT.
6. **Analyse:**  $3 \times O(N' \log N') + O(N') = O(N \log N)$ .

## 10.9 P9 — Extend the DFT to more dimensions

**Looks like:** “The 2D DFT is defined as ... Give the naive complexity, interpret it as two successive 1D DFTs, propose a 2D FFT.”

### Steps

1. **Naive:** there are  $N_1 N_2$  output points; each is a double sum over  $N_1 N_2$  input points  $\Rightarrow O(N_1^2 N_2^2)$ .
2. **Separate the kernel:**  $W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}$  is a product, so the double sum *factorises*. Pull the  $n_1$ -independent part out:

$$X[k_1, k_2] = \sum_{n_1} \left( \sum_{n_2} x[n_1, n_2] W_{N_2}^{k_2 n_2} \right) W_{N_1}^{k_1 n_1} \quad (10.3)$$

— the inner bracket is a 1D DFT along each *row*; the outer sum is a 1D DFT down each *column* of the result. This is the **row-column decomposition**.

3. **2D FFT:** do  $N_1$  row-FFTs and then  $N_2$  column-FFTs:

$$O(N_1 N_2 \log N_2 + N_1 N_2 \log N_1) = O(N_1 N_2 \log(N_1 N_2)). \quad (10.4)$$

4. **Compare:** for an  $N \times N$  image,  $O(N^4) \rightarrow O(N^2 \log N)$ .

## 10.10 P10 — Compare DFT vs FFT complexity

**Looks like:** “Compare the computational complexities of the DFT and the radix-2 FFT.” (15 marks — they want a structured answer, not one line.)

### Structure your answer like this

1. **Naive DFT, counted:** for each of  $N$  values of  $k$ ,  $N$  complex multiplications and  $N - 1$  complex additions  $\Rightarrow N^2$  mults,  $N(N - 1)$  adds,  $O(N^2)$ .
2. **Why it can be beaten:** the  $N^2$  kernel values  $W_N^{kn}$  are not distinct;  $W_N^{k+N/2} = -W_N^k$  and  $W_N^2 = W_{N/2}$ .
3. **The FFT structure:**  $\log_2 N$  stages  $\times N/2$  butterflies; each butterfly is one complex multiplication, one addition, one subtraction.
4. **Counted:**  $\frac{N}{2} \log_2 N$  mults,  $N \log_2 N$  adds,  $O(N \log_2 N)$ , plus  $O(N)$  for bit-reversal.
5. **Ratio:**  $\frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N}$  — and give a table of numbers (see Part III).
6. **Caveats:** radix-2 requires  $N = 2^m$ ; the FFT needs  $O(N)$  extra storage for twiddles; it is numerically *more* accurate than the naive sum (fewer accumulated roundings).

## 10.11 P11 — Circular vs linear convolution

### Steps

1. Note  $L$ ,  $M$ , and the required  $N$ .
2. **Linear:** flip  $h$ , slide, multiply, add. Length  $L + M - 1$ .
3. **Circular ( $N$  points):** pad both to  $N$ , then either (a)  $y[n] = \sum_{m=0}^{N-1} x[m]h[(n-m))_N]$  directly, or (b) compute the linear convolution and fold: add sample  $L + M - 1 + i$  back onto sample  $i$ . Method (b) is usually faster by hand.
4. **Count corruptions:**  $\max(0, L + M - 1 - N)$ , always the first outputs.
5. If asked “what  $N$  makes them equal?” answer  $N \geq L + M - 1$ .

## 10.12 P12 — Get a new DFT from a known one

**Looks like:** “Given the DFT of  $x[n]$ , find the DFT of  $x[n - 2] / (-1)^n x[n] / x[-n] / x^*[n]$ .”

### Steps

1. Identify *which single property* relates the new signal to the old one. Match it against the table in Part IV.
2. Apply the property. Keep everything on the clock:  $((\cdot))_N$  every index.
3. **Check:** a shift must not change  $|X[k]|$ ; a real signal’s answer must still satisfy  $X[N - k] = X^*[k]$ .

## 10.13 P13 — Prove a property from the analysis sum

### The universal three-line proof

1. Substitute the modified signal into  $X[k] = \sum_{n=0}^{N-1} x[n]W_N^{kn}$ .
2. Change variable to make the exponent look like  $W_N^{k(\text{new index})}$ ; factor out whatever is left over (it will be a constant in  $n$ ).



3. State: both  $\tilde{x}$  and  $W_N^{kn}$  are  $N$ -periodic, so any window of  $N$  consecutive terms has the same sum — slide the window back to  $0, \dots, N-1$ .

Always write step 3 out in words. It is the step that is actually being examined.

## 10.14 P14 — Apply or verify Parseval

### Steps

1. Left side:  $\sum_n |x[n]|^2$  — for real  $x$ , just square and add.
2. Right side:  $\frac{1}{N} \sum_k |X[k]|^2$  — remember  $|a + jb|^2 = a^2 + b^2$  (no square roots).
3. If asked for the energy in one bin:  $|X[k]|^2 / N$ .
4. If asked to *prove* it: substitute the conjugated IDFT for  $x^*[n]$  and swap sums.

## 10.15 P15 — Deduce signal properties from the spectrum (and back)

| If you see ...                                            | conclude ...                            |
|-----------------------------------------------------------|-----------------------------------------|
| $ X[k] $ symmetric about $N/2$ , $\angle X$ antisymmetric | $x[n]$ is real                          |
| $X[k]$ purely real                                        | $x[n]$ is real and even (on the clock)  |
| $X[k]$ purely imaginary                                   | $x[n]$ is real and odd                  |
| Peaks at $k = 0$ and $k \approx N$ , dip at $N/2$         | real <b>lowpass</b>                     |
| Peak at $k \approx N/2$                                   | real <b>highpass</b>                    |
| $ X[k] $ flat                                             | $x[n]$ is an impulse (possibly shifted) |
| $ X[k] $ flat but $\angle X[k]$ linear in $k$             | impulse shifted by the slope            |
| Only bins $k_0$ and $N - k_0$ nonzero, equal              | a real sinusoid at bin $k_0$            |

## 10.16 P16 — Is this discrete-time signal periodic?

**Looks like:** “Is  $x[n] = A \cos(\omega_0 n + \phi)$  periodic? Explain.”

### Steps

1. A discrete-time sinusoid is periodic iff  $\frac{\omega_0}{2\pi}$  is **rational**. (Unlike continuous time, where every sinusoid is periodic.)
2. Write  $\frac{\omega_0}{2\pi} = \frac{p}{q}$  in lowest terms.
3. The fundamental period is  $N = q$ .
4. Justify: we need  $\omega_0 N = 2\pi m$  for integers  $N > 0, m$ , i.e.  $N = 2\pi m / \omega_0$ ; the smallest integer solution is  $q$ .

**Worked instance (term final Q5c).**  $x[n] = 2 \cos(\frac{16\pi n}{63} + \frac{\pi}{3})$ . Here  $\omega_0 = \frac{16\pi}{63}$ , so

$$\frac{\omega_0}{2\pi} = \frac{16\pi}{63 \cdot 2\pi} = \frac{8}{63},$$

which is rational and already in lowest terms ( $\gcd(8, 63) = 1$ ). **Yes, periodic**, with fundamental period  $N = 63$ .

## 10.17 P17 — Lowest sampling rate for perfect reconstruction

**Looks like:** “ $X(e^{j\omega}) = 0$  for  $\omega_M \leq |\omega| \leq \pi$ . Determine the lowest rate at which  $x[n]$  may be sampled for perfect reconstruction.”

### Steps (discrete-time sampling / decimation)

1. Identify the band limit  $\omega_M$ : the largest  $|\omega|$  where the spectrum is nonzero.
2. Sampling a *sequence* by keeping every  $N$ -th sample creates spectral copies at spacing  $\frac{2\pi}{N}$ .
3. No overlap requires  $\frac{2\pi}{N} \geq 2\omega_M$ , i.e.

$$N \leq \frac{\pi}{\omega_M} \quad (10.5)$$

4. Take the largest integer  $N$  satisfying it. The sampling *rate* is  $1/N$  (one kept sample per  $N$ ).

**Worked instance (term final Q4b).**  $X(j\omega) = 0$  for  $\frac{2\pi}{9} \leq |\omega| \leq \pi$ , so  $\omega_M = \frac{2\pi}{9}$ .

$$N \leq \frac{\pi}{2\pi/9} = \frac{9}{2} = 4.5 \implies N_{\max} = 4.$$

So we may keep one sample in every 4: the lowest sampling rate is  $\frac{1}{4}$  of the original rate. (Equivalently, in the continuous-time picture,  $\omega_s \geq 2\omega_M$  — the same Nyquist condition, read on the digital-frequency axis.)

## 10.18 P18 — Map an application task to a signal property

**Looks like:** “For each of the following image-processing tasks, name the specific signal property that makes it possible.”

| Task                  | Property                                                                  |
|-----------------------|---------------------------------------------------------------------------|
| Image compression     | energy compaction / Parseval — keep the large $ X[k] $ , discard the rest |
| Blurring              | convolution with a lowpass kernel = multiplication of spectra             |
| Sharpening            | convolution with a highpass kernel; or modulation by $(-1)^n$ ideas       |
| Flipping the image    | time (space) reversal: $x[(-n)_N] \leftrightarrow X[(-k)_N]$              |
| Zooming in/out        | time scaling (with interpolation / decimation)                            |
| Image contrast        | amplitude scaling — linearity (and the dc term $X[0]$ )                   |
| Edge detection        | differencing = highpass filtering = convolution                           |
| Translating the image | shift property — magnitude unchanged, phase linear                        |

**The pattern behind the pattern.** Nearly every image operation is one of: *convolution* (blur, sharpen, edges), a *property of the transform* (flip, translate, scale), or *throwing away small coefficients* (compression). Say which of the three, then name the specific property.

PART VI

## Practice questions with solutions

# Chapter 11

## Class Test 4 — July 2025 (past paper)

**Source:** CSE 219 Class Test 4, July 2025. Time 25 minutes, 20 marks. *Answer exactly 2 out of Questions 1–4. Question 5 is mandatory.* All four questions are solved below.

### 11.1 Question 1 — 4-point radix-2 DIT FFT of an impulse

[8]

Consider the signal  $x[n]$  defined as follows for  $N = 4$ :

$$x[n] = \begin{cases} 1, & n = 0 \\ 0, & 1 \leq n < N \end{cases}$$

- (a) Compute the 4-point radix-2 DIT FFT of the signal. Draw a butterfly diagram and show your calculations on the diagram.
- (b) If  $x[n]$  is viewed as samples of an aperiodic continuous-time signal, what might be a continuous-time signal corresponding to it?
- (c) Based on your answer in (b), interpret your result from (a). If the sequence is zero-padded to length 32, what is its 32-point DFT? Briefly justify.
- (d) In a radix-2 DIT FFT we typically use bit-reversed order for the input  $x[n]$ . Imagine you forgot to use bit-reversed order and used natural order instead, resulting in a sequence  $X'[k]$ . Determine the signal  $x'[n]$  whose DFT is actually  $X'[k]$ .

#### Solution (a)

$x = \{1, 0, 0, 0\} = \delta[n]$ , and  $N = 4$ , so  $\log_2 4 = 2$  stages.

**Step 1 — bit-reverse the input.** With 2 bits:  $0 = 00_2 \rightarrow 00_2 = 0$ ,  $1 = 01_2 \rightarrow 10_2 = 2$ ,  $2 = 10_2 \rightarrow 01_2 = 1$ ,  $3 = 11_2 \rightarrow 11_2 = 3$ . So the input order is  $x[0], x[2], x[1], x[3] = 1, 0, 0, 0$ .

**Step 2 — stage 1** (two 2-point DFTs, twiddle  $W_2^0 = 1$ ):

$$\begin{aligned} G[0] &= x[0] + x[2] = 1 + 0 = 1, & G[1] &= x[0] - x[2] = 1 - 0 = 1, \\ H[0] &= x[1] + x[3] = 0 + 0 = 0, & H[1] &= x[1] - x[3] = 0 - 0 = 0. \end{aligned}$$

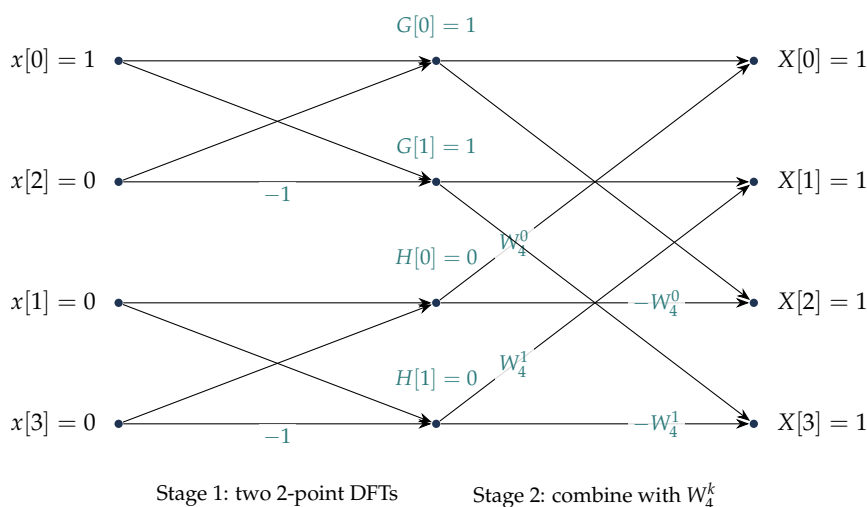
**Step 3 — stage 2** (combine,  $W_4^0 = 1, W_4^1 = -j$ ):

$$\begin{aligned} X[0] &= G[0] + W_4^0 H[0] = 1 + 0 = 1, & X[2] &= G[0] - W_4^0 H[0] = 1 - 0 = 1, \\ X[1] &= G[1] + W_4^1 H[1] = 1 + (-j)(0) = 1, & X[3] &= G[1] - W_4^1 H[1] = 1 - 0 = 1. \end{aligned}$$

**Answer**

$$X[k] = 1 \quad \text{for } k = 0, 1, 2, 3 \quad (\text{a perfectly flat spectrum})$$

**Check:**  $X[0]$  must equal  $\sum_n x[n] = 1$ . ✓



**Figure 11.1:** 4-point radix-2 DIT FFT of  $x = \{1, 0, 0, 0\}$ , with every intermediate value written on the diagram. Inputs bit-reversed, outputs natural.

**Solution (b)**

$x[n] = \delta[n]$ : a single unit value at  $n = 0$  and nothing anywhere else.

The continuous-time signal with that shape is the **unit impulse**  $x(t) = \delta(t)$  — or, if you prefer something physically realisable, a very short unit-area pulse (a click, a hammer tap, a camera flash). Any narrow pulse of unit area whose width is much less than  $T_s$  would sample to  $\{1, 0, 0, 0\}$ .

**Solution (c)**

**Interpretation.** The CTFT of an impulse is flat:

$$\delta(t) \longleftrightarrow \int_{-\infty}^{\infty} \delta(t) e^{-j\omega t} dt = 1 \quad \text{for every } \omega. \quad (11.1)$$

Our DFT came out flat too,  $X[k] = 1$  for all  $k$ . So the discrete result mirrors the continuous one exactly: **an impulse contains every frequency in equal amount**. That is the whole reason impulse responses characterise LTI systems — you excite all frequencies at once.

**Zero-padded to 32.** The DTFT of the padded sequence is

$$X(e^{j\omega}) = \sum_{n=0}^{31} x[n] e^{-j\omega n} = 1 \cdot e^{-j\omega \cdot 0} = 1, \quad (11.2)$$

because only  $x[0] = 1$  contributes and the appended zeros add nothing. The 32-point DFT samples that constant curve at  $\omega = \frac{2\pi}{32}k$ :

**Answer**

$$X_{32}[k] = 1 \quad \text{for } k = 0, 1, \dots, 31.$$

**Justification in one sentence:** zero-padding does not change the DTFT, and the DTFT here is a constant, so sampling it at 32 points instead of 4 gives the same constant — more pixels of the same flat picture, no new information.

**Solution (d)**

**What the algorithm assumes.** A radix-2 DIT butterfly network expects the array it is handed to already be in bit-reversed order — i.e. it expects

$$\text{array}[i] = x[\text{bitrev}(i)].$$

**What you actually handed it.**  $\text{array}[i] = x[i]$ .

**Solve for the effective input.** If the network is to be *correct*, then the signal  $x'$  it really transformed must satisfy  $x'[\text{bitrev}(i)] = x[i]$ . Substituting  $n = \text{bitrev}(i)$  and using the fact that bit reversal is its own inverse ( $\text{bitrev}(\text{bitrev}(n)) = n$ ):

**Answer**

$$x'[n] = x[\text{bitrev}(n)] \quad (11.3)$$

The output  $X'[k]$  is the *correct* DFT of the **bit-reversed permutation** of the input.

For  $N = 4$  the permutation is  $0 \rightarrow 0, 1 \rightarrow 2, 2 \rightarrow 1, 3 \rightarrow 3$ , so

$$x' = \{x[0], x[2], x[1], x[3]\}.$$

**For the given data:**  $x = \{1, 0, 0, 0\}$ , so  $x' = \{1, 0, 0, 0\} = x$  — the permutation only moves zeros. Hence  $X'[k] = X[k] = 1$  for all  $k$ : in this particular case the mistake is invisible.

**Worth adding for the extra mark:** in general the mistake is *not* invisible. For  $N = 8$  and, say,  $x = \{a, b, c, d, e, f, g, h\}$  you would get the DFT of  $\{a, e, c, g, b, f, d, h\}$  instead — a completely different signal, and a completely wrong answer.

**11.2 Question 2 — deriving the radix-4 DIT FFT**

[8]

You have already established that for  $0 \leq k < N/4$ :

$$X[k] = G_0[k] + W_N^k G_1[k] + W_N^{2k} G_2[k] + W_N^{3k} G_3[k], \quad G_m[k] = \sum_{r=0}^{N/4-1} x[4r+m] W_{N/4}^{kr}.$$

- (a) Derive the formulas for  $X[k + N/4]$ ,  $X[k + 2N/4]$  and  $X[k + 3N/4]$ .
- (b) Draw the radix-4 butterfly for the case  $N = 4$ . *Hint: instead of just  $\pm 1$ , there will also be multiplications involving  $\pm j$ .*

**Solution (a)**

Replace  $k$  by  $k + p \frac{N}{4}$  in the given expression, for  $p = 1, 2, 3$ . Two things happen.

**(i) The sub-DFTs are unchanged.** Each  $G_m$  is an  $\frac{N}{4}$ -point DFT, hence periodic with period  $\frac{N}{4}$ :

$$G_m[k + p \frac{N}{4}] = G_m[k]. \quad (11.4)$$

**(ii) Only the twiddles change.** For the term with index  $m$ :

$$W_N^{m(k+p\frac{N}{4})} = W_N^{mk} \cdot W_N^{\frac{mpN}{4}} = W_N^{mk} \cdot \left(e^{-j\frac{2\pi}{N} \cdot \frac{N}{4}}\right)^{mp} = W_N^{mk} \cdot \left(e^{-j\frac{\pi}{2}}\right)^{mp} = W_N^{mk} (-j)^{mp}. \quad (11.5)$$

**General result**

$$X\left[k + p\frac{N}{4}\right] = \sum_{m=0}^3 (-j)^{mp} W_N^{mk} G_m[k], \quad p = 0, 1, 2, 3 \quad (11.6)$$

Writing the three requested cases out, using  $(-j)^0 = 1$ ,  $(-j)^1 = -j$ ,  $(-j)^2 = -1$ ,  $(-j)^3 = j$  (and reading exponents  $mp$  modulo 4):

$$X\left[k + \frac{N}{4}\right] = G_0[k] - j W_N^k G_1[k] - W_N^{2k} G_2[k] + j W_N^{3k} G_3[k] \quad (11.7)$$

$$X\left[k + \frac{2N}{4}\right] = G_0[k] - W_N^k G_1[k] + W_N^{2k} G_2[k] - W_N^{3k} G_3[k] \quad (11.8)$$

$$X\left[k + \frac{3N}{4}\right] = G_0[k] + j W_N^k G_1[k] - W_N^{2k} G_2[k] - j W_N^{3k} G_3[k] \quad (11.9)$$

**Check the pattern.** The matrix of multipliers  $(-j)^{mp}$  is

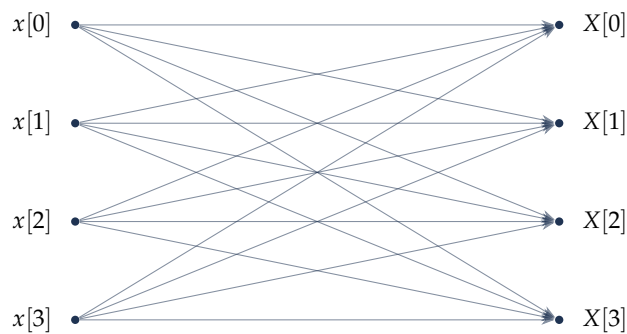
$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix}$$

— which is exactly the  $4 \times 4$  DFT matrix. It *has* to be: after the twiddles  $W_N^{mk}$  are applied, combining the four sub-transforms is a 4-point DFT.

**Solution (b) — the radix-4 butterfly for  $N = 4$** 

With  $N = 4$ :  $\frac{N}{4} = 1$ , so  $k = 0$  only, and each  $G_m$  is a 1-point DFT, i.e.  $G_m[0] = x[m]$ . All the twiddles  $W_4^{mk} = W_4^0 = 1$ . So the butterfly reduces to the 4-point DFT itself:

$$\begin{aligned} X[0] &= x[0] + x[1] + x[2] + x[3] \\ X[1] &= x[0] - j x[1] - x[2] + j x[3] \\ X[2] &= x[0] - x[1] + x[2] - x[3] \\ X[3] &= x[0] + j x[1] - x[2] - j x[3] \end{aligned}$$



**Figure 11.2:** The radix-4 butterfly for  $N = 4$ : a full  $4 \times 4$  crossbar. Every input reaches every output; the multiplier on the edge from  $x[m]$  to  $X[p]$  is  $(-j)^{mp}$  (see the table below).

| edge multiplier $(-j)^{mp}$ | to $X[0]$ | to $X[1]$ | to $X[2]$ | to $X[3]$ |
|-----------------------------|-----------|-----------|-----------|-----------|
| from $x[0]$                 | 1         | 1         | 1         | 1         |
| from $x[1]$                 | 1         | $-j$      | $-1$      | $+j$      |
| from $x[2]$                 | 1         | $-1$      | 1         | $-1$      |
| from $x[3]$                 | 1         | $+j$      | $-1$      | $-j$      |

#### Why radix-4 is worth the trouble

For  $N = 4^m$  the algorithm needs only  $\log_4 N = \frac{1}{2} \log_2 N$  stages, i.e. *half* as many. And the internal multipliers within a butterfly are  $\pm 1, \pm j$ , which cost no real multiplications at all — multiplying by  $j$  just swaps real and imaginary parts and flips a sign. Net effect: about **25% fewer real multiplications** than radix-2.

### 11.3 Question 3 — polynomial multiplication

[8]

$A(x) = \sum_{n=0}^{N-1} a[n]x^n$  and  $B(x) = \sum_{n=0}^{N-1} b[n]x^n$ . Compute  $C(x) = A(x)B(x)$ .

- Provide a simple naive algorithm and analyse its time complexity.
- If  $C(x) = \sum c[n]x^n$ , determine the valid range of  $n$  and give a general formula for  $c[n]$ .
- Express  $c[n]$  as a convolution.
- Propose a more efficient algorithm and analyse its complexity.

#### Solution (a) — naive algorithm

Multiply every term of  $A$  by every term of  $B$  and accumulate:

```

c[0..2N-2] ← 0
for i = 0 to N-1:
    for j = 0 to N-1:
        c[i+j] += a[i] · b[j]
```

**Complexity:** the inner statement runs  $N \times N$  times, so  $N^2$  multiplications and  $N^2$  additions:  $O(N^2)$ .

#### Solution (b) — range and formula

$A$  has degree  $N-1$  and  $B$  has degree  $N-1$ , so  $C$  has degree  $2N-2$ . Therefore

$$\boxed{0 \leq n \leq 2N-2} \quad (2N-1 \text{ coefficients in total}). \quad (11.10)$$

The coefficient of  $x^n$  collects every pair  $(i, j)$  with  $i + j = n$ :

$$\boxed{c[n] = \sum_{k=\max(0, n-N+1)}^{\min(n, N-1)} a[k] b[n-k]} \quad (11.11)$$

The clipped limits are just bookkeeping: they enforce  $0 \leq k \leq N-1$  (so  $a[k]$  exists) and  $0 \leq n-k \leq N-1$  (so  $b[n-k]$  exists). If you define  $a[\cdot]$  and  $b[\cdot]$  to be zero outside  $0, \dots, N-1$ , this is simply  $c[n] = \sum_k a[k] b[n-k]$ .



**Solution (c) — it is a convolution****Answer**

$$c[n] = \sum_{k=-\infty}^{\infty} a[k] b[n-k] = (a * b)[n] \quad (11.12)$$

The coefficient sequence of the product polynomial is the **linear convolution** of the two coefficient sequences. (Consistent with the length rule:  $L + M - 1 = N + N - 1 = 2N - 1$ . ✓)

**Solution (d) — the fast algorithm**

Use the convolution theorem: convolution in the index domain is pointwise multiplication in the DFT domain — provided the transform is long enough that circular convolution equals linear convolution.

**FFT-based polynomial multiplication**

1. Choose  $N' \geq 2N - 1$ , rounded up to a power of two:  $N' = 2^{\lceil \log_2(2N-1) \rceil}$ .
2. Zero-pad  $a$  and  $b$  to length  $N'$ .
3. Compute  $A[k] = \text{FFT}(a)$  and  $B[k] = \text{FFT}(b)$ .  $2 \times O(N' \log N')$
4. Multiply pointwise:  $C[k] = A[k]B[k]$ .  $O(N')$
5. Compute  $c = \text{IFFT}(C)$ ; keep the first  $2N - 1$  samples.  $O(N' \log N')$

**Complexity:**  $3 O(N' \log N') + O(N') = O(N' \log N')$ , and  $N' < 4N$ , so

$$O(N \log N) \quad (11.13)$$

which beats  $O(N^2)$  for all but tiny  $N$ .

**The other way to say it (evaluation–interpolation), worth a mark:** a polynomial of degree  $< N'$  is uniquely determined by its values at  $N'$  distinct points. The FFT *evaluates*  $A$  and  $B$  at the  $N'$  roots of unity in  $O(N' \log N')$ ; multiplying the values pointwise is  $O(N')$ ; the inverse FFT *interpolates* the product back to coefficients in  $O(N' \log N')$ .

**Do not forget the padding.** With  $N' = N$  you would get the *circular* convolution and the top  $N - 1$  coefficients would be folded back onto the bottom ones — wrong answer.

**11.4 Question 4 — the 2-D DFT**

[8]

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}, \quad W_N^k = e^{-j2\pi k/N}$$

- (a) Time complexity of a naive algorithm?
- (b) How can the formula be interpreted as two successive 1-D DFT operations?
- (c) Propose an efficient 2-D FFT and analyse its complexity.
- (d) What is the primary difference between your 2-D FFT and Bailey's FFT algorithm?

**Solution (a) — naive complexity**

There are  $N_1 N_2$  output values  $X[k_1, k_2]$ . Each one is a double sum over all  $N_1 N_2$  input values, i.e.  $N_1 N_2$  complex multiply–accumulates. Hence

**Answer**

$$O(N_1^2 N_2^2) = O((N_1 N_2)^2). \quad (11.14)$$

For a square  $N \times N$  image this is  $O(N^4)$  — for a modest  $512 \times 512$  image, about  $6.9 \times 10^{10}$  operations.

**Solution (b) — two successive 1-D DFTs (row–column decomposition)**

The kernel **separates**:  $W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}$  is a product of one factor that depends only on  $n_1$  and one that depends only on  $n_2$ . So the factor  $W_{N_1}^{k_1 n_1}$  can be pulled out of the inner sum:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \underbrace{\left( \sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{k_2 n_2} \right)}_{F[n_1, k_2]} W_{N_1}^{k_1 n_1} \quad (11.15)$$

**Answer**

- The inner bracket  $F[n_1, k_2]$  is the **1-D DFT along**  $n_2$  — i.e. take each **row**  $n_1$  of the image and transform it. Result: an intermediate array  $F$  of the same size.
- The outer sum is then the **1-D DFT along**  $n_1$  of  $F$  — i.e. take each **column**  $k_2$  of  $F$  and transform it.

So the 2-D DFT = ( $N_1$  row transforms) followed by ( $N_2$  column transforms). The order can be swapped; the result is identical.

**Solution (c) — the 2-D FFT and its complexity****Row–column 2-D FFT**

1. For each of the  $N_1$  rows, compute an  $N_2$ -point 1-D FFT.
2. For each of the  $N_2$  columns of the result, compute an  $N_1$ -point 1-D FFT.

**Complexity:**

$$\text{step 1: } N_1 \times O(N_2 \log N_2) = O(N_1 N_2 \log N_2)$$

$$\text{step 2: } N_2 \times O(N_1 \log N_1) = O(N_1 N_2 \log N_1)$$

$$\text{total: } O(N_1 N_2 (\log N_1 + \log N_2)) = \boxed{O(N_1 N_2 \log(N_1 N_2))}$$

For a square  $N \times N$  image:  $O(N^2 \log N)$  instead of  $O(N^4)$ . For  $512 \times 512$  that is about  $2.4 \times 10^6$  versus  $6.9 \times 10^{10}$  — roughly a  $29,000\times$  speed-up.

**Solution (d) — versus Bailey's FFT****Answer**

They *look* similar — both reshape data into a matrix and do FFTs along rows and then columns — but they solve different problems and differ in one crucial step.

**My 2-D FFT** computes the DFT of genuinely **two-dimensional** data. Because the kernel separates exactly, the row pass and the column pass are independent: **no extra factors appear between them**.

**Bailey's FFT** (the “four-step” or “six-step” algorithm) computes a **one-dimensional** DFT

of length  $N = N_1 N_2$  by *artificially* reshaping the 1-D array into an  $N_1 \times N_2$  matrix. Because the data is not really 2-D, the factorisation  $n = n_1 + N_1 n_2$ ,  $k = k_2 + N_2 k_1$  leaves a leftover cross-term, so the algorithm must **multiply the intermediate array by twiddle factors**  $W_N^{n_1 k_2}$  **between the two passes** (plus matrix transposes).

**Primary difference:** *the intermediate twiddle-factor multiplication.* Bailey's needs it; a true 2-D FFT does not.

**Secondary difference — purpose:** Bailey's exists for *memory locality*. Its motivation is that a huge 1-D FFT thrashes cache or does not fit in RAM; reshaping into a matrix turns it into many small transforms that fit. The row-column 2-D FFT exists simply because the data is 2-D.

## 11.5 Question 5

[4]

"Thank you for attending the classes and participating in this CT."

Mandatory, and free. Write a courteous sentence and move on — but do it, because it is 4 of the 20 marks.

## Chapter 12

# Term Final, January 2025 — on-syllabus parts

**Source:** CSE 219, L-2/T-2 B.Sc. Engineering Examinations 2022–2023, 26 January 2025. Full marks 210, 3 hours.

That paper covers the *whole* course (z-transform, Laplace, CTFT, convolution, system properties). Only the parts below fall inside this CT's syllabus; those are solved here.

### 12.1 Q4(b) — lowest sampling rate for perfect reconstruction

[10]

Consider a sequence  $x[n]$  whose Fourier transform  $X(j\omega)$  has the property  $X(j\omega) = 0$  for  $\frac{2\pi}{9} \leq |\omega| \leq \pi$ . Determine the lowest rate at which  $x[n]$  may be sampled for its perfect reconstruction.

#### Solution

$x[n]$  is a *sequence*, so its transform is the DTFT, which is  $2\pi$ -periodic. Within one period  $[-\pi, \pi]$  the spectrum is nonzero only for  $|\omega| < \frac{2\pi}{9}$ . So  $x[n]$  is a **band-limited discrete-time signal** with band limit

$$\omega_M = \frac{2\pi}{9}. \quad (12.1)$$

**What sampling a sequence does.** Keeping every  $N$ -th sample (and zeroing the rest) produces a spectrum that is the sum of copies of  $X(e^{j\omega})$  shifted by multiples of  $\frac{2\pi}{N}$ :

$$X_s(e^{j\omega}) = \frac{1}{N} \sum_{r=0}^{N-1} X\left(e^{j(\omega - \frac{2\pi r}{N})}\right). \quad (12.2)$$

**No-aliasing condition.** Each copy occupies  $(-\omega_M, \omega_M)$ , a width of  $2\omega_M$ . Adjacent copies are  $\frac{2\pi}{N}$  apart. They must not overlap:

$$\frac{2\pi}{N} \geq 2\omega_M \quad \Longleftrightarrow \quad N \leq \frac{\pi}{\omega_M}. \quad (12.3)$$

**Substitute.**

$$N \leq \frac{\pi}{2\pi/9} = \frac{9}{2} = 4.5. \quad (12.4)$$

$N$  must be a positive integer, so the largest usable value is  $N = 4$ .

**Answer**

$N_{\max} = 4$ : one sample may be kept out of every 4. The **lowest sampling rate** is therefore

$$\frac{1}{N} = \frac{1}{4} \text{ of the original rate,} \quad (12.5)$$

i.e.  $x[n]$  can be reduced to a quarter of its samples without losing any information, and recovered exactly by ideal lowpass interpolation.

**Why not  $N = 4.5$ ?** Because you cannot keep “every 4.5-th” sample. This is the one place where the discrete-time sampling theorem differs from the continuous-time one: the Nyquist bound  $\omega_s \geq 2\omega_M$  is the same inequality, but  $N$  is forced to be an integer, so you must round down.

**Sanity check with  $N = 5$ :** then  $\frac{2\pi}{5} = 1.257$  while  $2\omega_M = \frac{4\pi}{9} = 1.396$ . Since  $1.257 < 1.396$ , the copies overlap — aliasing. So  $N = 5$  fails, confirming  $N = 4$ .

**12.2 Q4(c) — compare DFT and radix-2 FFT complexity**

[15]

Compare the computational complexities of the discrete Fourier transform and the radix-2 fast Fourier transform.

**Solution****1. The direct DFT, counted**

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad W_N = e^{-j2\pi/N}, \quad k = 0, \dots, N-1 \quad (12.6)$$

For one value of  $k$ :  $N$  complex multiplications and  $N-1$  complex additions. Over all  $N$  values of  $k$ :

|                         |                            |
|-------------------------|----------------------------|
| complex multiplications | $N^2$                      |
| complex additions       | $N(N-1)$                   |
| <b>order</b>            | <b><math>O(N^2)</math></b> |

Doubling  $N$  quadruples the work. For one second of audio at 44.1 kHz ( $N = 44,100$ ) that is  $\approx 1.95 \times 10^9$  complex multiplications.

**2. Why the direct sum is wasteful**

The  $N^2$  kernel values  $W_N^{kn}$  are *not*  $N^2$  distinct numbers. Since  $W_N^N = 1$ , the exponent only matters modulo  $N$  — there are only  $N$  distinct values, each used  $N$  times. Two structural identities let us exploit that:

$$W_N^{k+N/2} = -W_N^k \quad (\text{symmetry}), \quad W_N^2 = W_{N/2} \quad (\text{halving}). \quad (12.7)$$

**3. The radix-2 FFT structure**

Decimation-in-time splits the  $N$ -point DFT into two  $\frac{N}{2}$ -point DFTs:

$$X[k] = G[k] + W_N^k H[k], \quad X[k + \frac{N}{2}] = G[k] - W_N^k H[k], \quad 0 \leq k < \frac{N}{2}, \quad (12.8)$$

where  $G$  and  $H$  are the DFTs of the even- and odd-indexed samples. Note the crucial point: **one complex multiplication  $W_N^k H[k]$  produces two outputs**, because of the symmetry identity. Recursing gives

|                          |                                                     |
|--------------------------|-----------------------------------------------------|
| stages                   | $\log_2 N$                                          |
| butterflies per stage    | $N/2$                                               |
| per butterfly            | 1 complex multiplication, 1 addition, 1 subtraction |
| complex multiplications  | $\frac{N}{2} \log_2 N$                              |
| complex additions        | $N \log_2 N$                                        |
| bit-reversal permutation | $O(N)$                                              |
| order                    | $O(N \log_2 N)$                                     |

#### 4. The comparison

$$\text{speed-up} = \frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N} \quad (12.9)$$

| $N$    | DFT: $N^2$ mults   | FFT: $\frac{N}{2} \log_2 N$ mults | speed-up              |
|--------|--------------------|-----------------------------------|-----------------------|
| 8      | 64                 | 12                                | $\approx 5\times$     |
| 64     | 4,096              | 192                               | $\approx 21\times$    |
| 1,024  | 1,048,576          | 5,120                             | $\approx 205\times$   |
| 4,096  | 16,777,216         | 24,576                            | $\approx 683\times$   |
| 65,536 | $4.29 \times 10^9$ | 524,288                           | $\approx 8,192\times$ |

The advantage *grows* with  $N$  — which is why the FFT did not merely speed things up, it made real-time spectral processing possible at all.

#### 5. Qualifications (say these for the last marks)

- **Radix-2 requires**  $N = 2^m$ . Otherwise zero-pad to  $N = 2^{\lceil \log_2 M \rceil}$ , or use a mixed-radix / Bluestein algorithm.
- **Memory.** The FFT is in-place, needing only  $O(N)$  storage plus a twiddle table; the direct DFT is also  $O(N)$  but with no restructuring.
- **Numerical accuracy.** The FFT is *more* accurate, not less: each output accumulates only  $\log_2 N$  roundings rather than  $N$ .
- **The result is identical.** The FFT is not an approximation — it is an exact reorganisation of the same arithmetic.

### 12.3 Q5(c) — is the signal periodic?

[5]

Is the signal  $x[n] = 2 \cos\left(\frac{16\pi n}{63} + \frac{\pi}{3}\right)$  a periodic signal? Explain your answer.

#### Solution

A discrete-time sinusoid  $\cos(\omega_0 n + \phi)$  is periodic if and only if there is a *positive integer*  $N$  with  $x[n + N] = x[n]$  for all  $n$ , which requires

$$\omega_0 N = 2\pi m \text{ for some integer } m \iff \frac{\omega_0}{2\pi} = \frac{m}{N} \in \mathbb{Q}. \quad (12.10)$$

#### The test

A discrete-time sinusoid is periodic  $\iff \frac{\omega_0}{2\pi}$  is **rational**. Then  $N$  is the denominator of  $\frac{\omega_0}{2\pi}$  in lowest terms.

Here  $\omega_0 = \frac{16\pi}{63}$ , so

$$\frac{\omega_0}{2\pi} = \frac{16\pi}{63} \cdot \frac{1}{2\pi} = \frac{16}{126} = \frac{8}{63}. \quad (12.11)$$

Since  $\gcd(8, 63) = 1$ , the fraction  $\frac{8}{63}$  is already in lowest terms and is rational.

#### Answer

**Yes,  $x[n]$  is periodic**, with fundamental period  $N = 63$  samples (and it completes  $m = 8$  full cycles in those 63 samples).

**Verification:**  $\omega_0 N = \frac{16\pi}{63} \cdot 63 = 16\pi = 2\pi \cdot 8$ . ✓ The phase  $\frac{\pi}{3}$  is irrelevant — a constant phase never affects periodicity.

#### Why this is not automatic (the point of the question)

In *continuous* time,  $\cos(\omega_0 t + \phi)$  is periodic for *every*  $\omega_0$ , with  $T_0 = 2\pi/\omega_0$ . In *discrete* time you can only shift by whole samples, so the period must be an integer — and if  $\omega_0/2\pi$  is irrational, no integer works and the signal is **not periodic at all**, even though it looks like a sinusoid.

Example:  $\cos(n)$  has  $\omega_0/2\pi = 1/(2\pi)$ , irrational  $\Rightarrow$  **not periodic**.

## 12.4 Q7(d) — image tasks and signal properties

[7]

Suppose you have an image that you are using as an example of a discrete signal. Mention the specific signal properties useful for each of: image compression, blurring, sharpening, flipping, zooming in/out, determining image contrast, detecting edges.

### Solution

| Task                       | Property used                                                                                                                                                                                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Image compression          | <b>Energy compaction / Parseval's relation.</b> $\sum_n  x[n] ^2 = \frac{1}{N} \sum_k  X[k] ^2$ , and for natural images most of that energy sits in a few low-frequency bins. Keep those, discard the rest, and the error is small (this is exactly what JPEG does with the DCT). |
| Blurring                   | <b>Convolution with a lowpass kernel</b> (e.g. a Gaussian), equivalently <i>multiplication of spectra</i> : $x \circledast h \leftrightarrow X[k]H[k]$ with $H$ attenuating high $k$ .                                                                                             |
| Sharpening                 | <b>Convolution with a highpass kernel</b> , i.e. the same theorem with $H$ boosting high $k$ . (Unsharp masking = original + $\alpha \cdot$ highpass, which is <i>linearity</i> plus convolution.)                                                                                 |
| Flipping the image         | <b>Time (spatial) reversal:</b> $x[((-n))_N] \leftrightarrow X[((-k))_N]$ . Magnitude is unchanged; only the spectrum is mirrored.                                                                                                                                                 |
| Zooming in / out           | <b>Time (spatial) scaling</b> , i.e. interpolation (upsampling) or decimation (downsampling), which stretches or compresses the spectrum correspondingly. Downsampling must be preceded by lowpass filtering to avoid aliasing.                                                    |
| Determining image contrast | <b>Linearity / amplitude scaling</b> , together with the <b>dc component</b> $X[0]$ : $X[0]/N$ is the mean brightness, and the spread of the remaining $ X[k] $ measures contrast. Scaling $x$ by $a$ scales every $X[k]$ by $a$ .                                                 |
| Detecting edges            | <b>Differencing = highpass filtering = convolution.</b> An edge is a rapid change, i.e. high-frequency content; a difference kernel (Sobel, Laplacian) convolved with the image isolates it.                                                                                       |

**The pattern to state up front if you have time:** nearly every image operation is one of three things — a *convolution* (blur, sharpen, edges), an application of a *transform property* (flip, translate, scale), or *discarding small coefficients* (compression).



## Chapter 13

# Extra practice (written for this guide)

### Read this first

Everything in this chapter is **written for this guide** — these are *not* past questions. They exist to cover the parts of the syllabus that the two available past papers happen not to touch (hand DFTs, properties, Parseval, circular convolution, 8-point FFTs, bin-to-hertz conversion).

Chapters 11 and 12 are the real past questions. This chapter is drill.

### 13.1 E1 — An 8-point DFT of a rectangular pulse

Compute the 8-point DFT of  $x[n] = \{1, 1, 1, 1, 0, 0, 0, 0\}$ . Verify conjugate symmetry.

#### Solution

Use the geometric-series shortcut rather than eight separate dot products. Only  $n = 0, 1, 2, 3$  contribute, so

$$X[k] = \sum_{n=0}^3 W_8^{kn} = \frac{1 - W_8^{4k}}{1 - W_8^k} \quad (k \neq 0), \quad X[0] = 4. \quad (13.1)$$

Since  $W_8^4 = e^{-j\pi} = -1$ , we get  $W_8^{4k} = (-1)^k$ , so

$$X[k] = \begin{cases} 0, & k \text{ even, } k \neq 0 \\ \frac{2}{1 - W_8^k}, & k \text{ odd} \end{cases} \quad (13.2)$$

With  $W_8^1 = \frac{1-j}{\sqrt{2}}$ ,  $W_8^3 = -\frac{1+j}{\sqrt{2}}$ ,  $W_8^5 = -\frac{1-j}{\sqrt{2}}$ ,  $W_8^7 = \frac{1+j}{\sqrt{2}}$ :

| $k$ | $X[k]$ (exact)        | $X[k]$ (decimal) | $ X[k] $ |
|-----|-----------------------|------------------|----------|
| 0   | 4                     | 4                | 4        |
| 1   | $1 - (1 + \sqrt{2})j$ | $1 - 2.4142j$    | 2.6131   |
| 2   | 0                     | 0                | 0        |
| 3   | $1 - (\sqrt{2} - 1)j$ | $1 - 0.4142j$    | 1.0824   |
| 4   | 0                     | 0                | 0        |
| 5   | $1 + (\sqrt{2} - 1)j$ | $1 + 0.4142j$    | 1.0824   |
| 6   | 0                     | 0                | 0        |
| 7   | $1 + (1 + \sqrt{2})j$ | $1 + 2.4142j$    | 2.6131   |

**Checks.**  $X[0] = \sum x[n] = 4 \checkmark$   $X[4] = \sum (-1)^n x[n] = 1 - 1 + 1 - 1 = 0 \checkmark$   $X[7] = X^*[1]$ ,  $X[5] = X^*[3] \checkmark$  Parseval:  $\sum |x|^2 = 4; \frac{1}{8}(16 + 6.828 + 0 + 1.172 + 0 + 1.172 + 0 + 6.828) = \frac{32}{8} = 4 \checkmark$

**Why the even bins (except 0) vanish.** The signal is exactly half a period of a square wave over  $N = 8$ . Its periodic extension is a square wave of period 8, and a square wave has no even harmonics. The DFT reports that directly.

## 13.2 E2 — Using the properties

Given  $x = \{1, 2, 3, 4\}$  with  $X = \{10, -2 + 2j, -2, -2 - 2j\}$ , find the DFT of each of the following *without* computing a new DFT from scratch: (a)  $y_1 = \{3, 4, 1, 2\}$  (b)  $y_2 = \{1, -2, 3, -4\}$  (c)  $y_3 = \{1, 4, 3, 2\}$  (d)  $y_4 = x^*[n] = x[n]$ .

### Solution

(a)  $y_1[n] = x[((n-2))_N]$  — a circular shift by  $m = 2$ . So  $Y_1[k] = W_4^{2k} X[k] = (-1)^k X[k]$ :

$$Y_1 = \{10, 2 - 2j, -2, 2 + 2j\}$$

Check:  $|Y_1| = |X| \checkmark$ , and  $Y_1[0] = 3 + 4 + 1 + 2 = 10 \checkmark$

(b)  $y_2[n] = (-1)^n x[n]$  — modulation by  $k_0 = N/2 = 2$ . So  $Y_2[k] = X[((k-2))_N]$ , i.e. rotate the bin list by 2:

$$Y_2 = \{X[2], X[3], X[0], X[1]\} = \{-2, -2 - 2j, 10, -2 + 2j\}$$

Check:  $Y_2[0] = 1 - 2 + 3 - 4 = -2 \checkmark$  The dc energy moved to bin 2: the signal is now highpass.

(c)  $y_3[n] = x[(-n)_N]$  — time reversal ( $x[0]$  fixed,  $x[1] \leftrightarrow x[3]$ ). So  $Y_3[k] = X[((-k))_N] = \{X[0], X[3], X[2], X[1]\}$ :

$$Y_3 = \{10, -2 - 2j, -2, -2 + 2j\}$$

which is also  $X^*[k]$  — as it must be, since reversing a *real* signal conjugates its spectrum.

(d)  $x$  is real, so  $x^* = x$  and the DFT is unchanged:  $Y_4 = X$ . Applying the conjugation property instead gives  $X^*[((-k))_N]$ , and setting these equal recovers exactly the conjugate symmetry relation  $X[k] = X^*[N - k]$ .

### 13.3 E3 — Circular versus linear convolution

$x = \{1, 2, 0, -1\}$  and  $h = \{1, 1, 1\}$ . (a) Compute the linear convolution. (b) Compute the 4-point circular convolution. (c) How many outputs are corrupted, and which? Show where the corruption came from. (d) What is the smallest  $N$  for which circular = linear? Verify.

#### Solution

**(a) Linear.**  $L = 4, M = 3$ , so length  $= 4 + 3 - 1 = 6$ . Slide  $h$  over  $x$  (each output is the sum of three consecutive  $x$  values, with zeros outside):

$$\begin{array}{lll} y[0] = 1 & y[1] = 1 + 2 = 3 & y[2] = 1 + 2 + 0 = 3 \\ y[3] = 2 + 0 - 1 = 1 & y[4] = 0 - 1 = -1 & y[5] = -1 \end{array}$$

$$x * h = \{1, 3, 3, 1, -1, -1\}$$

**(b) Circular,  $N = 4$ .** Fold the linear result: add  $y[4]$  onto index 0 and  $y[5]$  onto index 1:

$$x \circledast h = \{1 + (-1), 3 + (-1), 3, 1\} = \{0, 2, 3, 1\}$$

*Direct check with the definition,  $y[0] = \sum_m x[m]h[(-m)]_N$  with  $h$  padded to  $\{1, 1, 1, 0\}$ :*  $y[0] = x[0]h[0] + x[1]h[3] + x[2]h[2] + x[3]h[1] = 1 + 0 + 0 - 1 = 0 \checkmark$

**(c) Corrupted count**  $= \max(0, L + M - 1 - N) = \max(0, 6 - 4) = 2$ , at  $n = 0$  and  $n = 1$ . They came from the linear tail  $y[4] = -1$  and  $y[5] = -1$  wrapping back onto the front.

**(d) Need  $N \geq L + M - 1 = 6$ , so  $N_{\min} = 6$ .** Pad  $x \rightarrow \{1, 2, 0, -1, 0, 0\}$  and  $h \rightarrow \{1, 1, 1, 0, 0, 0\}$ ; then  $x \circledast_6 h = \{1, 3, 3, 1, -1, -1\}$  — exactly the linear result.  $\checkmark$

**Global check that works every time:**  $\sum_n y[n] = (\sum_n x[n])(\sum_n h[n])$ , because  $Y[0] = X[0]H[0]$ . Here  $\sum x = 2$ ,  $\sum h = 3$ , so  $\sum y$  should be 6. Linear:  $1 + 3 + 3 + 1 - 1 - 1 = 6 \checkmark$  Circular:  $0 + 2 + 3 + 1 = 6 \checkmark$  (Note the *total* is preserved even when individual values are corrupted — folding moves energy around, it does not destroy it.)

### 13.4 E4 — Bins to hertz

A signal is sampled at  $f_s = 16$  kHz and a 64-point DFT is taken. (a) What is the frequency resolution? (b) Which bin does a 2.5 kHz tone land in? (c) What frequency does bin 50 represent? (d) A 9 kHz tone is present. Which bin does it appear in, and why is that a problem? (e) You need 50 Hz resolution. What must change?

#### Solution

**(a)**  $\Delta f = \frac{f_s}{N} = \frac{16000}{64} = 250$  Hz per bin. (Equivalently the recording lasts  $T_0 = NT_s = 64/16000 = 4$  ms, and  $1/T_0 = 250$  Hz.)

**(b)**  $k = \frac{2500}{250} = 10$ . ( $10 < N/2 = 32$ , so it is a positive frequency — fine.)

**(c)**  $50 > N/2 = 32$ , so bin 50 is a *negative* frequency:

$$f_{50} = (50 - 64) \times 250 = -14 \times 250 = -3500 \text{ Hz.}$$

Bin 50 is the negative-frequency partner of bin 14 (= 3.5 kHz).

(d)  $f_s/2 = 8$  kHz, and  $9 > 8$ , so the tone violates the sampling theorem and **aliases**. Fold it:  $9 - 16 = -7$  kHz. So it appears at  $-7$  kHz, i.e.

$$k = \frac{-7000}{250} + 64 = -28 + 64 = 36.$$

The problem: bin 36 is also where a genuine  $-7$  kHz component would appear. Once sampled, the two are **indistinguishable** — no amount of processing can separate them. The fix is an analogue anti-alias lowpass filter *before* the ADC.

(e)  $\Delta f = 1/T_0$ , so 50 Hz needs  $T_0 = 20$  ms, i.e.  $N = T_0 f_s = 0.020 \times 16000 = 320$  samples. You must **record for longer**. Zero-padding the existing 64 samples to 320 would plot 320 points but would *not* improve the resolution — the underlying DTFT was already fixed by the 64 real samples.

### 13.5 E5 — Parseval and the power spectrum

$x = \{2, -1, 0, 1\}$ . (a) Compute  $X[k]$ . (b) Verify Parseval. (c) Which bin holds the most energy, and what fraction of the total is it?

#### Solution

(a)  $N = 4$ ,  $W_4^{kn} = (-j)^{kn}$ :

$$X[0] = 2 - 1 + 0 + 1 = 2$$

$$X[1] = 2 + (-1)(-j) + 0(-1) + 1(j) = 2 + j + j = 2 + 2j$$

$$X[2] = 2 + (-1)(-1) + 0(1) + 1(-1) = 2 + 1 - 1 = 2$$

$$X[3] = 2 + (-1)(j) + 0(-1) + 1(-j) = 2 - j - j = 2 - 2j$$

Check:  $X[3] = X^*[1] \checkmark$ ,  $X[0]$  and  $X[2]$  real  $\checkmark$

(b) Time side:  $4 + 1 + 0 + 1 = 6$ .

Bin side:  $\frac{1}{4}(|2|^2 + |2 + 2j|^2 + |2|^2 + |2 - 2j|^2) = \frac{1}{4}(4 + 8 + 4 + 8) = \frac{24}{4} = 6 \checkmark$

(c) Energy in bin  $k$  is  $|X[k]|^2/N$ :

$$\left\{\frac{4}{4}, \frac{8}{4}, \frac{4}{4}, \frac{8}{4}\right\} = \{1, 2, 1, 2\} \quad (\text{total } 6 \checkmark)$$

Bins 1 and 3 tie for the most, each holding  $2/6 = 1/3$  of the total energy. They are a conjugate pair — one real sinusoidal component at  $\omega = \pi/2$ , split across its positive and negative frequency bins.

### 13.6 E6 — Filling in a spectrum from conjugate symmetry

A real 8-point sequence has  $X[0] = 6$ ,  $X[1] = 1 - 2j$ ,  $X[2] = -3j$ ,  $X[3] = 2 + j$ ,  $X[4] = -1$ . Write down  $X[5]$ ,  $X[6]$ ,  $X[7]$ , and state  $x[0]$  and  $\sum_n x[n]$ .

#### Solution

Real  $x \Rightarrow X[N - k] = X^*[k]$  with  $N = 8$ :

$$X[5] = X^*[3] = 2 - j$$

$$X[6] = X^*[2] = +3j$$

$$X[7] = X^*[1] = 1 + 2j$$

*Consistency:*  $X[0] = 6$  and  $X[4] = X[N/2] = -1$  are both real, as required. ✓

$$\sum_n x[n] = X[0] = 6.$$

$x[0] = \frac{1}{N} \sum_k X[k]$ . The imaginary parts cancel in conjugate pairs, so

$$x[0] = \frac{1}{8} (6 + 2 \operatorname{Re}\{1 - 2j\} + 2 \operatorname{Re}\{-3j\} + 2 \operatorname{Re}\{2 + j\} + (-1)) = \frac{1}{8} (6 + 2 + 0 + 4 - 1) = \frac{11}{8}.$$

### 13.7 E7 — Prove the modulation property

Prove from the analysis sum that  $W_N^{-k_0 n} x[n] \longleftrightarrow X[((k - k_0))_N]$ , and state clearly where periodicity is used.

#### Solution

**Step 1 — substitute.** Let  $y[n] = W_N^{-k_0 n} x[n]$ . Then

$$Y[k] = \sum_{n=0}^{N-1} y[n] W_N^{kn} = \sum_{n=0}^{N-1} W_N^{-k_0 n} x[n] W_N^{kn}. \quad (13.3)$$

**Step 2 — combine the exponentials.** Powers of the same base add:

$$W_N^{-k_0 n} W_N^{kn} = W_N^{(k-k_0)n}, \quad \text{so} \quad Y[k] = \sum_{n=0}^{N-1} x[n] W_N^{(k-k_0)n}. \quad (13.4)$$

**Step 3 — recognise it.** That sum is the definition of  $X$  evaluated at frequency index  $k - k_0$ . But  $k - k_0$  may lie outside  $\{0, \dots, N-1\}$ , and  $X$  is only *defined* there. This is where periodicity is used: we proved  $X[k + N] = X[k]$ , so  $X$  extends  $N$ -periodically to all integers, and

$$Y[k] = X[k - k_0] = X[((k - k_0))_N]. \quad \blacksquare \quad (13.5)$$

**Sanity check.** Put  $x[n] = 1$  for all  $n$ , so  $X[k] = N\delta[k]$  (all energy at dc). Then  $y[n] = W_N^{-k_0 n} = e^{j\frac{2\pi}{N}k_0 n}$ , a pure tone at bin  $k_0$ , and the property predicts  $Y[k] = N\delta[k - k_0]$  — exactly one bin lit, at  $k_0$ . ✓

### 13.8 E8 — An 8-point radix-2 DIT FFT by hand

Compute the 8-point DIT FFT of  $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ , showing all three stages.

#### Solution

**Bit-reversed input order** (3 bits):  $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$ , i.e.

$$\text{array} = [1, 0, 1, 0, 1, 0, 1, 0]$$

**Stage 1** — four 2-point DFTs on adjacent pairs, twiddle  $W_2^0 = 1$ . Every pair is  $(1, 0)$ , giving  $(1 + 0, 1 - 0) = (1, 1)$ :

$$[1, 1, 1, 1, 1, 1, 1, 1]$$

**Stage 2** — two 4-point blocks, twiddles  $W_4^0 = 1, W_4^1 = -j$ . Take the first block  $(a_0, a_1, a_2, a_3) = (1, 1, 1, 1)$ :

$$\begin{aligned} k=0: \quad g &= a_0 = 1, \quad h = W_4^0 a_2 = 1 & \Rightarrow a_0 \leftarrow 2, \quad a_2 \leftarrow 0 \\ k=1: \quad g &= a_1 = 1, \quad h = W_4^1 a_3 = -j & \Rightarrow a_1 \leftarrow 1-j, \quad a_3 \leftarrow 1+j \end{aligned}$$

Both blocks are identical, so

$$[2, 1-j, 0, 1+j, \quad 2, 1-j, 0, 1+j]$$

**Stage 3** — one 8-point block. Here  $G = (2, 1-j, 0, 1+j)$  and  $H = (2, 1-j, 0, 1+j)$ , with twiddles

$$W_8^0 = 1, \quad W_8^1 = \frac{1-j}{\sqrt{2}}, \quad W_8^2 = -j, \quad W_8^3 = -\frac{1+j}{\sqrt{2}}.$$

Compute  $W_8^k H[k]$  first:

$$\begin{aligned} k=0: \quad W_8^0 H[0] &= 2 \\ k=1: \quad W_8^1 H[1] &= \frac{1-j}{\sqrt{2}}(1-j) = \frac{(1-j)^2}{\sqrt{2}} = \frac{-2j}{\sqrt{2}} = -\sqrt{2}j \\ k=2: \quad W_8^2 H[2] &= (-j)(0) = 0 \\ k=3: \quad W_8^3 H[3] &= -\frac{1+j}{\sqrt{2}}(1+j) = -\frac{(1+j)^2}{\sqrt{2}} = -\frac{2j}{\sqrt{2}} = -\sqrt{2}j \end{aligned}$$

Then  $X[k] = G[k] + W_8^k H[k]$  and  $X[k+4] = G[k] - W_8^k H[k]$ :

$$\begin{aligned} X[0] &= 2 + 2 = 4 & X[4] &= 2 - 2 = 0 \\ X[1] &= (1-j) - \sqrt{2}j = 1 - (1 + \sqrt{2})j & X[5] &= (1-j) + \sqrt{2}j = 1 + (\sqrt{2} - 1)j \\ X[2] &= 0 + 0 = 0 & X[6] &= 0 - 0 = 0 \\ X[3] &= (1+j) - \sqrt{2}j = 1 - (\sqrt{2} - 1)j & X[7] &= (1+j) + \sqrt{2}j = 1 + (1 + \sqrt{2})j \end{aligned}$$

Identical to the direct computation in E1. ✓

**Operation count for this run:**  $\frac{N}{2} \log_2 N = \frac{8}{2} \cdot 3 = 12$  complex multiplications, versus  $N^2 = 64$  for the direct sum. And notice how many of the 12 were trivial ( $\times 1$  or  $\times 0$ ) — in a hand computation most of the work is bookkeeping, not arithmetic.

## 13.9 E9 — The same signal by DIF

Redo E8 using the 8-point radix-2 DIF FFT and confirm the outputs emerge bit-reversed.

### Solution

Input in **natural** order:  $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ .

**Stage 1** — for  $n = 0, \dots, 3$ :

$$\begin{aligned} a[n] &= x[n] + x[n+4] = \{1+0, 1+0, 1+0, 1+0\} = \{1, 1, 1, 1\} \\ b[n] &= (x[n] - x[n+4])W_8^n = \{1 \cdot W_8^0, 1 \cdot W_8^1, 1 \cdot W_8^2, 1 \cdot W_8^3\} \\ &= \left\{1, \frac{1-j}{\sqrt{2}}, -j, -\frac{1+j}{\sqrt{2}}\right\} \end{aligned}$$

**Stage 2** — 4-point DFTs of  $a$  and of  $b$ .

For  $a = \{1, 1, 1, 1\}$  (constant):  $\text{DFT}_4\{a\} = \{4, 0, 0, 0\}$ . These are the **even** bins:

$$X[0] = 4, \quad X[2] = 0, \quad X[4] = 0, \quad X[6] = 0.$$

For  $b$ : with  $W_4^{kn} = (-j)^{kn}$ ,

$$B[0] = 1 + \frac{1-j}{\sqrt{2}} - j - \frac{1+j}{\sqrt{2}} = 1 - j + \frac{(1-j)-(1+j)}{\sqrt{2}} = 1 - j - \sqrt{2}j = 1 - (1 + \sqrt{2})j$$

$$B[1] = 1 + \frac{1-j}{\sqrt{2}}(-j) + (-j)(-1) + \left(-\frac{1+j}{\sqrt{2}}\right)(j) = 1 + j - \sqrt{2}j = 1 - (\sqrt{2} - 1)j$$

$$B[2] = 1 - \frac{1-j}{\sqrt{2}} + (-j)(1) - \left(-\frac{1+j}{\sqrt{2}}\right) = 1 - j + \frac{(1+j)-(1-j)}{\sqrt{2}} = 1 - j + \sqrt{2}j = 1 + (\sqrt{2} - 1)j$$

$$B[3] = 1 + j + \sqrt{2}j = 1 + (1 + \sqrt{2})j$$

These are the **odd** bins:  $X[1] = B[0]$ ,  $X[3] = B[1]$ ,  $X[5] = B[2]$ ,  $X[7] = B[3]$ .

#### Output order

The algorithm produced, in array positions 0..7:

$$\underbrace{X[0], X[2], X[4], X[6]}_{\text{from } a}, \underbrace{X[1], X[3], X[5], X[7]}_{\text{from } b}$$

which is exactly **bit-reversed** order. Un-shuffling gives the same  $X$  as E8. ✓

### 13.10 E10 — Bit reversal for $N = 16$

Give the bit-reversed input order for a 16-point radix-2 DIT FFT.

#### Solution

$\log_2 16 = 4$ , so use **4 bits with leading zeros**.

| $n$ | binary | reversed | new | $n$ | binary | reversed | new |
|-----|--------|----------|-----|-----|--------|----------|-----|
| 0   | 0000   | 0000     | 0   | 8   | 1000   | 0001     | 1   |
| 1   | 0001   | 1000     | 8   | 9   | 1001   | 1001     | 9   |
| 2   | 0010   | 0100     | 4   | 10  | 1010   | 0101     | 5   |
| 3   | 0011   | 1100     | 12  | 11  | 1011   | 1101     | 13  |
| 4   | 0100   | 0010     | 2   | 12  | 1100   | 0011     | 3   |
| 5   | 0101   | 1010     | 10  | 13  | 1101   | 1011     | 11  |
| 6   | 0110   | 0110     | 6   | 14  | 1110   | 0111     | 7   |
| 7   | 0111   | 1110     | 14  | 15  | 1111   | 1111     | 15  |

**Input order:**

$$x[0], x[8], x[4], x[12], x[2], x[10], x[6], x[14], x[1], x[9], x[5], x[13], x[3], x[11], x[7], x[15]$$

#### Two shortcuts.

1. The first half of the  $N = 16$  order is the  $N = 8$  order with every index doubled ( $0, 4, 2, 6, 1, 5, 3, 7 \rightarrow 0, 8, 4, 12, 2, 10, 6, 14$ ); the second half is the first half + 1.
2. Only *swaps* are needed:  $1 \leftrightarrow 8, 2 \leftrightarrow 4, 3 \leftrightarrow 12, 5 \leftrightarrow 10, 7 \leftrightarrow 14, 11 \leftrightarrow 13$ . The rest ( $0, 6, 9, 15$ ) are binary palindromes and stay put.

### 13.11 E11 — Three kinds of zero-padding

You have  $x[n]$  of length  $L = 100$  and a filter  $h[n]$  of length  $M = 29$ , sampled at  $f_s = 8$  kHz. For each goal, state the padded length and what it buys you. (a) Plot the spectrum of  $x$  more smoothly. (b) Compute the LTI output  $y = x * h$  by multiplying DFTs. (c) Do (b) with a radix-2 FFT. (d) Improve the frequency resolution of the spectrum of  $x$  to 20 Hz.

#### Solution

(a) Any  $M' > 100$  you like — e.g. 512 or 1024. The DTFT is unchanged; you are sampling the same curve at more points. *Buys*: a smoother-looking plot. *Does not buy*: any new information, and  $\Delta f$  (as a measure of what you can *resolve*) is still governed by the 100 real samples.

(b)  $N \geq L + M - 1 = 100 + 29 - 1 = 128$ . Pad *both*  $x$  and  $h$  to  $N$ . *Buys*: correctness — circular convolution now equals linear convolution, so no output sample is corrupted by wrap-around.

(c)  $N$  must be a power of two *and*  $\geq 128$ . Here  $128 = 2^7$  already, so  $N = 128$  works for both requirements. (If  $L + M - 1$  had been, say, 130, you would take  $N = 256$ .)

(d)  $\Delta f = 1/T_0 = 20$  Hz requires  $T_0 = 50$  ms, i.e.  $N = T_0 f_s = 0.050 \times 8000 = 400$  real samples. **Padding cannot do this.** You must acquire 400 actual samples — record for 50 ms instead of 12.5 ms.

#### The distinction to state in an exam

Zero-padding changes how *densely* you sample a spectrum. Recording longer changes *what the spectrum is*. Only the second improves resolution.

### 13.12 E12 — DTFT, and the bridge to the DFT

$x = \{1, -1, 1, -1\}$ . (a) Write the DTFT  $X(e^{j\omega})$  in closed form. (b) Evaluate it at  $\omega = \frac{2\pi k}{4}$  for  $k = 0, 1, 2, 3$  and confirm you get the 4-point DFT. (c) At which digital frequency is the energy concentrated, and what does that mean?

#### Solution

(a)

$$X(e^{j\omega}) = \sum_{n=0}^3 x[n]e^{-j\omega n} = 1 - e^{-j\omega} + e^{-j2\omega} - e^{-j3\omega}. \quad (13.6)$$

This is a geometric series with ratio  $-e^{-j\omega}$ :

$$X(e^{j\omega}) = \frac{1 - (-e^{-j\omega})^4}{1 + e^{-j\omega}} = \frac{1 - e^{-j4\omega}}{1 + e^{-j\omega}}. \quad (13.7)$$

(b) Substitute  $\omega_k = \frac{2\pi k}{4} = \frac{\pi k}{2}$ , so  $e^{-j\omega_k} = (-j)^k$ :

$$\begin{aligned} k=0: 1 - 1 + 1 - 1 &= 0 & k=1: 1 - (-j) + (-j)^2 - (-j)^3 &= 1 + j - 1 - j = 0 \\ k=2: 1 - (-1) + 1 - (-1) &= 4 & k=3: 1 - (j) + (j)^2 - (j)^3 &= 1 - j - 1 + j = 0 \end{aligned}$$

So  $X[k] = \{0, 0, 4, 0\}$  — and this is indeed the 4-point DFT of  $\{1, -1, 1, -1\}$ . ✓

(The closed form is 0/0 at these points except  $k = 2$ ; take the limit, or just use the sum directly, which is what we did.)



(c) All the energy sits in bin  $k = 2 = N/2$ , i.e. at  $\omega = \pi$  radians per sample. That is the **highest representable digital frequency**: the sequence flips sign every sample. Physically,  $\omega = \pi$  corresponds to  $f = f_s/2$ , half the sampling rate — the Nyquist frequency. There is literally nothing faster a sampled signal can do.

### 13.13 E13 — Choosing a transform size

A processor performs  $10^8$  complex multiplications per second. You must transform  $M = 5000$  samples. (a) How long does a direct DFT take? (b) How long does a radix-2 FFT take (after zero-padding)? (c) The result must be ready in 1 ms. Which is feasible?

#### Solution

(a)  $M^2 = 5000^2 = 2.5 \times 10^7$  multiplications  $\Rightarrow \frac{2.5 \times 10^7}{10^8} = 0.25$  s.

(b)  $\lceil \log_2 5000 \rceil = 13$  (since  $2^{12} = 4096 < 5000 \leq 8192 = 2^{13}$ ), so pad to  $N = 8192$ . Then

$$\frac{N}{2} \log_2 N = \frac{8192}{2} \times 13 = 4096 \times 13 = 53,248 \text{ multiplications} \Rightarrow 0.53 \text{ ms.} \quad (13.8)$$

(c) The direct DFT takes 250 ms —  $250 \times$  too slow. The FFT takes 0.53 ms, comfortably inside the budget. **Only the FFT is feasible.**

**The counter-intuitive bit worth noticing:** the FFT transforms *more* data (8192 points instead of 5000) and is still about  $470 \times$  faster. Increasing  $N$  to reach a power of two is essentially always worth it.

### 13.14 E14 — Filtering via the DFT, end to end

$x = \{1, 2, 1\}$ ,  $h = \{1, 0, -1\}$  (a simple edge detector). (a) Compute  $y = x * h$  directly. (b) Compute the 4-point circular convolution and explain the difference. (c) Carry out the full DFT filtering recipe with the correct  $N$  and verify.

#### Solution

(a)  $L = M = 3$ , so length = 5:

$$y[n] = \{1, 2, 0, -2, -1\}$$

(Check:  $\sum y = 0 = (\sum x)(\sum h) = 4 \times 0$  ✓)

(b) With  $N = 4$ , pad both to  $\{1, 2, 1, 0\}$  and  $\{1, 0, -1, 0\}$ . Fold the linear result:  $y[4] = -1$  wraps onto  $n = 0$ :

$$x \circledast_4 h = \{1 + (-1), 2, 0, -2\} = \{0, 2, 0, -2\}$$

Corrupted count =  $\max(0, 3 + 3 - 1 - 4) = 1$ , at  $n = 0$  — and indeed  $y[0]$  changed from 1 to 0.

(c) Required  $N \geq L + M - 1 = 5$ ; the next power of two is  $N = 8$ . Pad  $x \rightarrow \{1, 2, 1, 0, 0, 0, 0, 0\}$ ,  $h \rightarrow \{1, 0, -1, 0, 0, 0, 0, 0\}$ .

Rather than grinding through eight complex products, verify the theorem at a couple of bins.

$$X[k] = 1 + 2W_8^k + W_8^{2k}, \quad H[k] = 1 - W_8^{2k}$$

- $k = 0$ :  $X[0] = 4$ ,  $H[0] = 0$ , so  $Y[0] = 0$ . And  $\sum_n y[n] = 0$ . ✓

- $k = 4$ :  $W_8^4 = -1$ ,  $W_8^8 = 1$ .  $X[4] = 1 - 2 + 1 = 0$ ,  $H[4] = 1 - 1 = 0$ , so  $Y[4] = 0$ . And  $\sum_n (-1)^n y[n] = 1 - 2 + 0 + 2 - 1 = 0$ . ✓

Inverting  $Y[k] = X[k]H[k]$  returns  $\{1, 2, 0, -2, -1, 0, 0, 0\}$ : the linear convolution in the first  $L + M - 1 = 5$  slots, zeros after. ✓

**Why  $H[0] = 0$  makes sense.**  $h = \{1, 0, -1\}$  is a *difference* operator, and a difference of a constant is zero.  $H[0] = \sum_n h[n] = 0$  says exactly that: the filter kills dc. That is what makes it an edge detector — flat regions vanish, transitions survive.

### 13.15 E15 — Reading a signal off its spectrum

For each 8-point DFT, describe  $x[n]$  as fully as you can without computing an IDFT.

- $X = \{8, 0, 0, 0, 0, 0, 0, 0\}$
- $X = \{0, 0, 4, 0, 0, 0, 4, 0\}$
- $|X| = \{1, 1, 1, 1, 1, 1, 1, 1\}$  with  $\angle X[k] = -\frac{2\pi}{8} \cdot 3k$
- $X = \{5, 2 + j, -1, 0, 3, 0, -1, 2 - j\}$

#### Solution

(a) All energy at dc.  $x[n] = \frac{1}{N} X[0] = \frac{8}{8} = 1$  for all  $n$ : a **constant signal**,  $x = \{1, 1, 1, 1, 1, 1, 1, 1\}$ .

(b) Exactly two bins,  $k = 2$  and  $k = 6 = N - 2$ , equal and real — a conjugate pair. That is a **real cosine** at bin 2:

$$x[n] = \frac{1}{8} \left( 4e^{j\frac{2\pi}{8}2n} + 4e^{-j\frac{2\pi}{8}2n} \right) = \cos\left(\frac{\pi n}{2}\right) = \{1, 0, -1, 0, 1, 0, -1, 0\}.$$

(General rule:  $\frac{N}{2}$  at bins  $k_0$  and  $N - k_0 \Rightarrow \cos(\frac{2\pi k_0}{N}n)$ .)

(c) Flat magnitude  $\Rightarrow$  an impulse. Linear phase with slope  $-\frac{2\pi}{N} \cdot 3 \Rightarrow$  shifted by  $m = 3$  (compare the shift property,  $W_N^{km} X[k]$ ). So

$$x[n] = \delta[n - 3] = \{0, 0, 0, 1, 0, 0, 0, 0\}.$$

(d) Check the symmetry:  $X[7] = 2 - j = X^*[1]$  ✓,  $X[6] = -1 = X^*[2]$  ✓,  $X[5] = 0 = X^*[3]$  ✓, and  $X[0] = 5$ ,  $X[4] = 3$  both real ✓. So  $x[n]$  is **real**. It is not even (the spectrum is not purely real) and not odd. Its mean is  $X[0]/N = 5/8$ , and its total energy is  $\frac{1}{8} \sum_k |X[k]|^2 = \frac{1}{8}(25 + 5 + 1 + 0 + 9 + 0 + 1 + 5) = \frac{46}{8} = 5.75$ .